



Вычисления на видеокартах



Лекция 4

- Транспонирование матрицы
- Умножение матриц
- Tensor Cores, WMMA
- Метод Штрассена
- Матрица матрицу
матрицево матрицит
матрицей в матрице

TENSOR CORES

$$A \times B = C$$



polarnick239@gmail.com



Николай Полярный

Activate Ubuntu
Go to Settings to activate Ubuntu.

План лекции

- 1) **Транспонирование матрицы:** через локальную память (учет *bank conflicts*)
- 2) **Умножение матриц:** через локальную память
- 3) **Умножение матриц:** *Tensor Cores*, *WMMA* (и `VK_KHR_cooperative_matrix`)
- 4) **Умножение матриц:** оптимизации *DeepSeek*
- 5) **Умножение матриц:** метод *Штрассена*





Task03: Умножение матриц на Tesla V100 (14130 GFlops)
🥇 Роберт Смайт - СПбГУ - 11720 GFlops, CUDA+WMMA
🥈 Mikhail Stulov - МФТИ, ТБанк - 9768 GFlops, OpenCL
🥉 Хулиган Серега - ИТМО - 7234 GFlops, OpenCL

Глава 1: Транспонирование

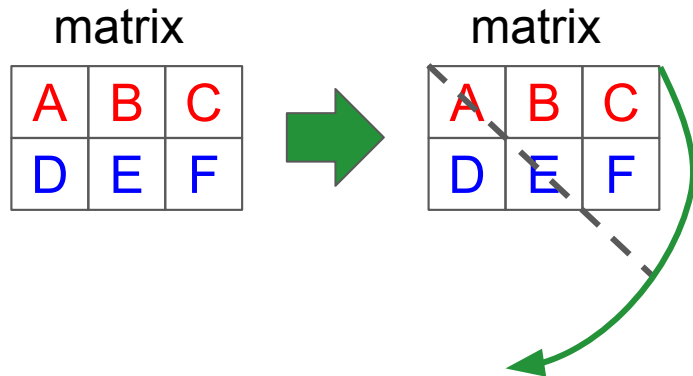
local memory, coalesced access, bank conflicts

Транспонирование матрицы

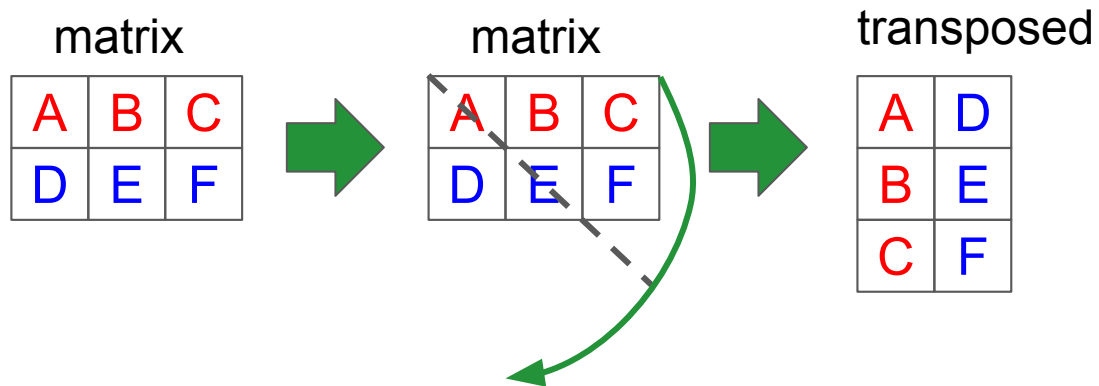
matrix

A	B	C
D	E	F

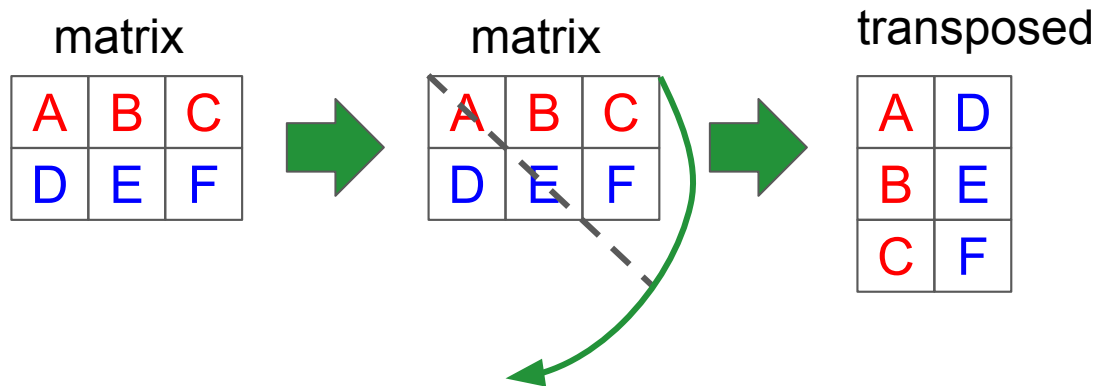
Транспонирование матрицы



Транспонирование матрицы

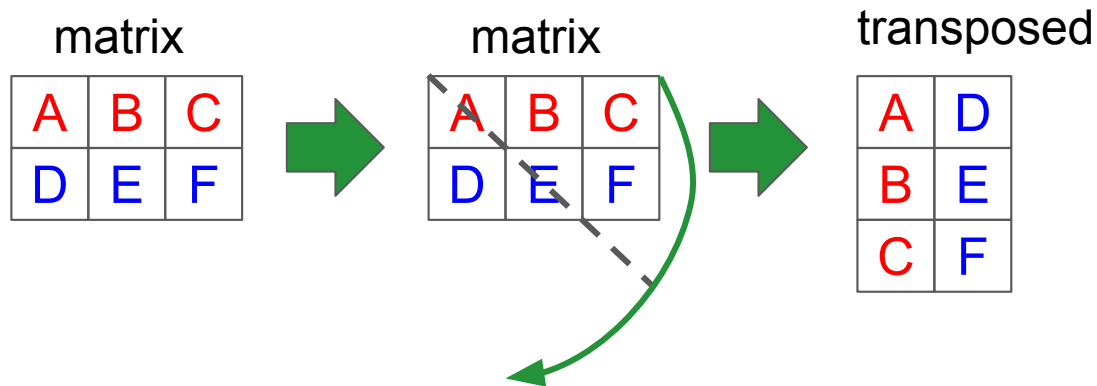


Транспонирование матрицы



Как выглядит:
- WorkRange?

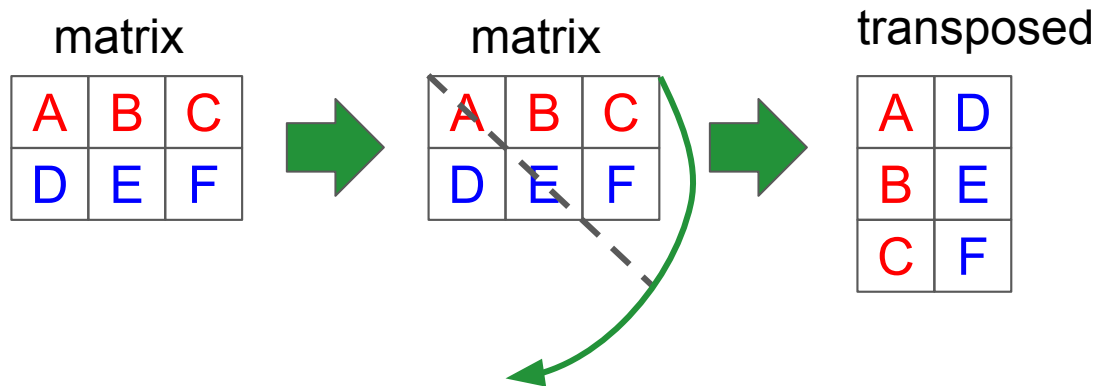
Транспонирование матрицы



Как выглядит:

- WorkRange?
- Задача WorkItem?

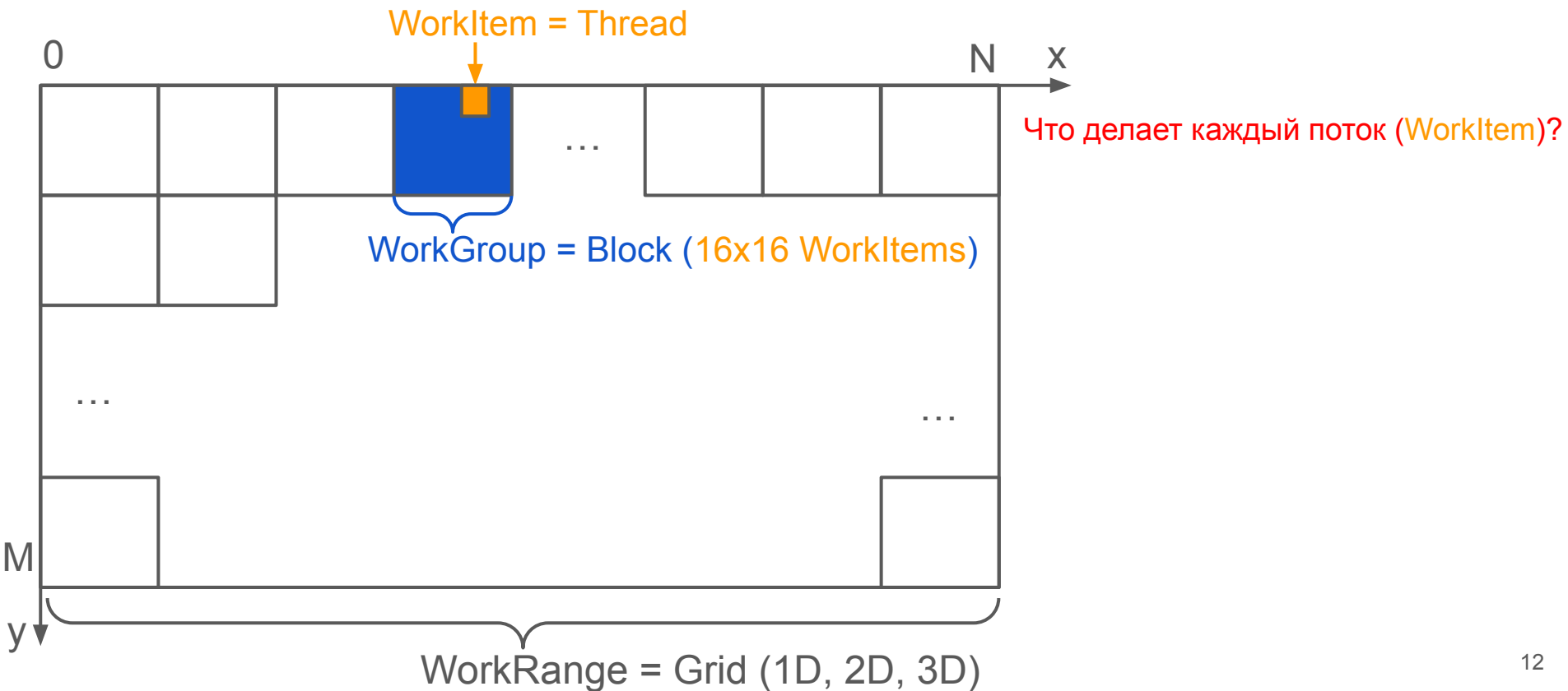
Транспонирование матрицы



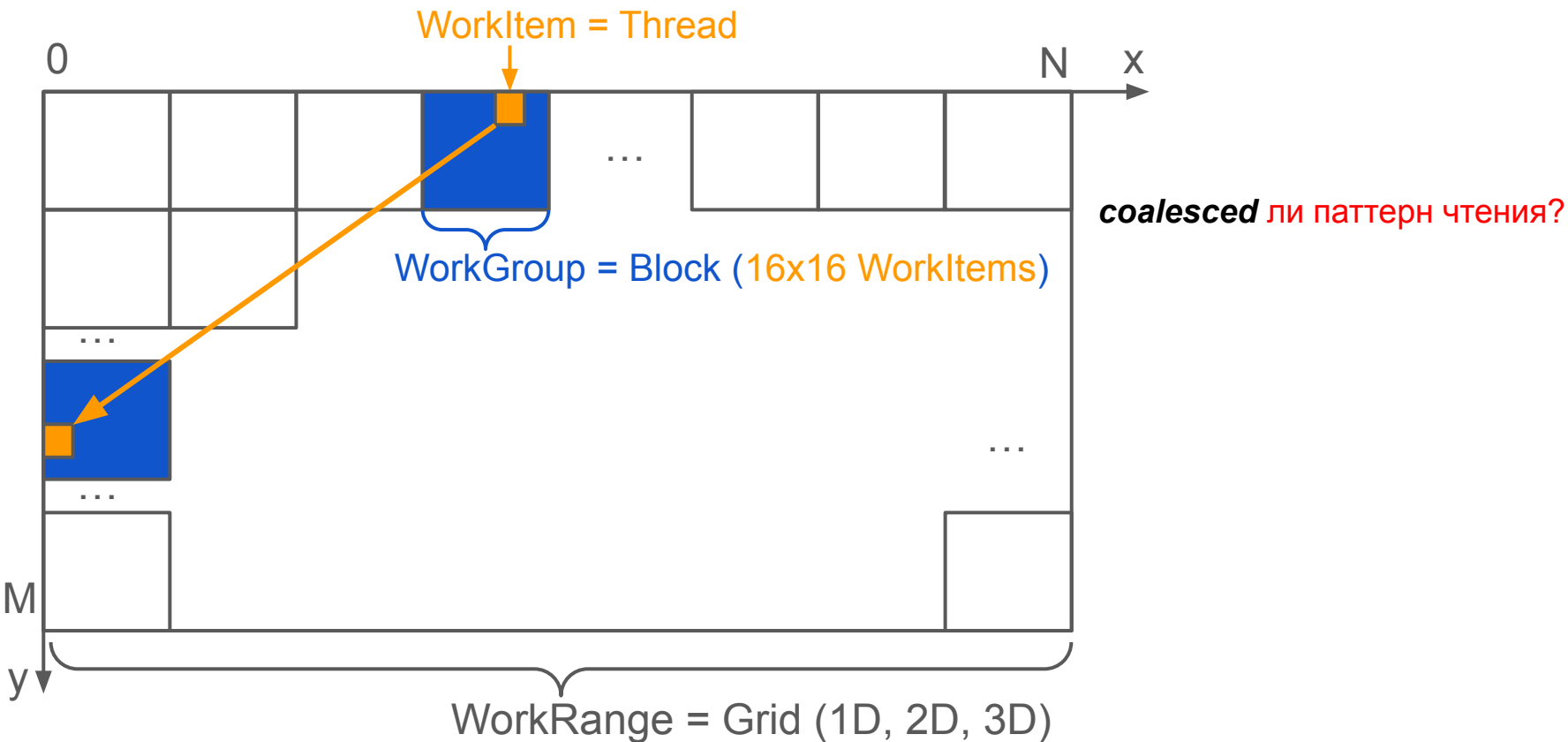
Как выглядит:

- WorkRange?
- Задача WorkItem?
- WorkGroup?

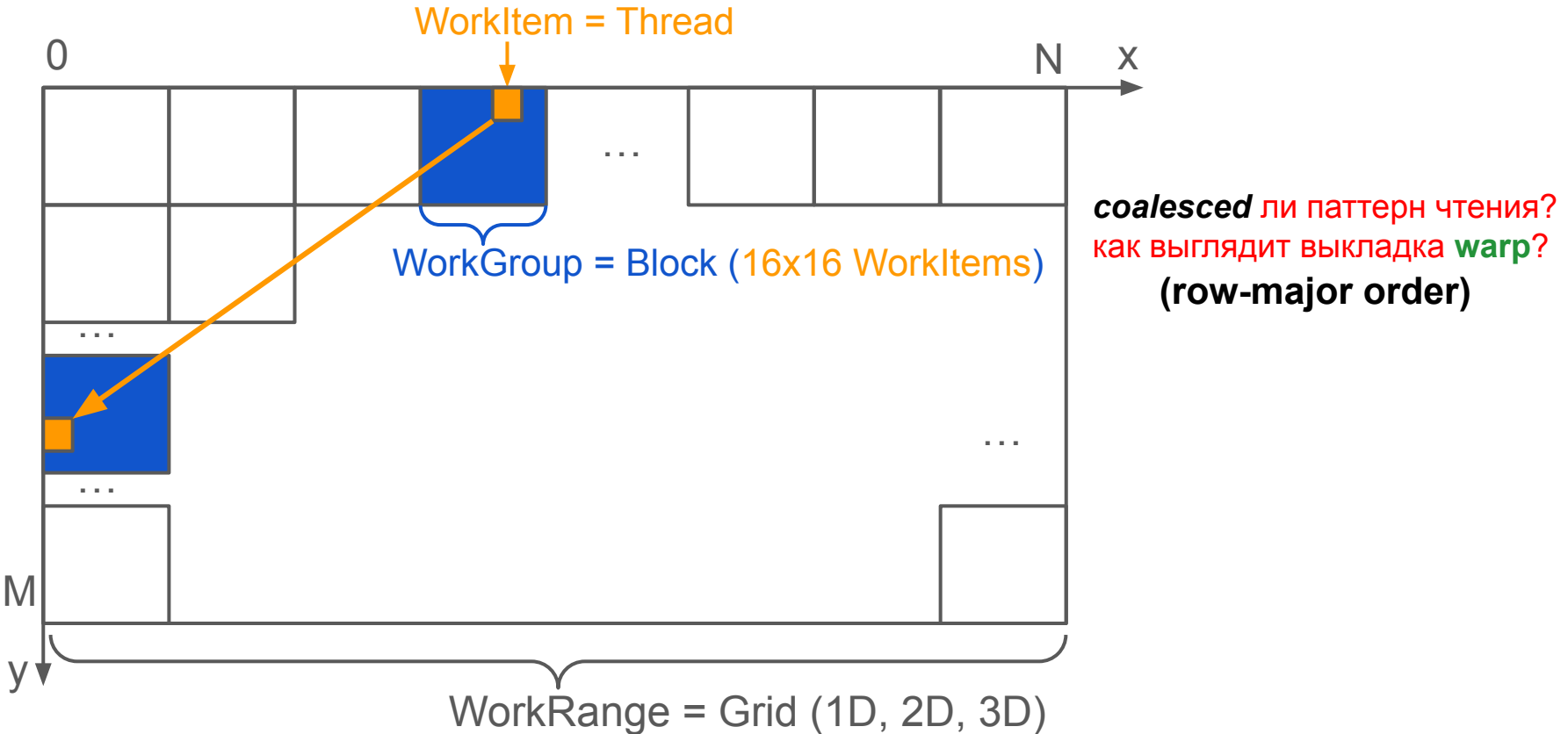
Транспонирование матрицы



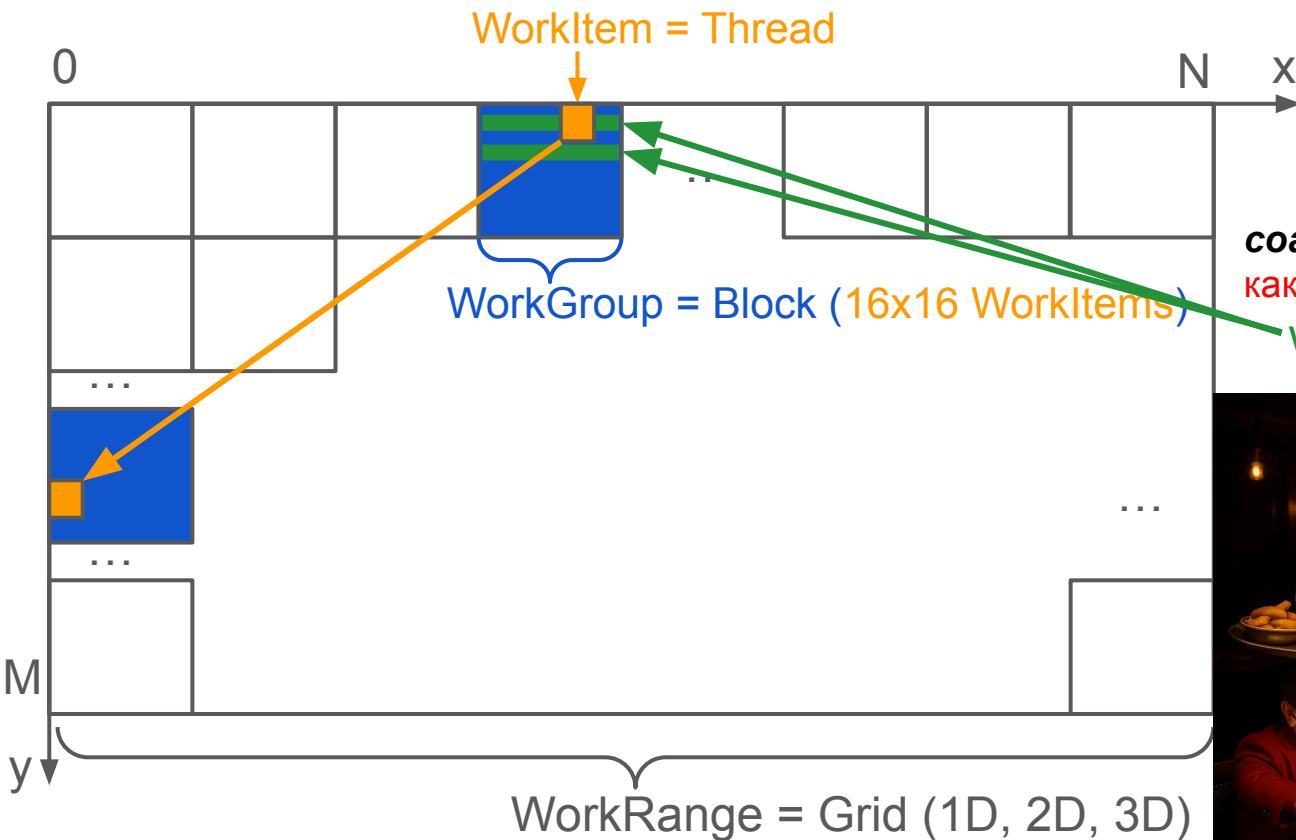
Транспонирование матрицы (*coalesced memory access*)



Транспонирование матрицы (*coalesced memory access*)



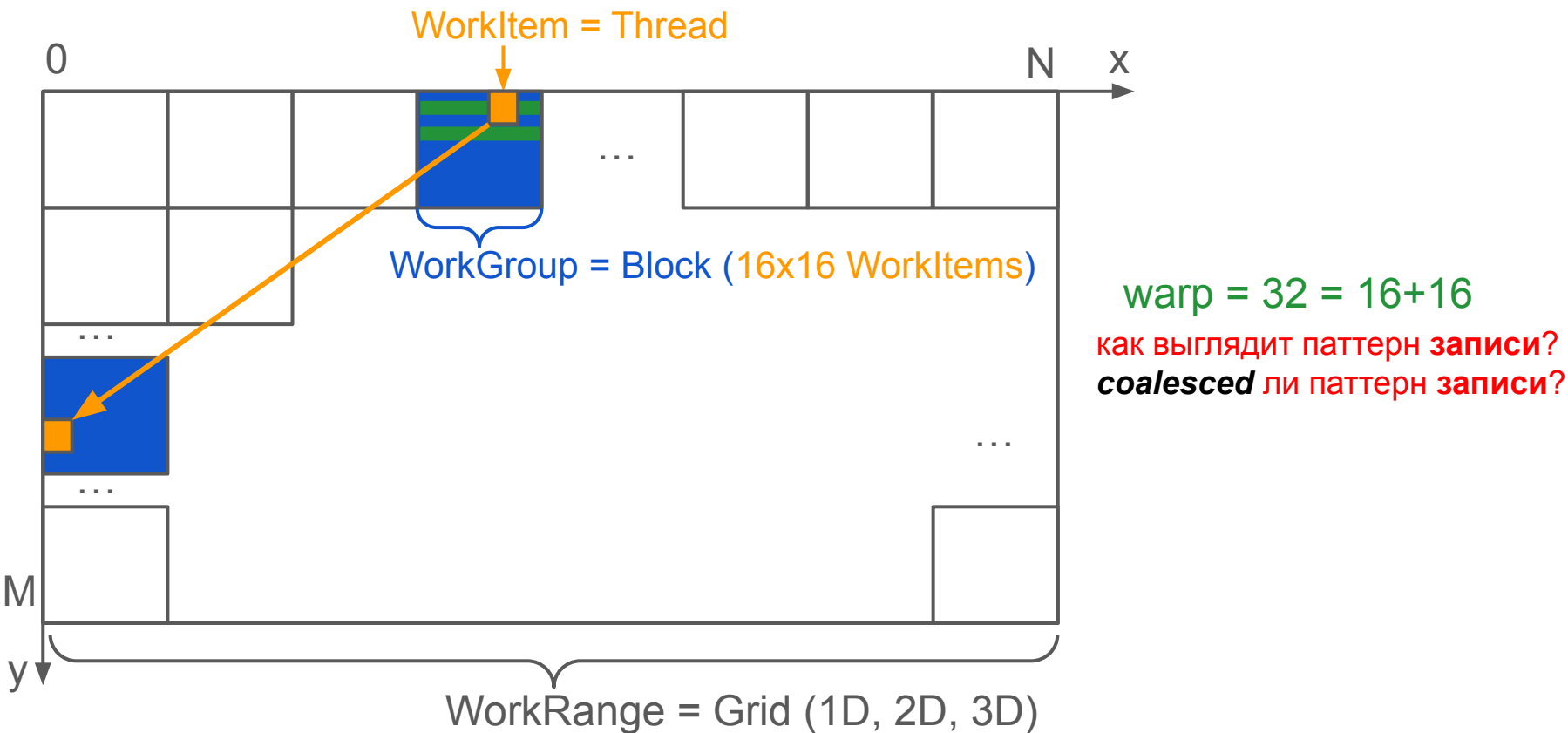
Транспонирование матрицы (*coalesced memory access*)



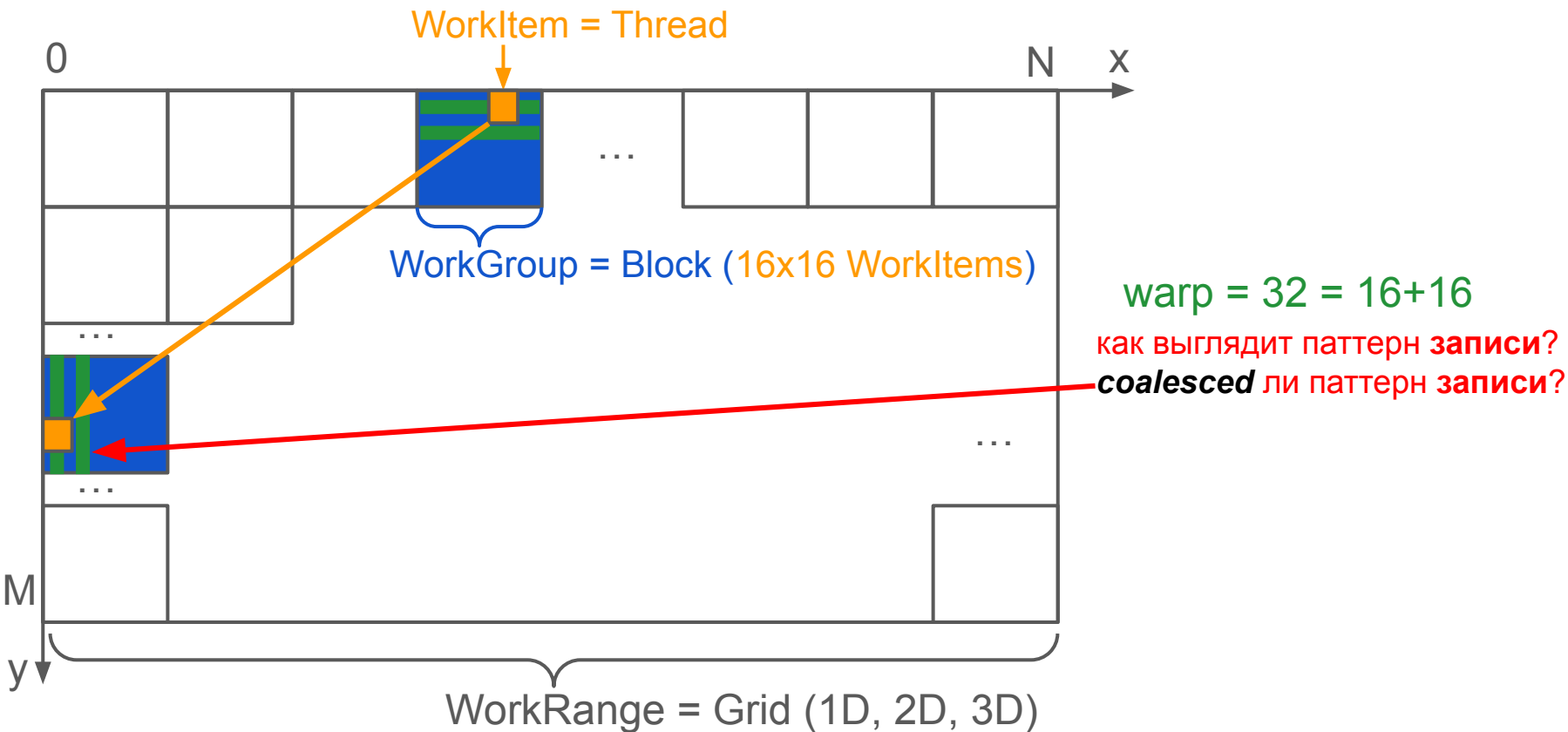
coalesced ли паттерн чтения?
как выглядит выкладка **warp**?
warp = 32 = 16+16



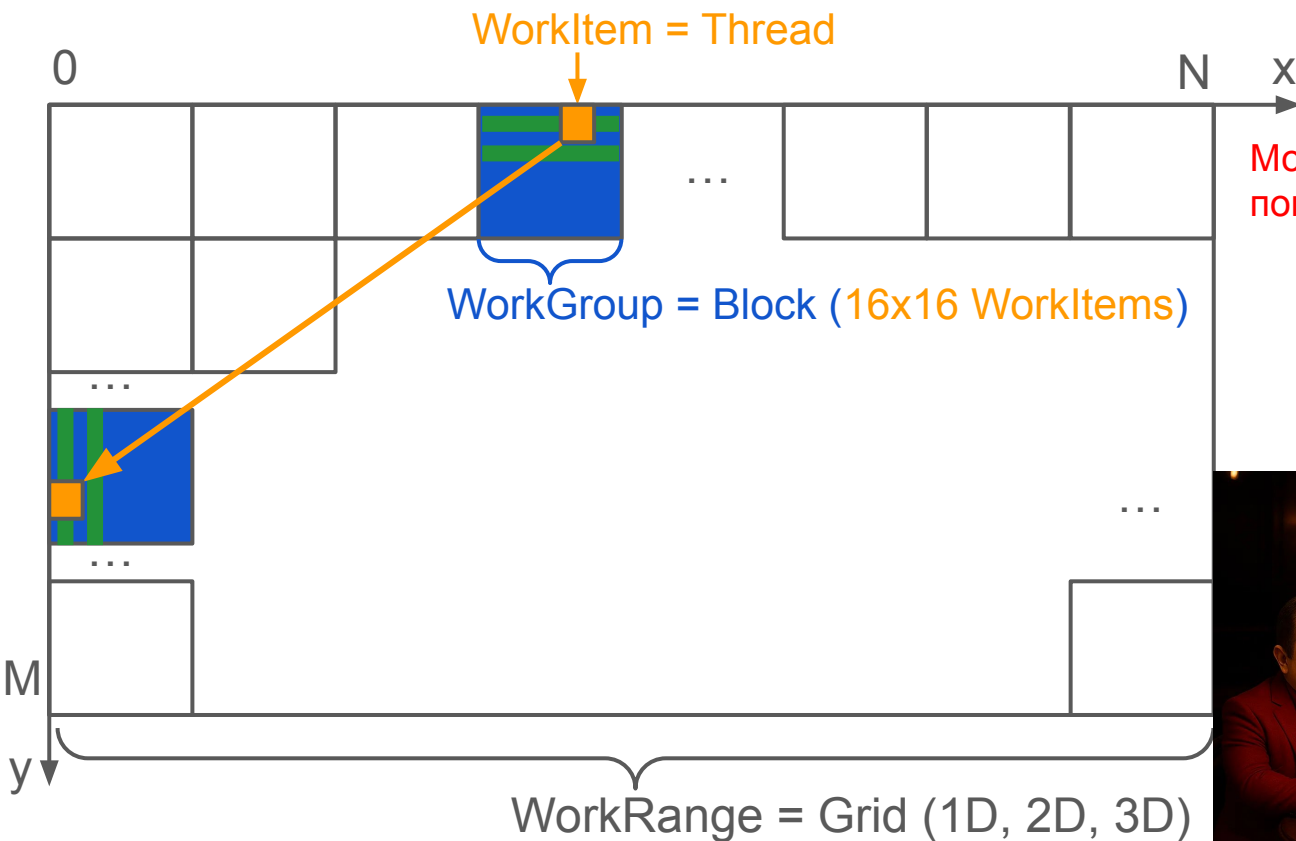
Транспонирование матрицы (*coalesced memory access*)



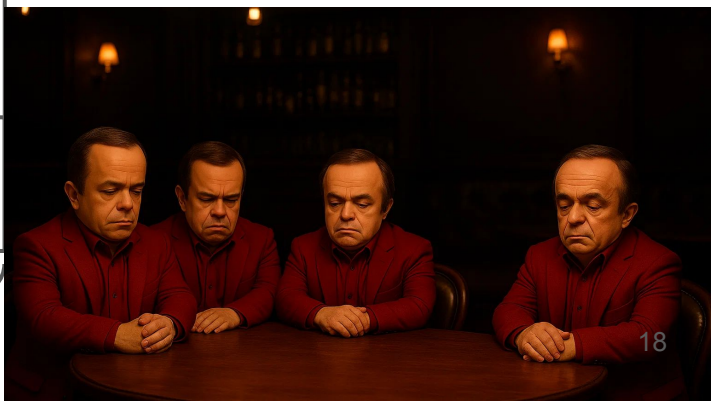
Транспонирование матрицы (*coalesced memory access*)



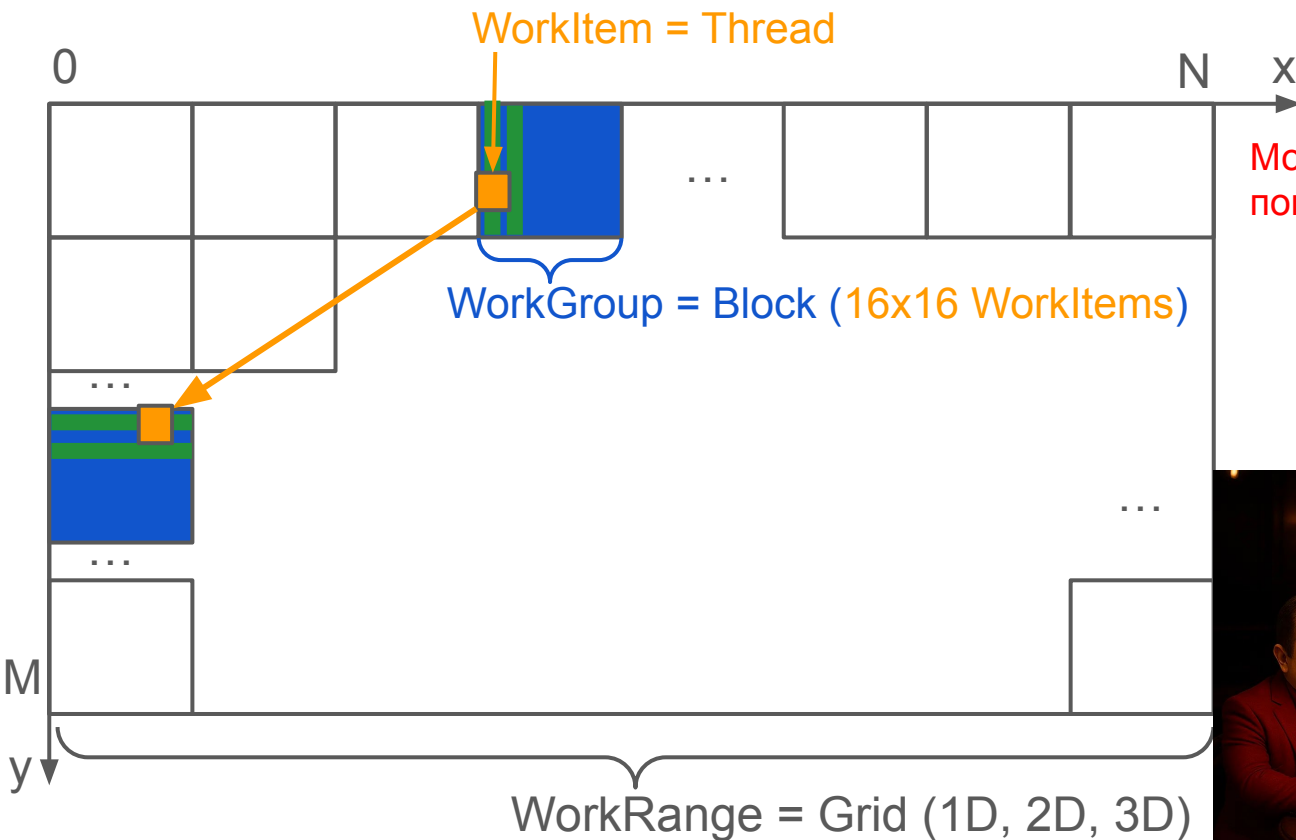
Транспонирование матрицы (*coalesced memory access*)



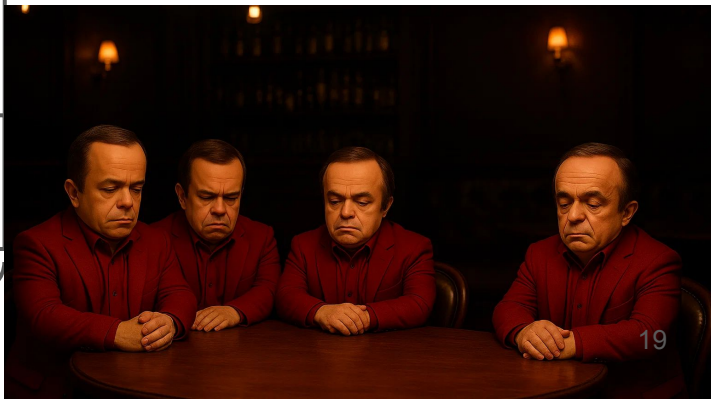
Можем ли мы исправить это
поменяв выкладку **warp**-а?



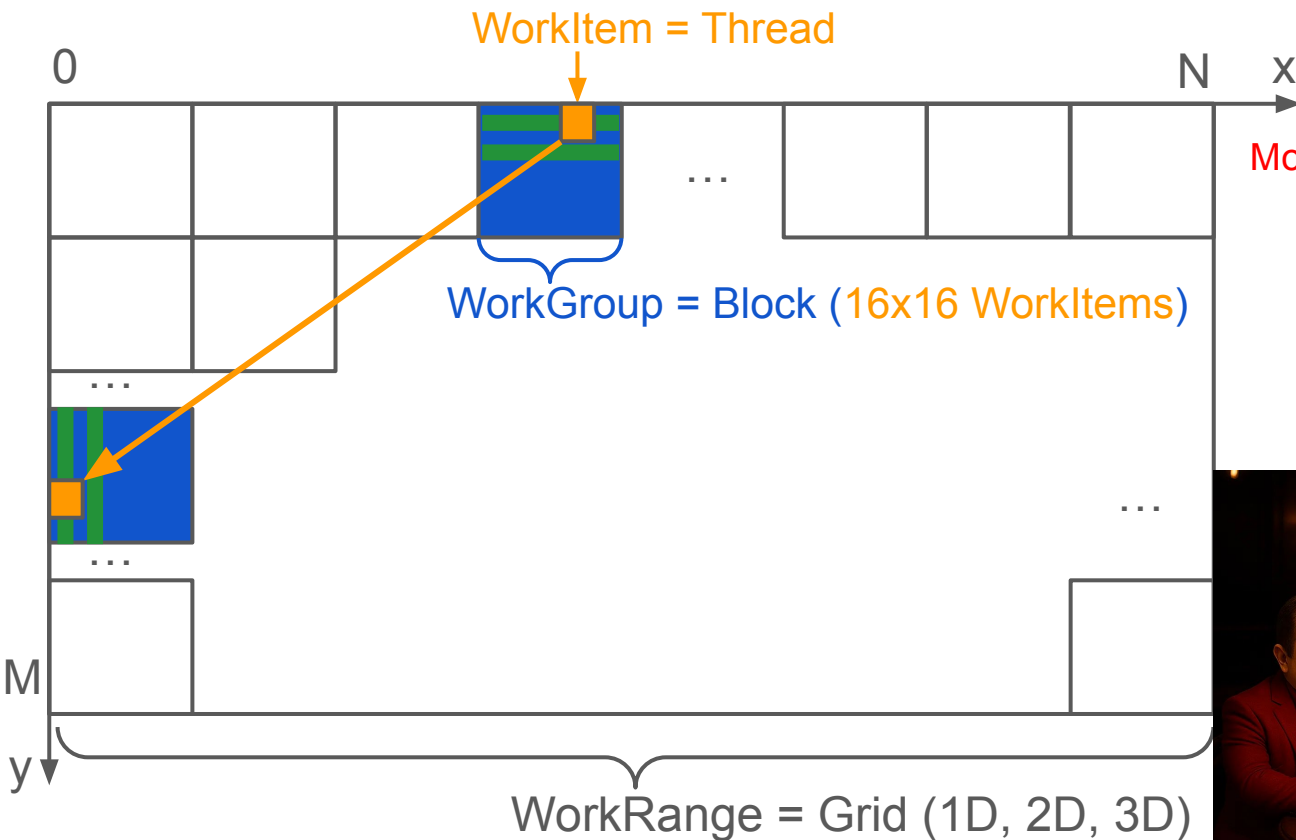
Транспонирование матрицы (*coalesced memory access*)



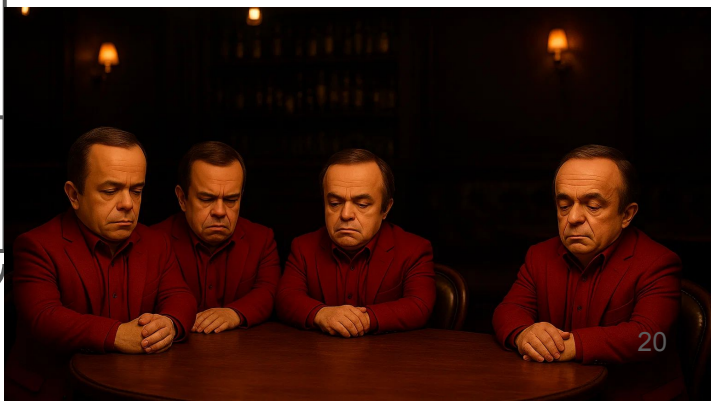
Можем ли мы исправить это
поменяв выкладку **warp**-а?



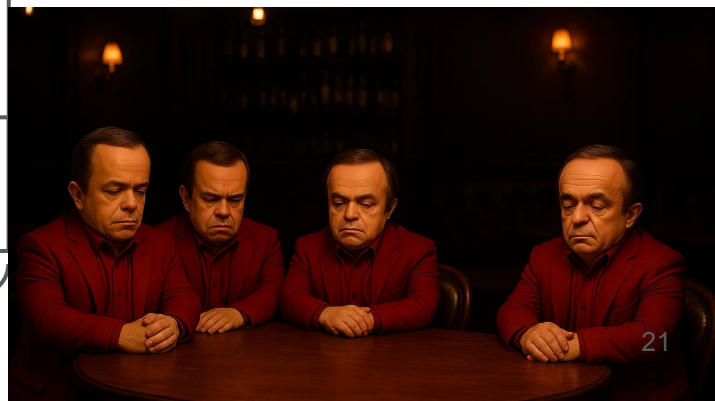
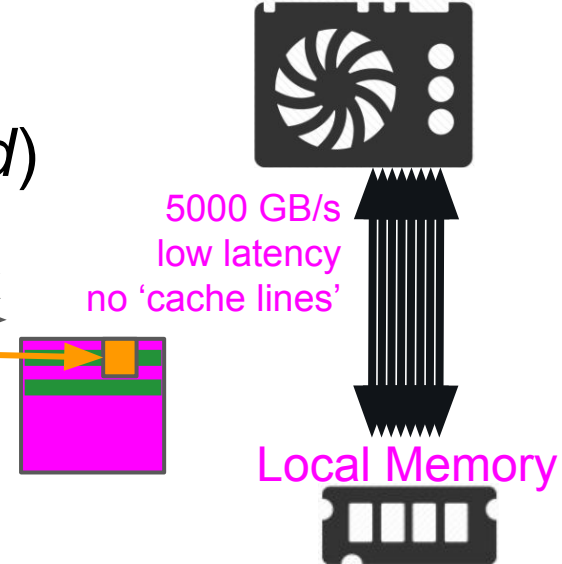
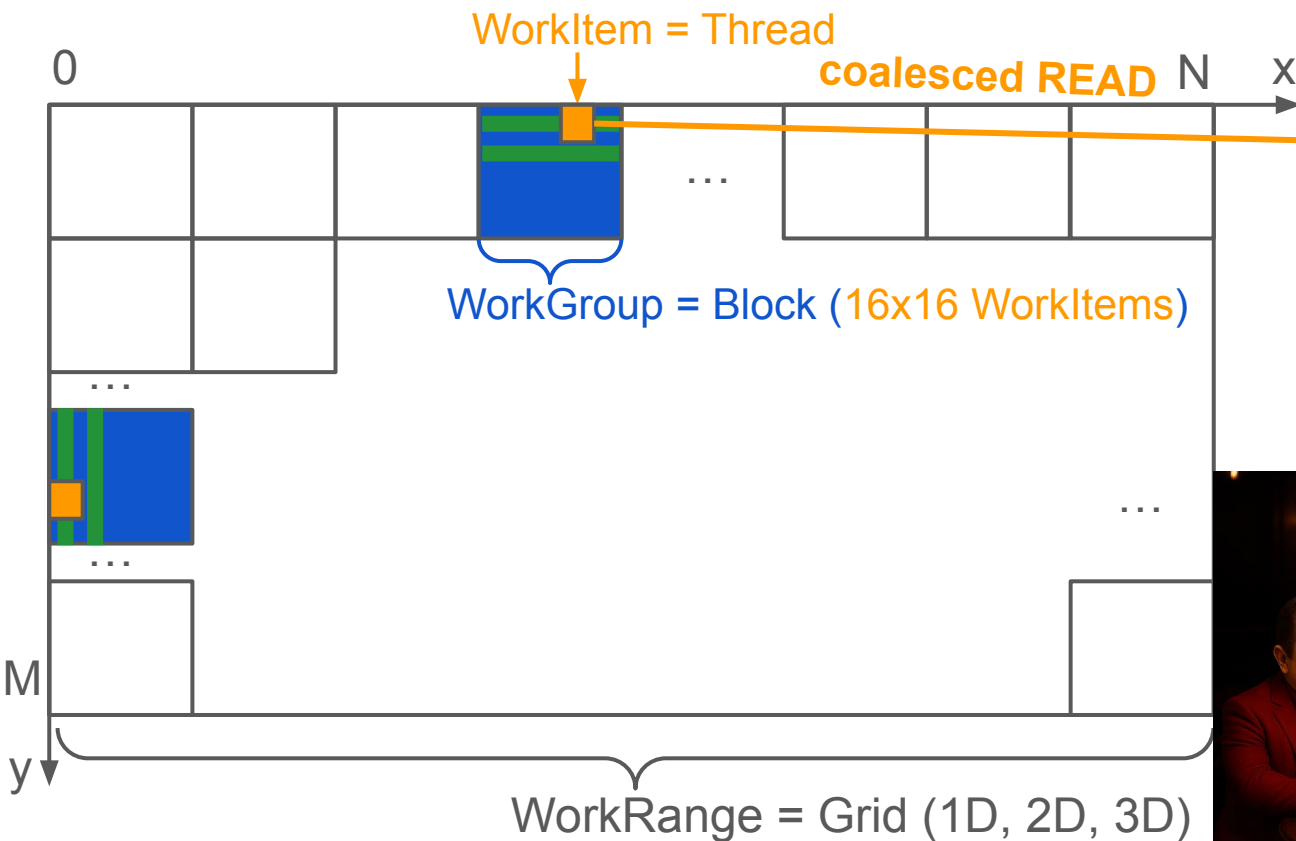
Транспонирование матрицы (*coalesced memory access*)



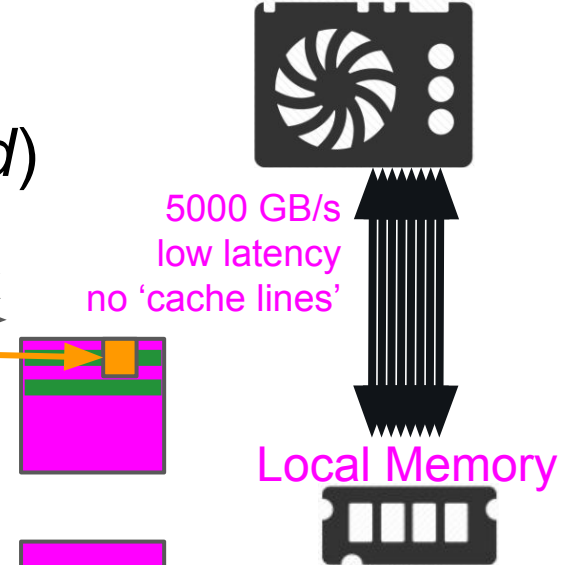
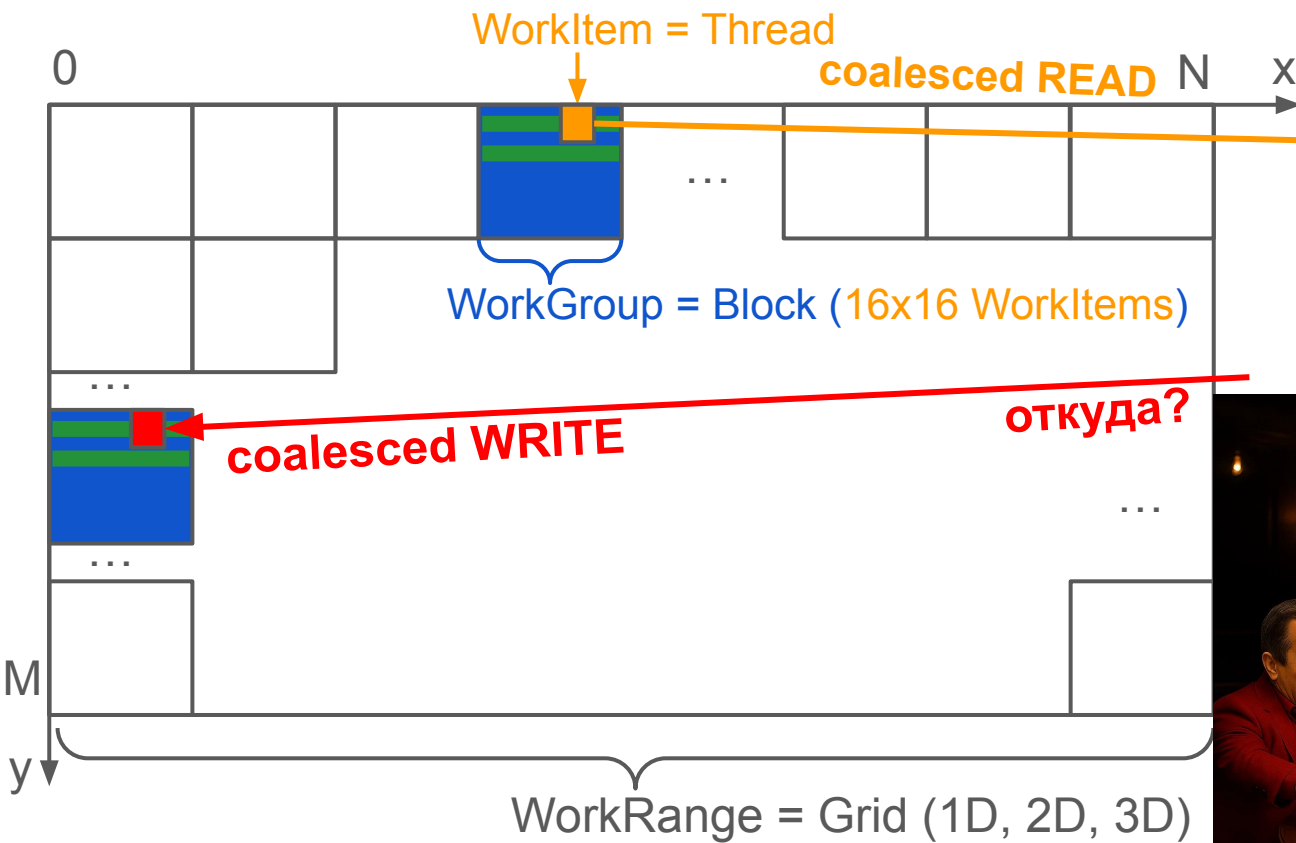
Можем ли мы исправить это?



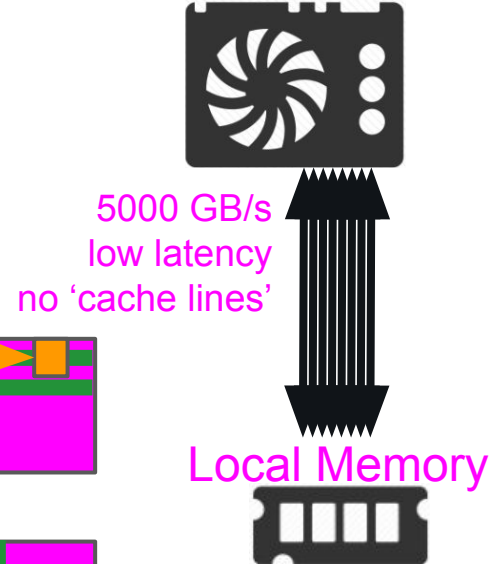
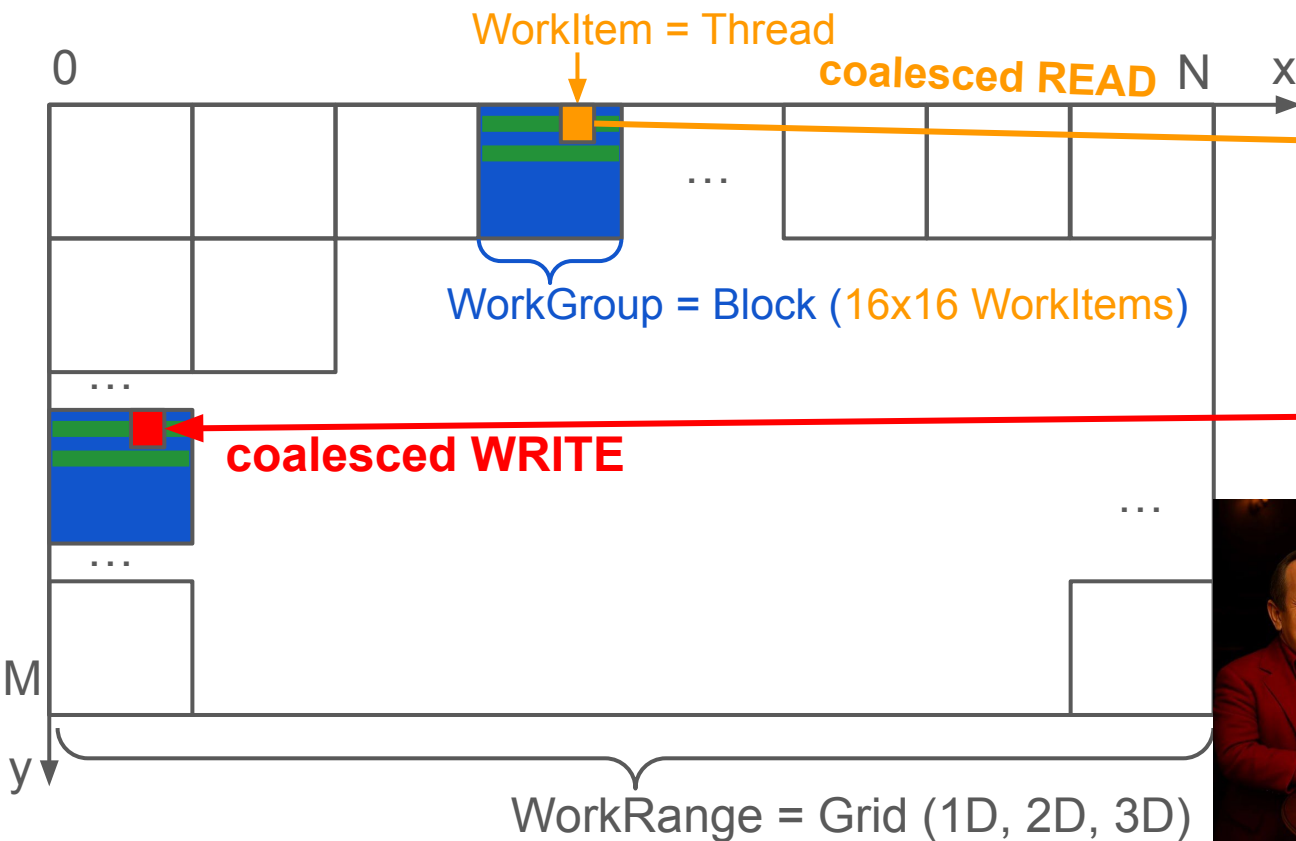
Транспонирование матрицы (coalesced)



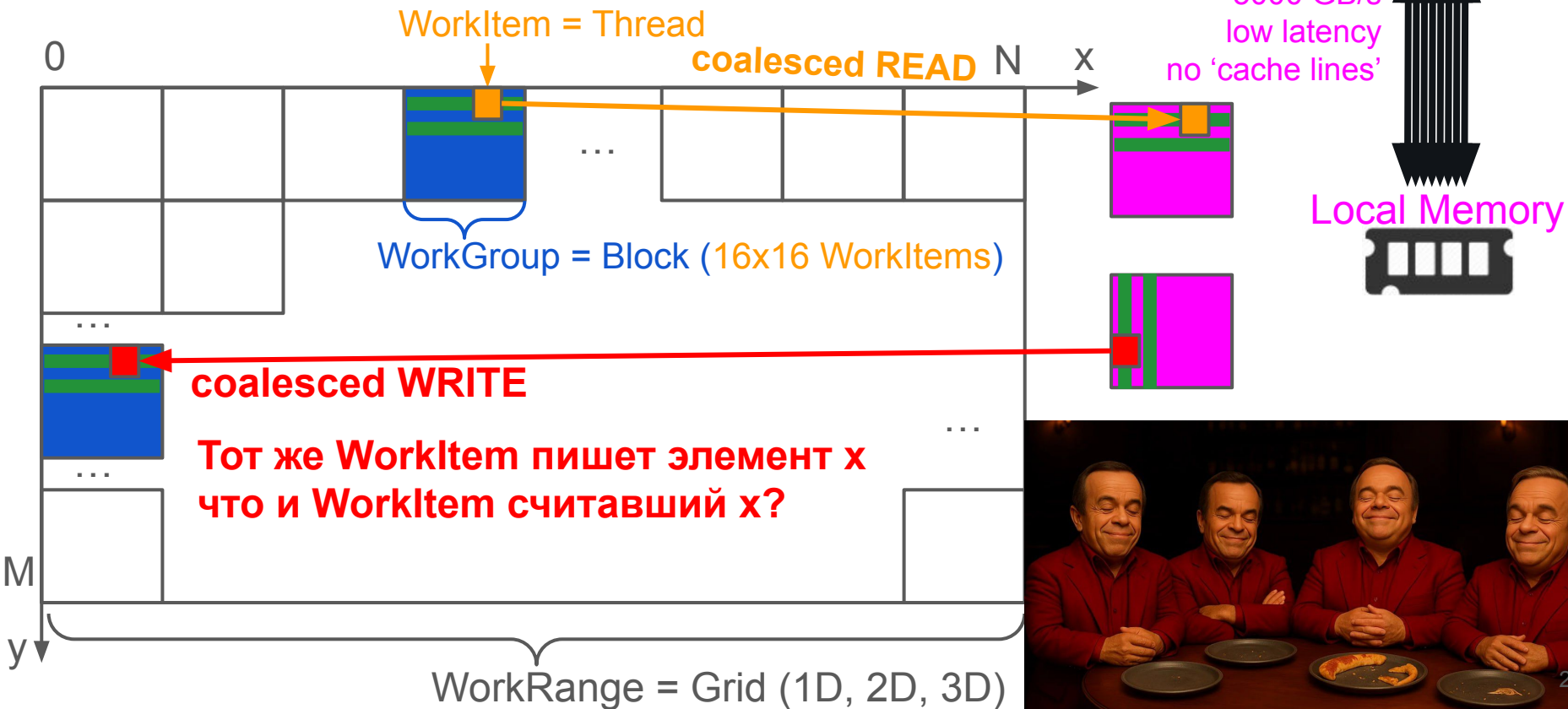
Транспонирование матрицы (coalesced)



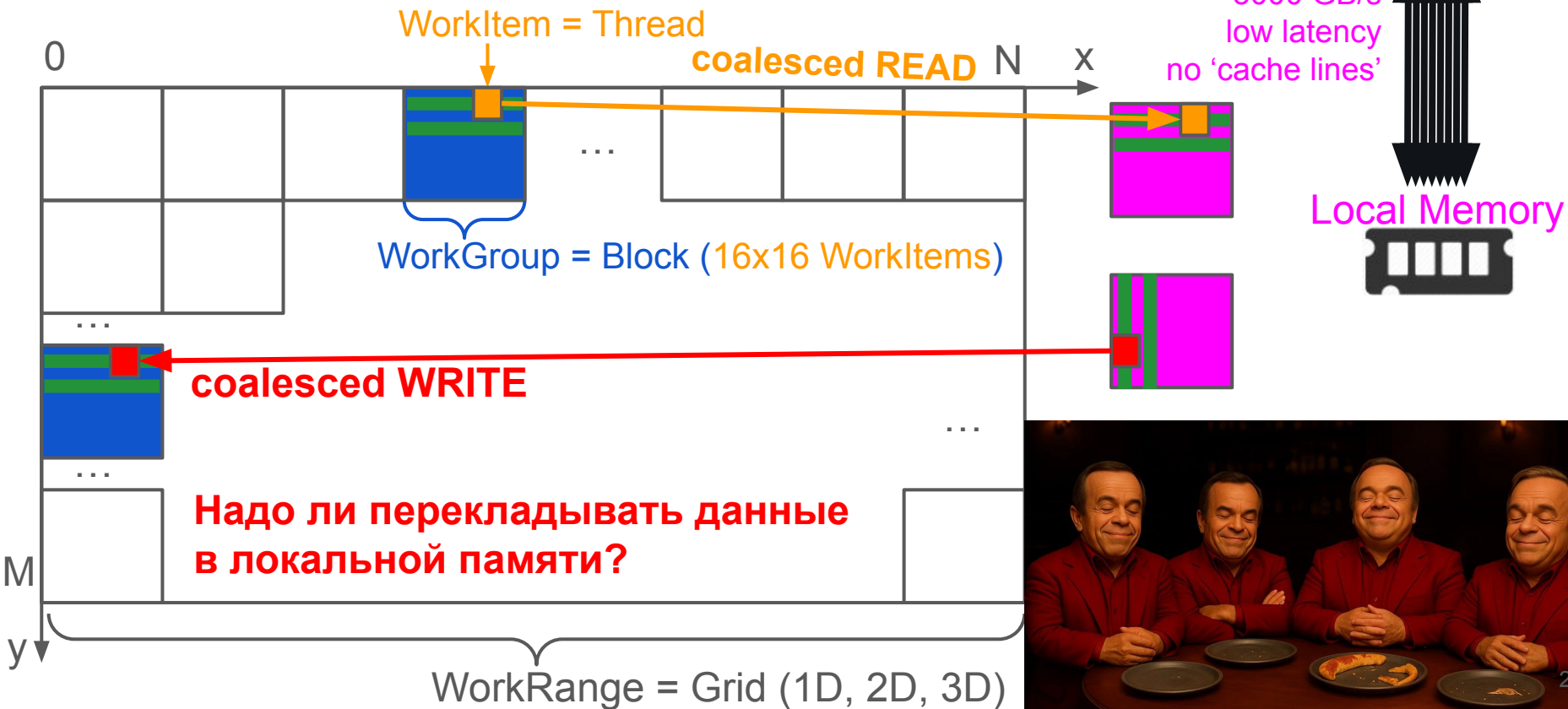
Транспонирование матрицы (*coalesced*)



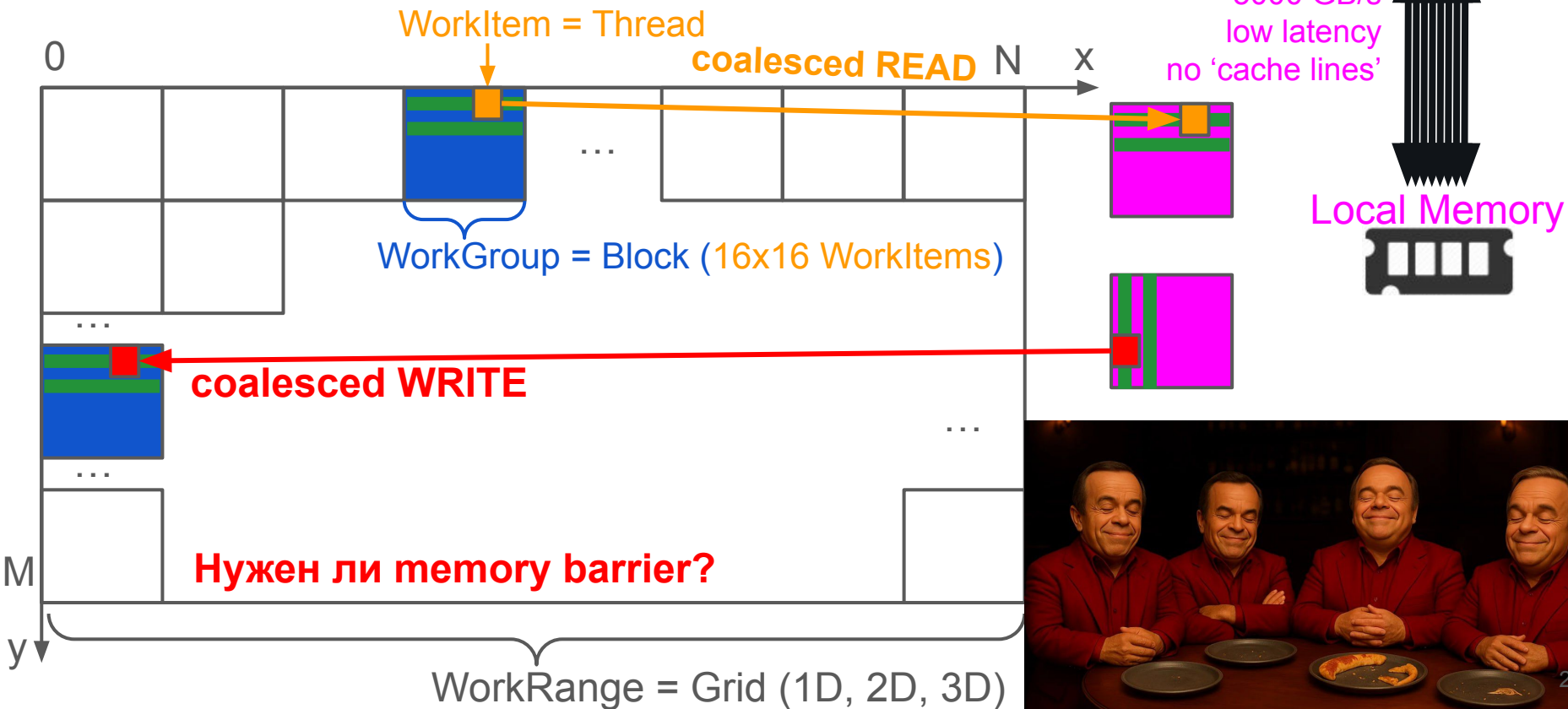
Транспонирование матрицы (coalesced)



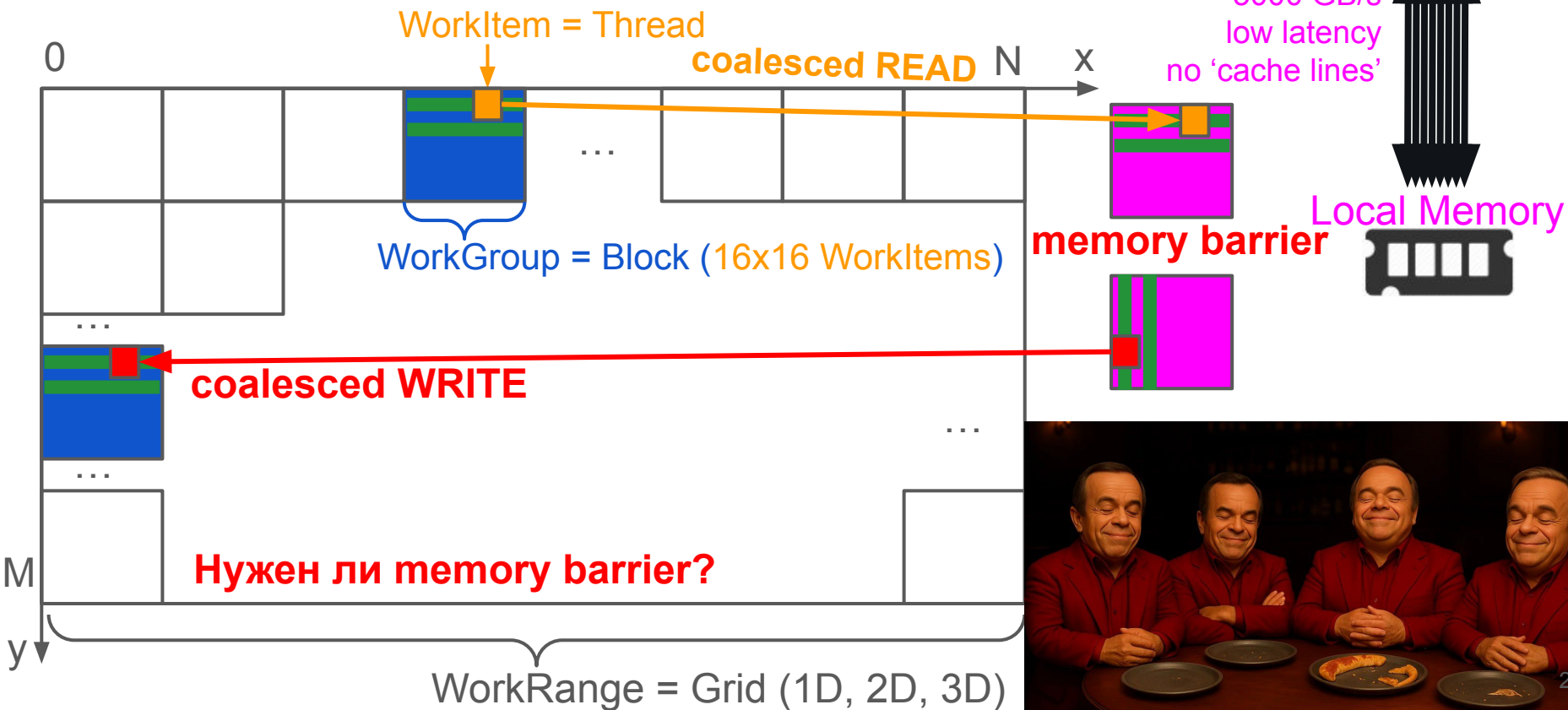
Транспонирование матрицы (coalesced)



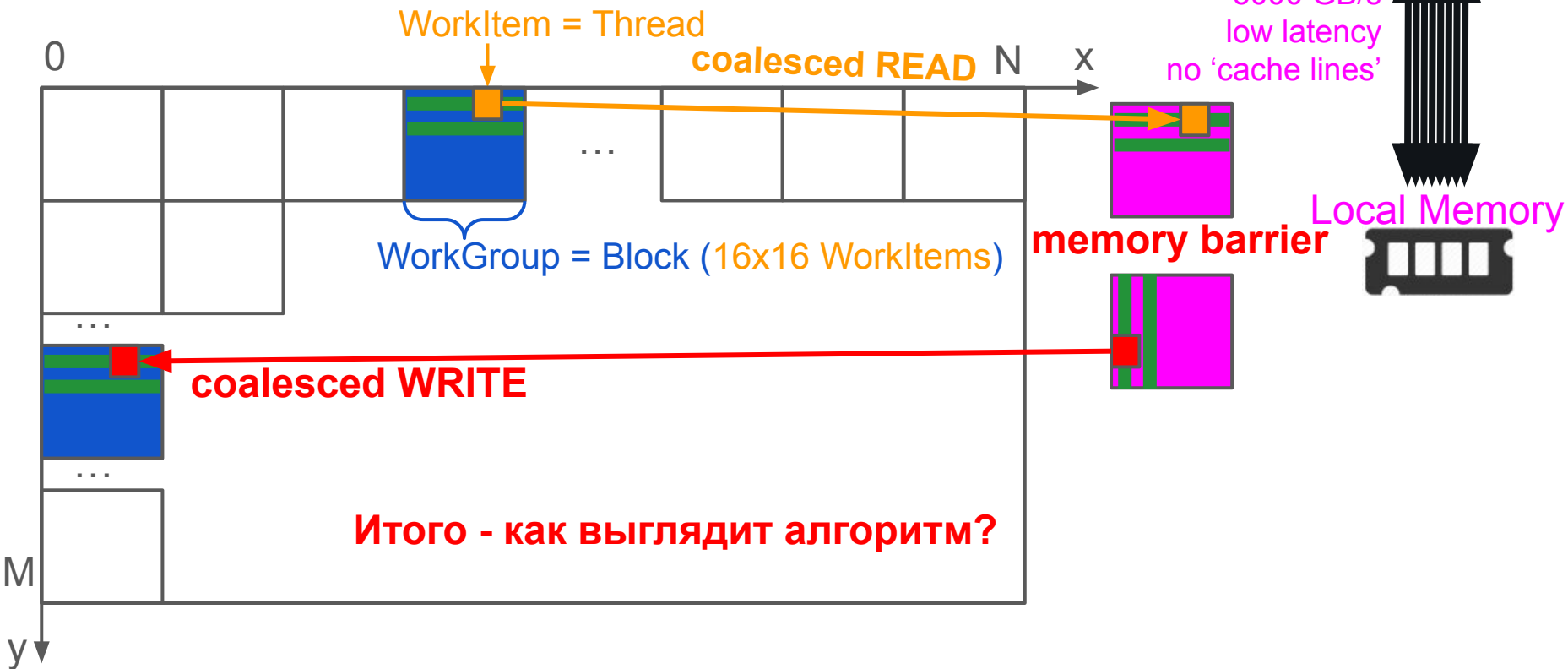
Транспонирование матрицы (coalesced)



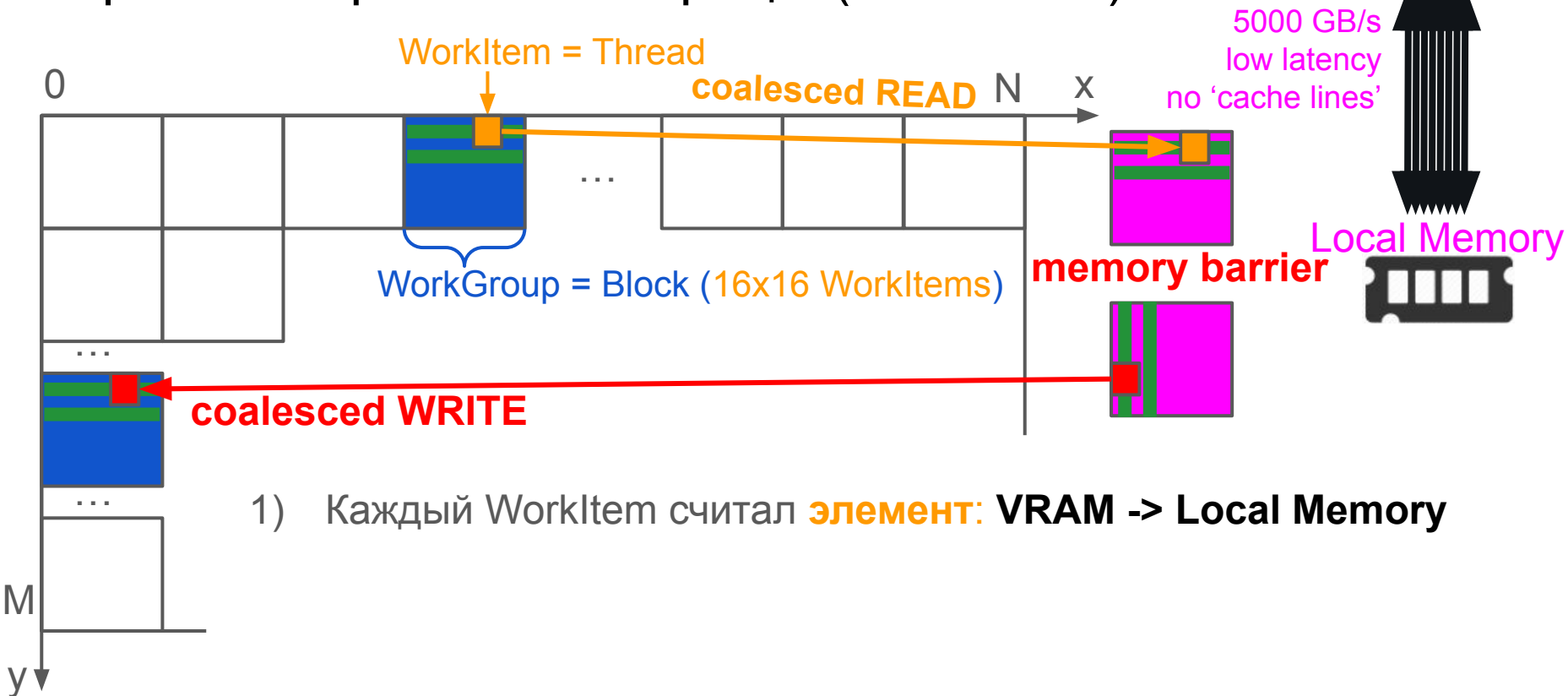
Транспонирование матрицы (coalesced)



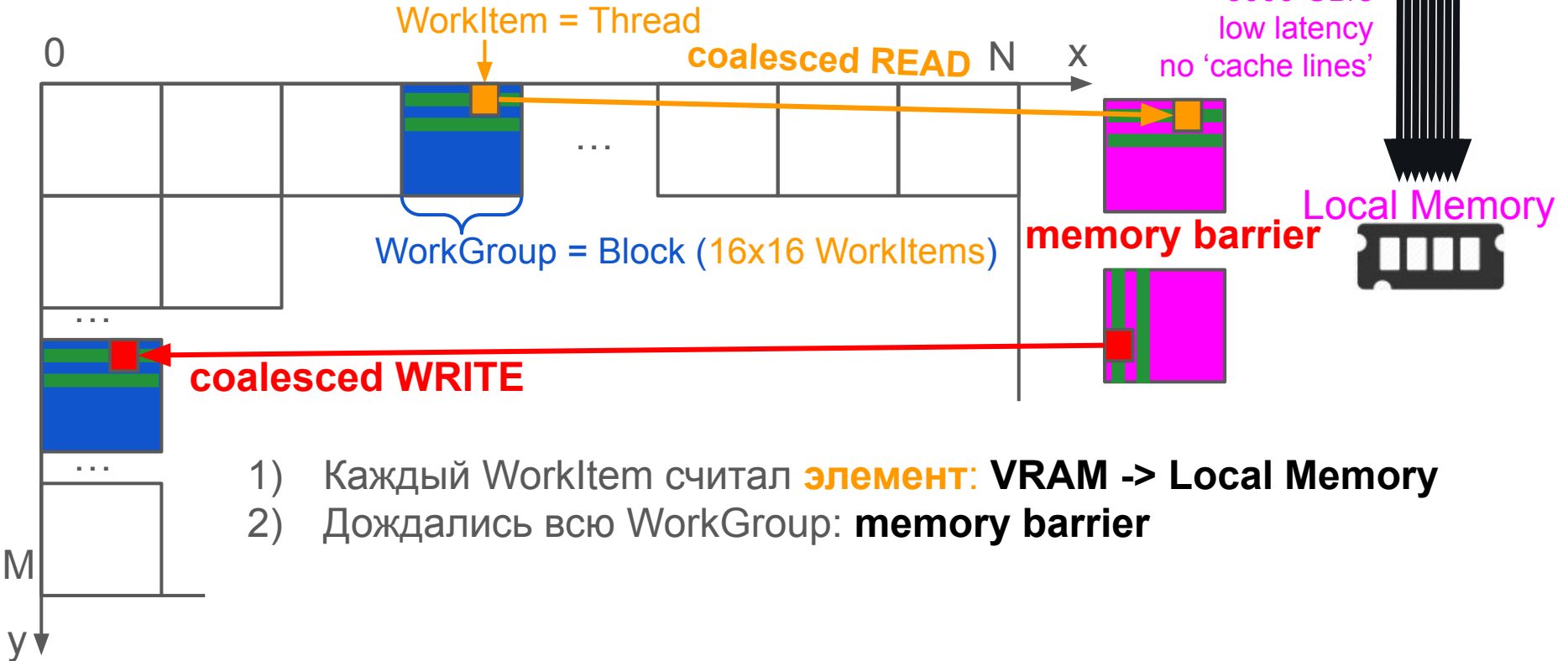
Транспонирование матрицы (coalesced)



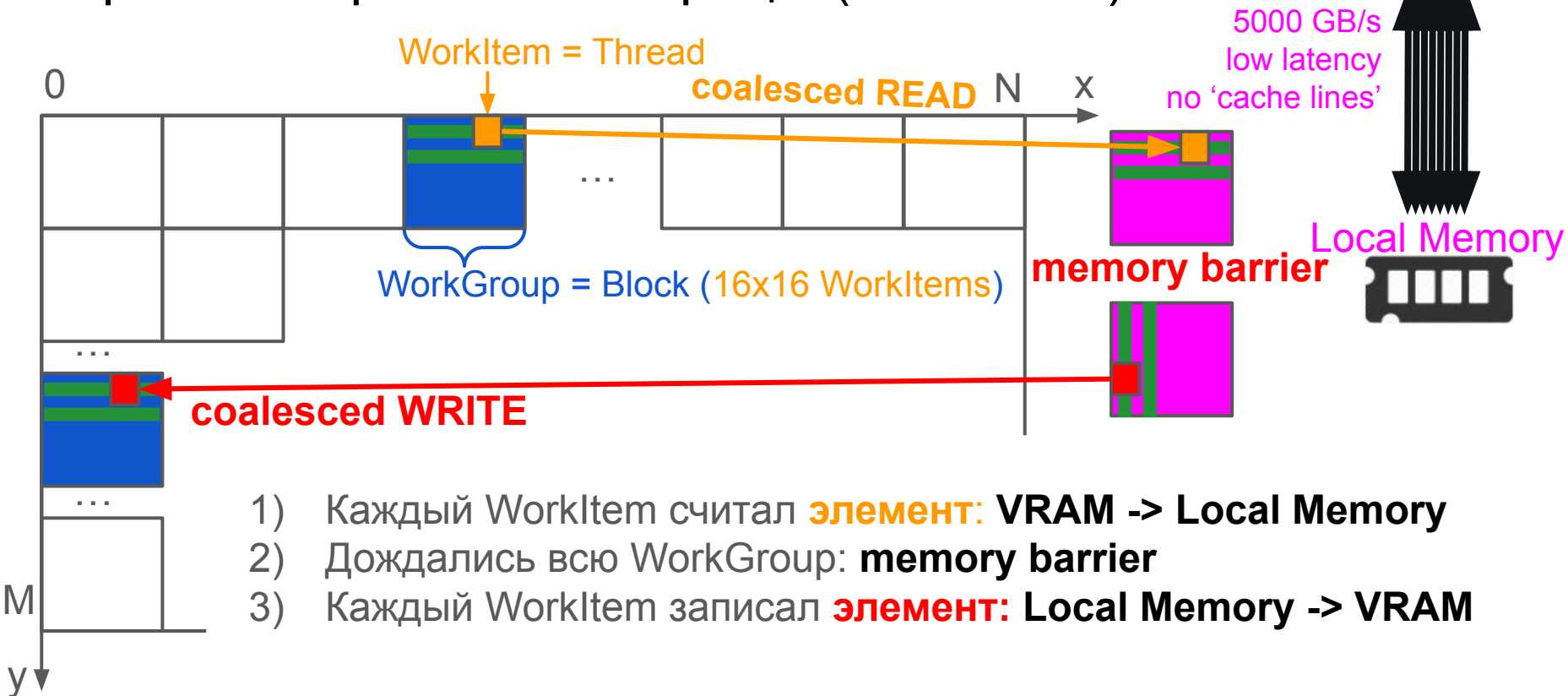
Транспонирование матрицы (coalesced)



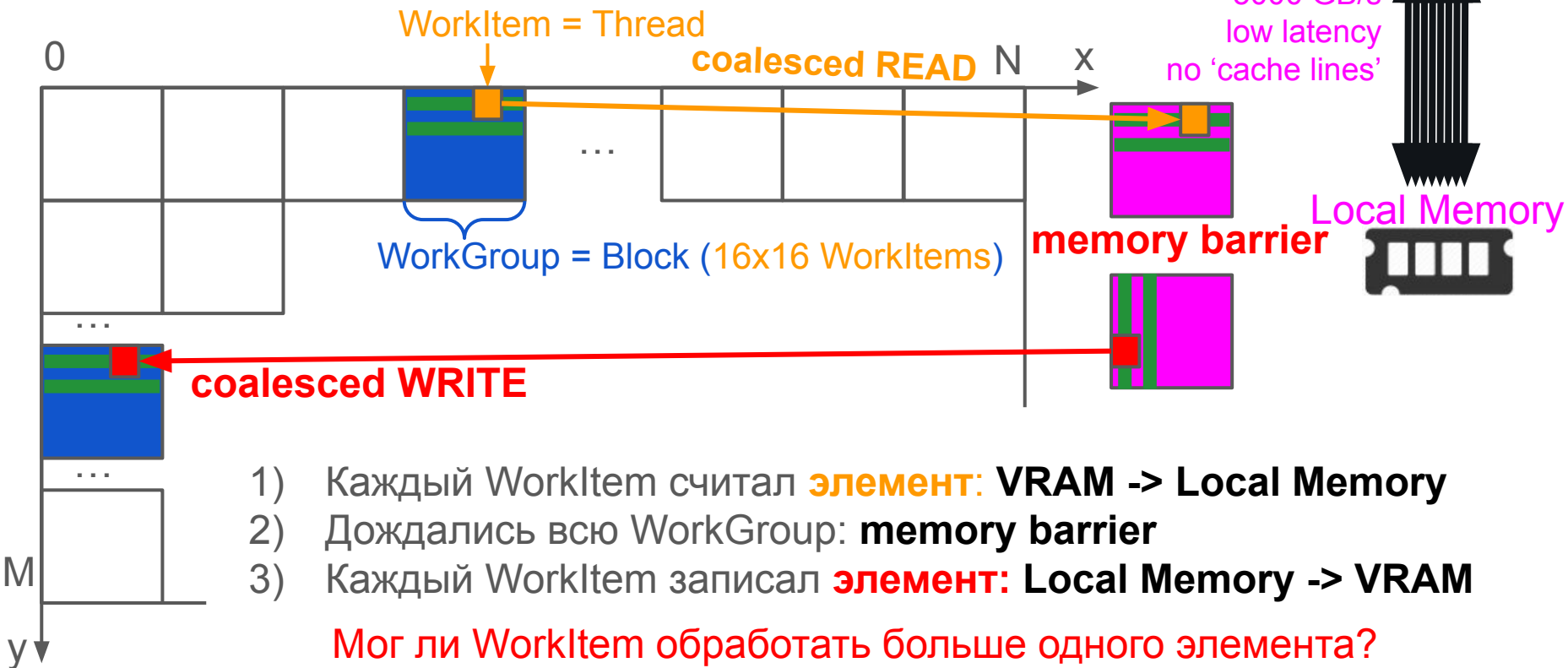
Транспонирование матрицы (coalesced)



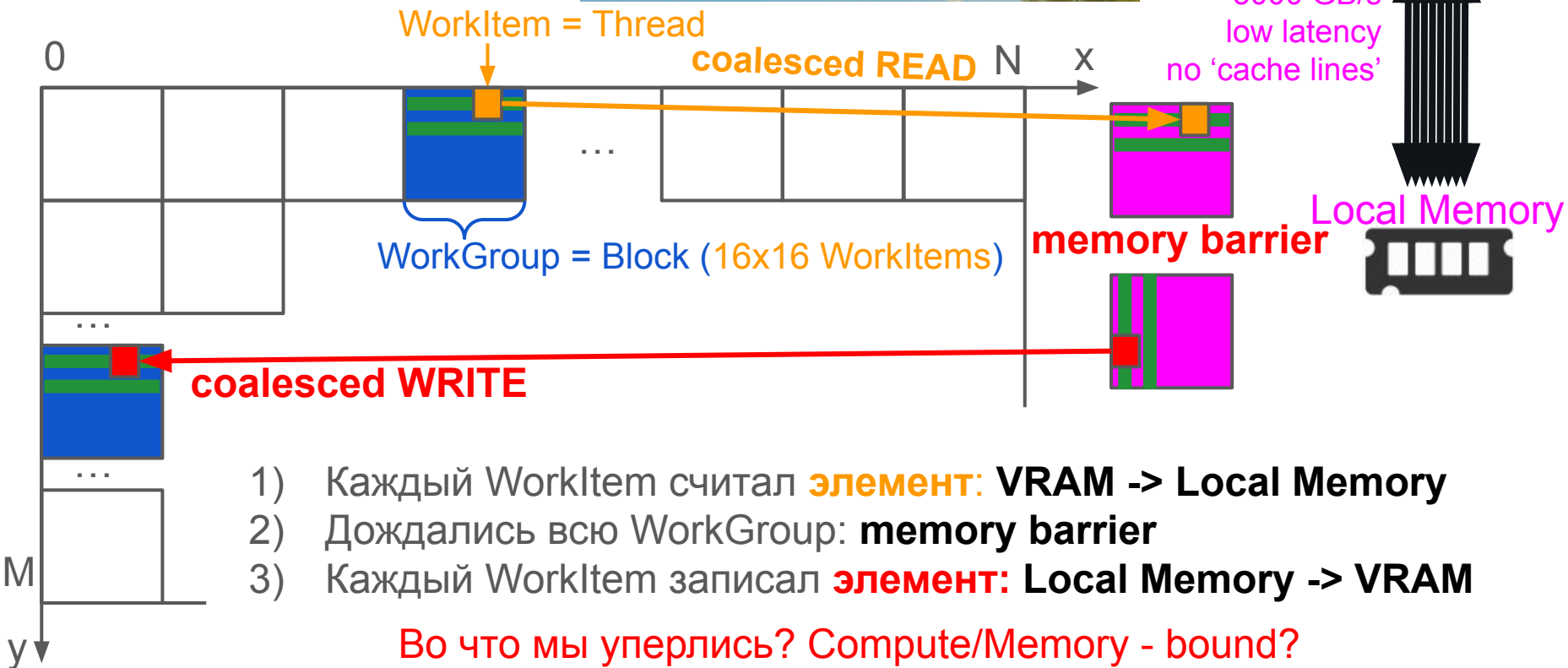
Транспонирование матрицы (coalesced)



Транспонирование матрицы (coalesced)



Транспонирование

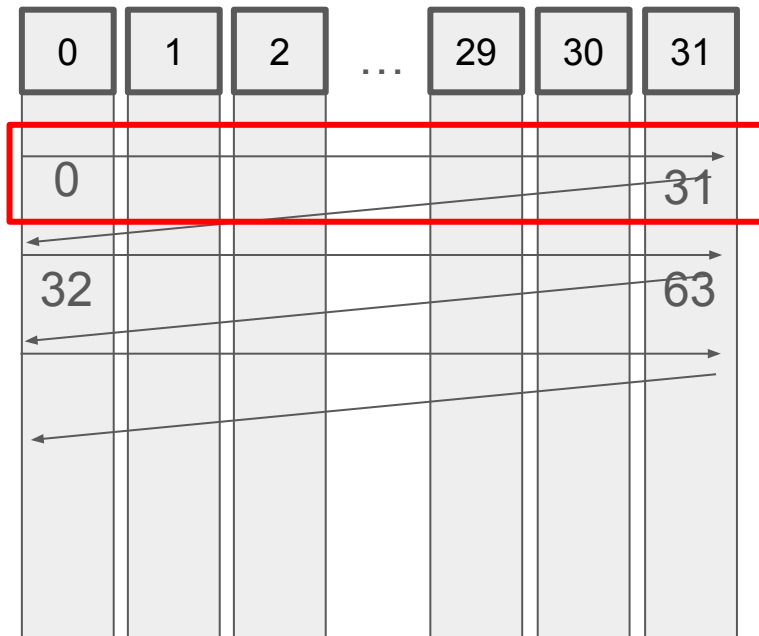


Локальная память - bank conflicts (напоминание)

```
__local unsigned int workgroup_data[256];  
workgroup_data[get_local_id(0)] = a[index];
```

Local memory banks:

Local address space:

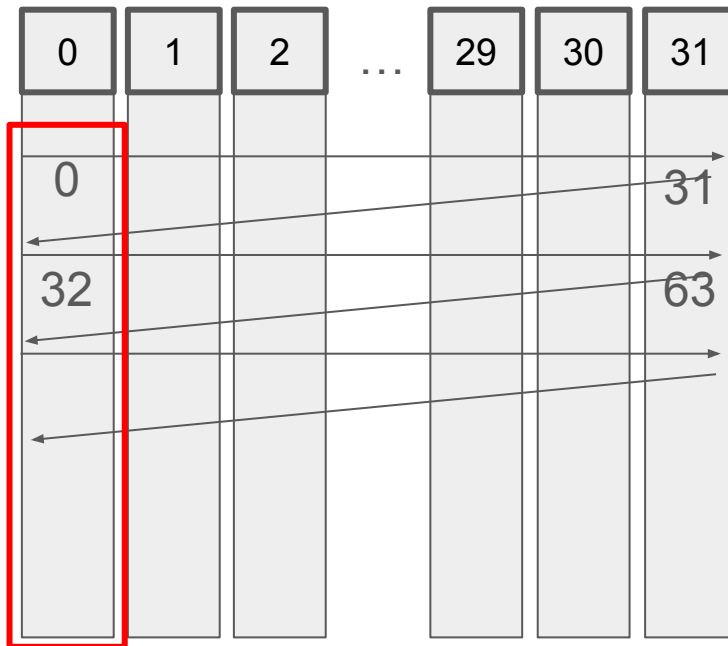


Локальная память - bank conflicts (напоминание)

```
__local unsigned int workgroup_data[256];  
workgroup_data[32 * get_local_id(0)] = a[index];
```

Local memory banks:

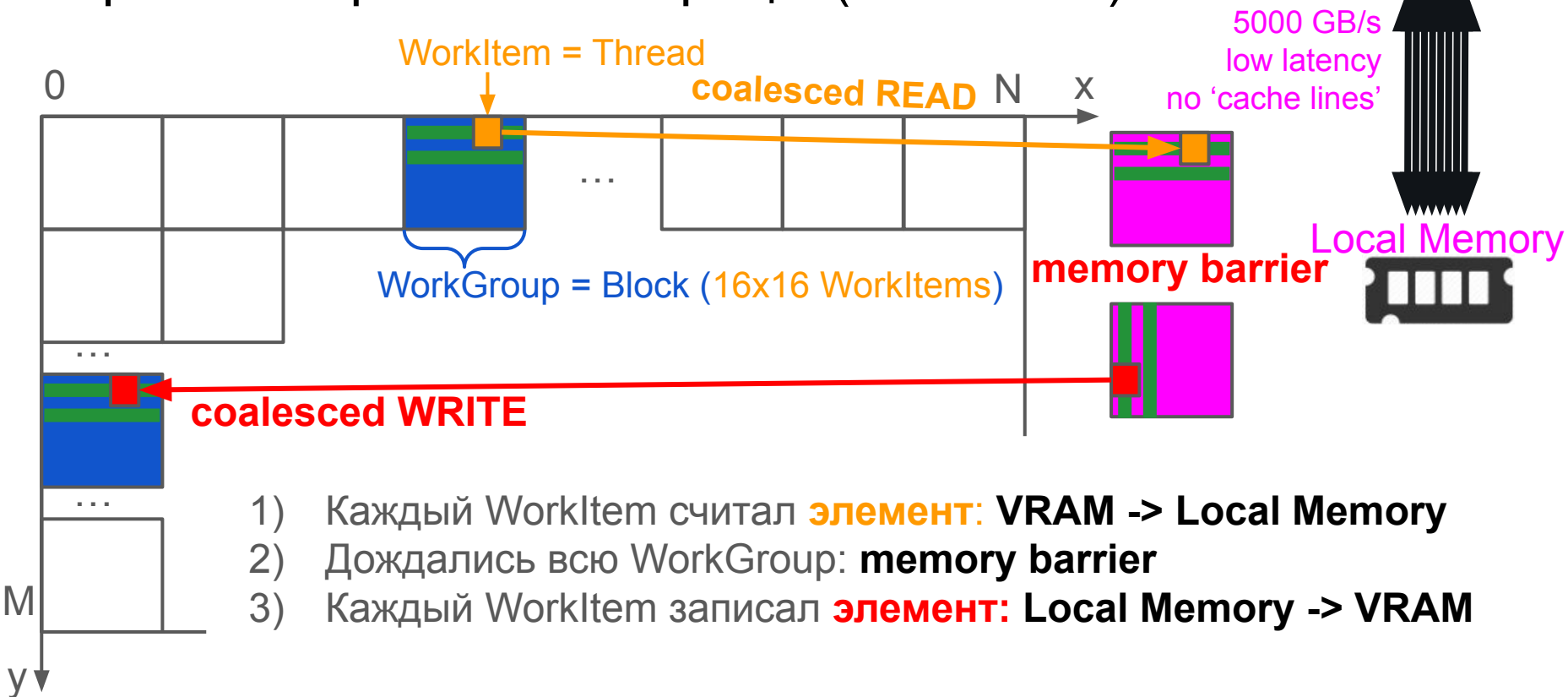
Local address space:



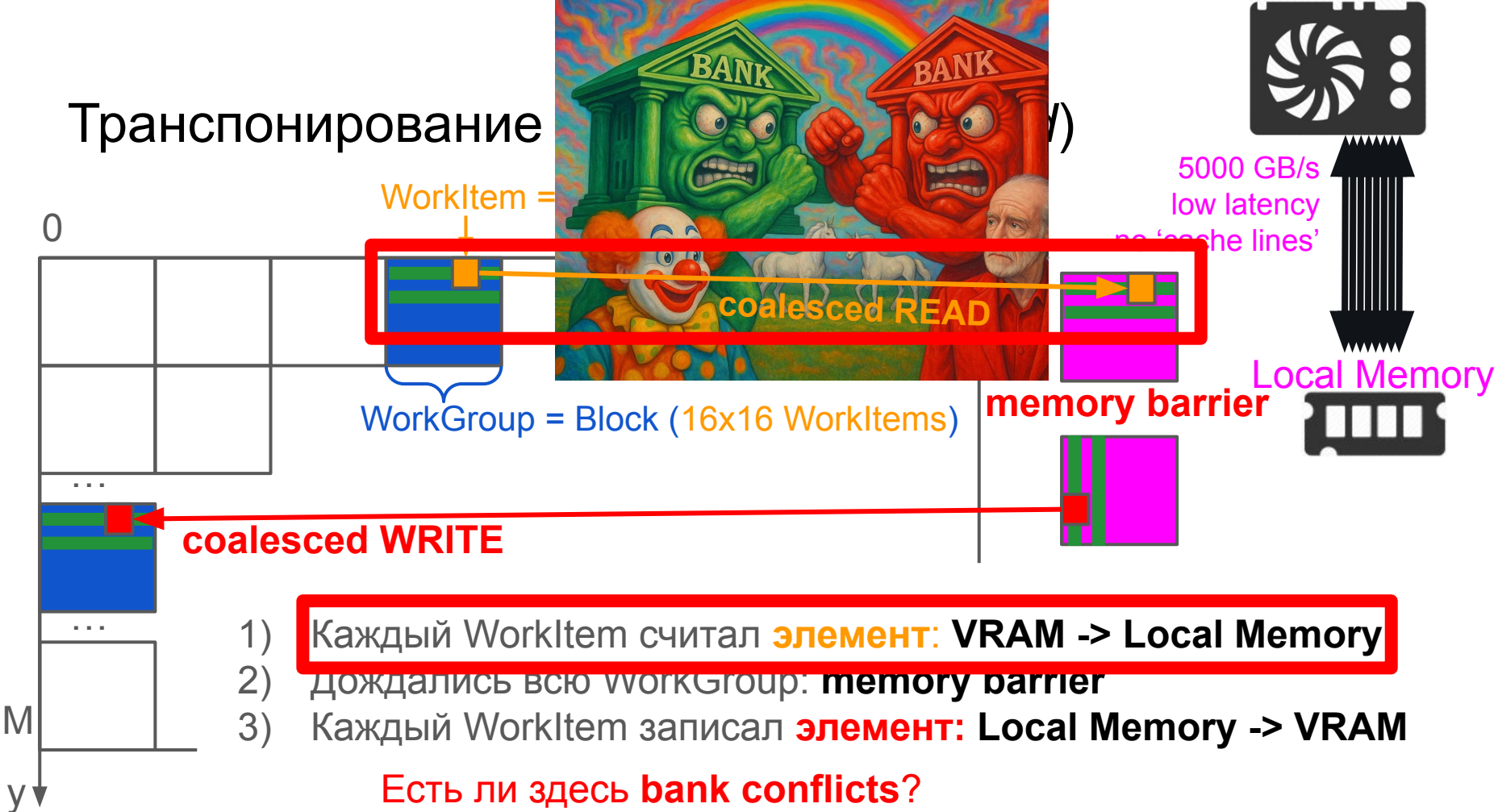
Bank conflict!



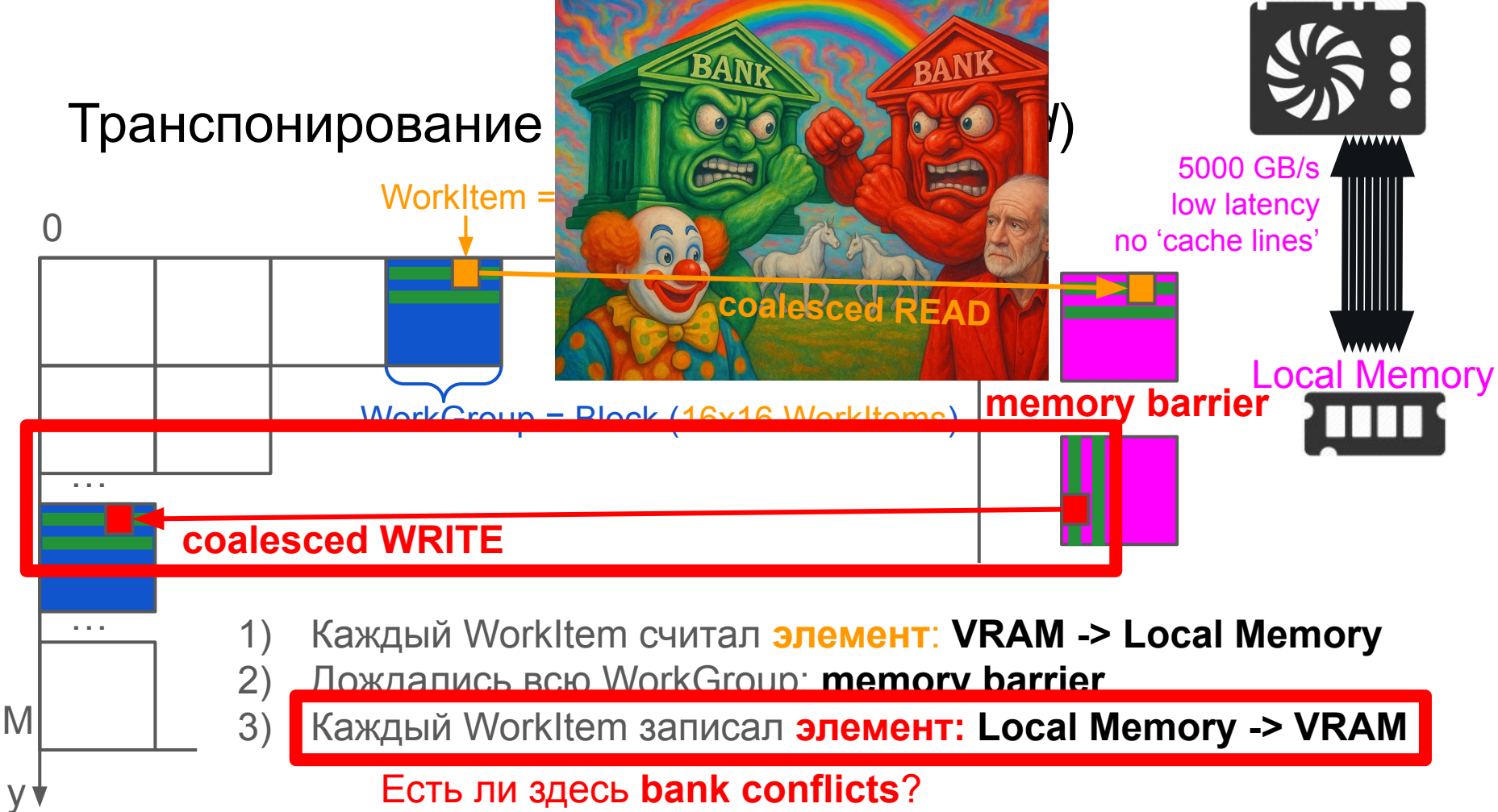
Транспонирование матрицы (coalesced)

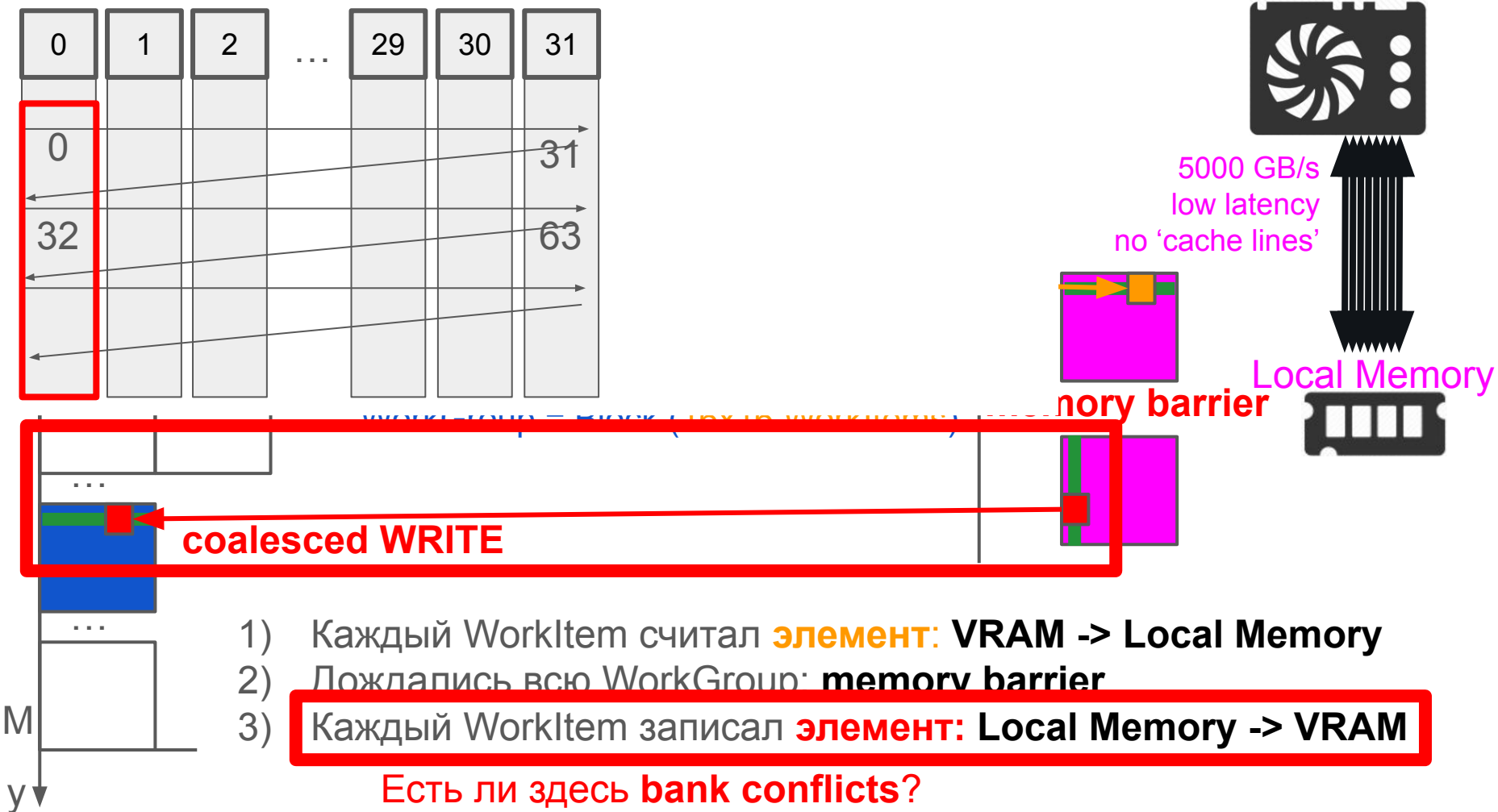


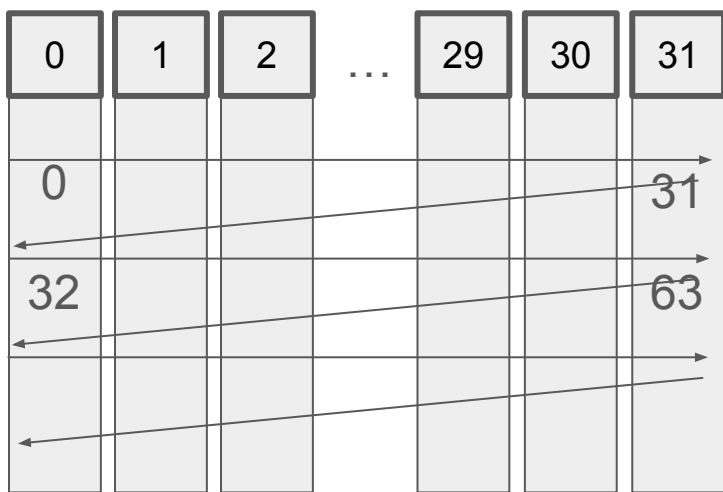
Транспонирование



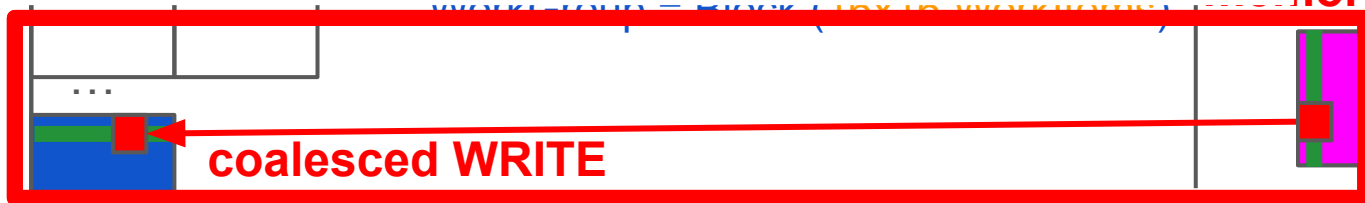
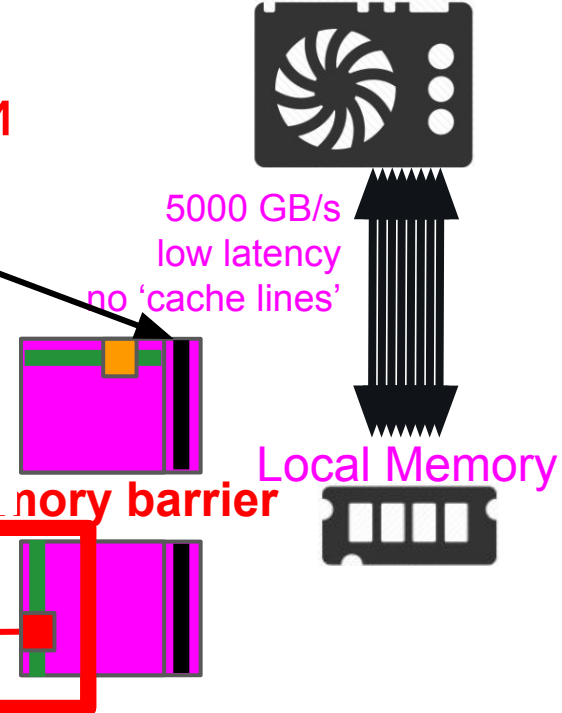
Транспонирование





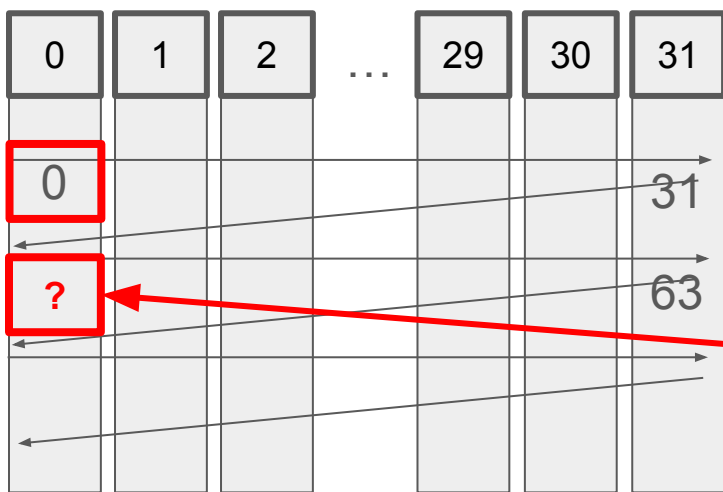


Что будет если
Tile - 33x32?



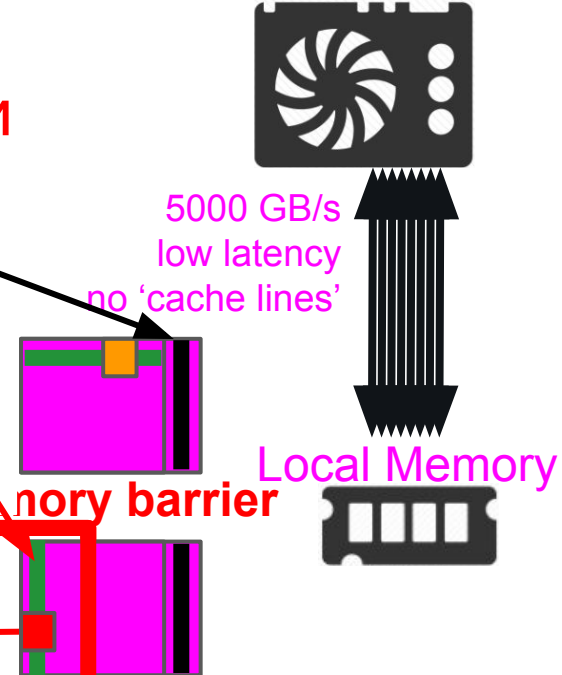
- 1) Каждый WorkItem считал элемент: VRAM -> Local Memory
- 2) Дождались всю WorkGroup: memory barrier
- 3) Каждый WorkItem записал элемент: Local Memory -> VRAM

Есть ли здесь bank conflicts?



Что будет если
Tile - 33x32?

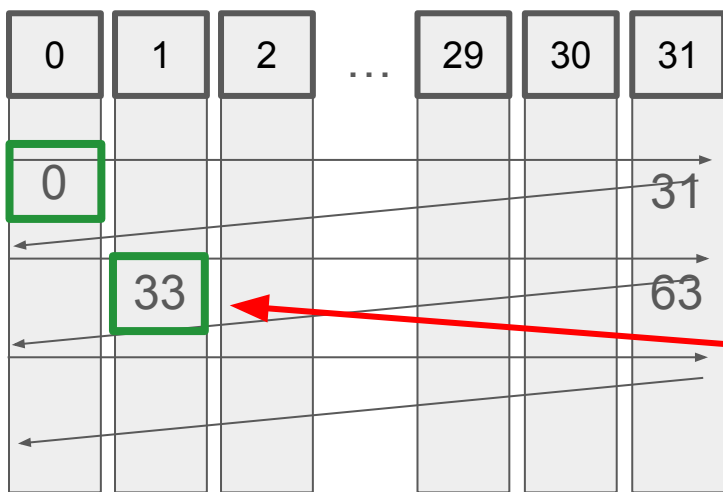
В какой банке второй
элемент столбика?



coalesced WRITE

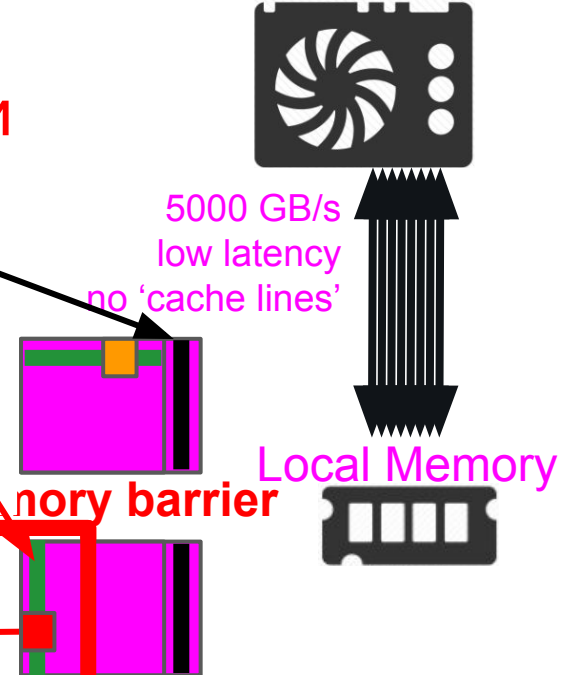
- 1) Каждый WorkItem считал элемент: VRAM -> Local Memory
- 2) Дождались всю WorkGroup: memory barrier
- 3) Каждый WorkItem записал элемент: Local Memory -> VRAM

Есть ли здесь bank conflicts?



Что будет если
Tile - 33x32?

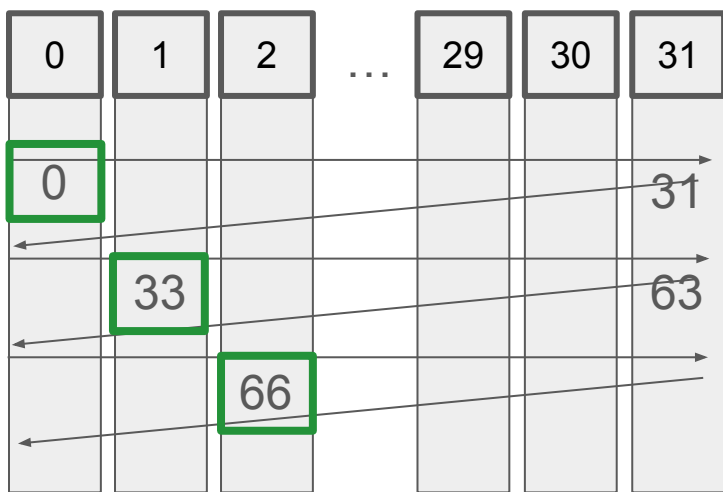
В какой банке второй
элемент столбика?



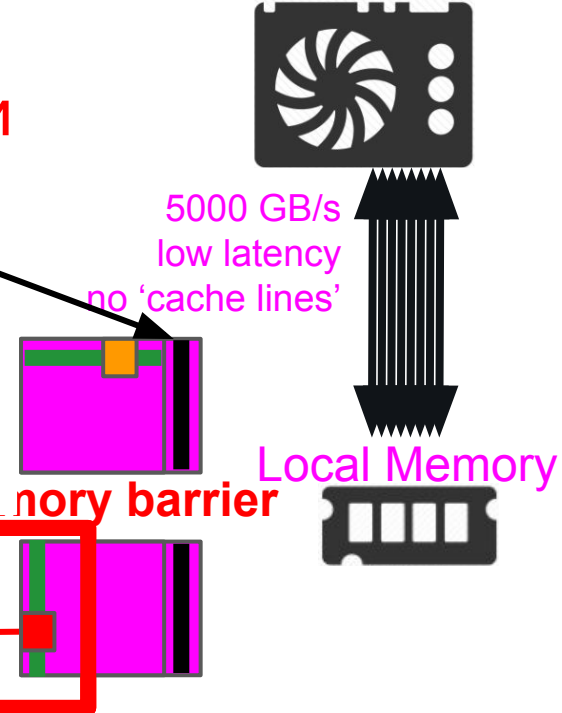
coalesced WRITE

- 1) Каждый WorkItem считал элемент: VRAM -> Local Memory
- 2) Дождались всю WorkGroup: memory barrier
- 3) Каждый WorkItem записал элемент: Local Memory -> VRAM

Есть ли здесь bank conflicts?

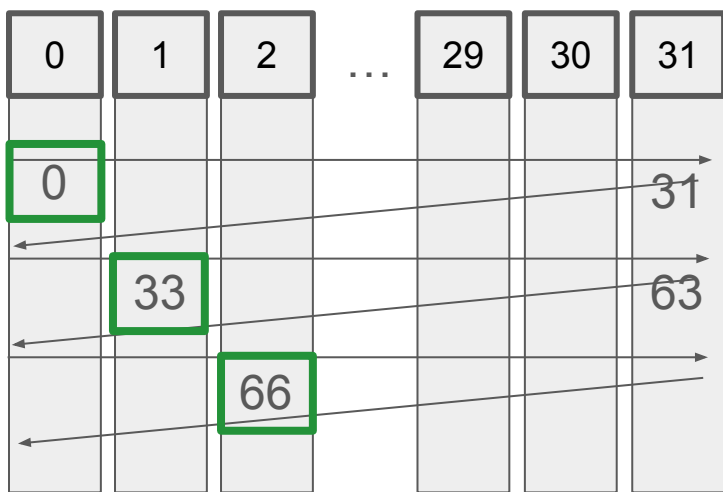


Что будет если
Tile - 33x32?



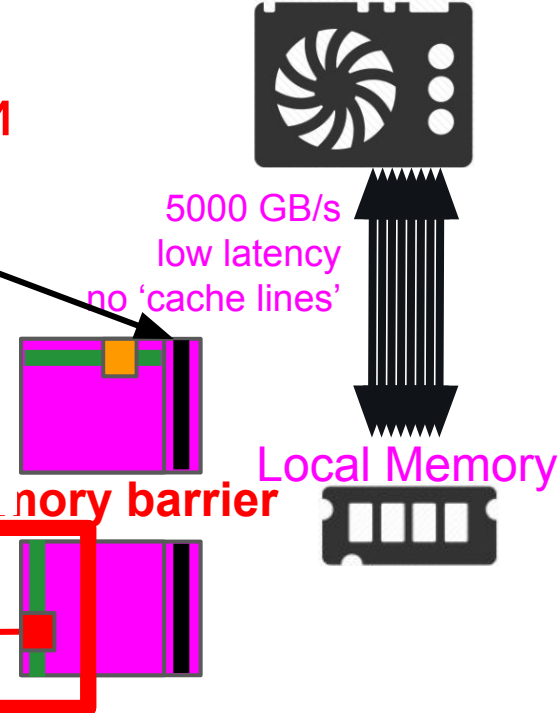
- 1) Каждый WorkItem считал элемент: VRAM -> Local Memory
- 2) Дождались всю WorkGroup: memory barrier
- 3) Каждый WorkItem записал элемент: Local Memory -> VRAM

Есть ли здесь bank conflicts?



Что будет если
Tile - 33x32?

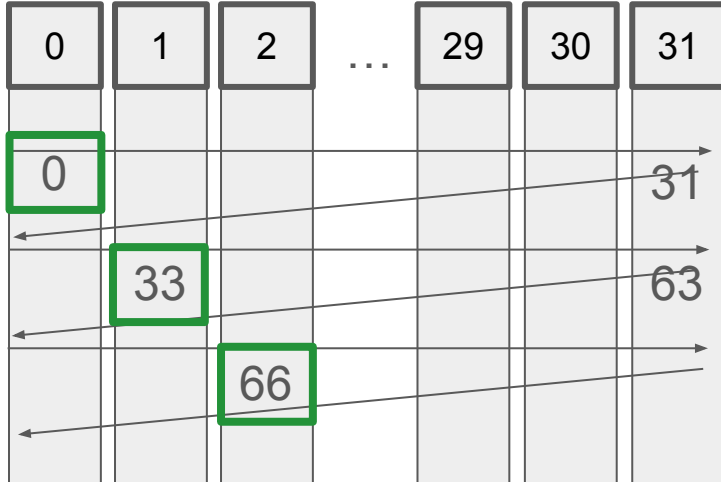
Можно ли без лишней
local memory?



coalesced WRITE

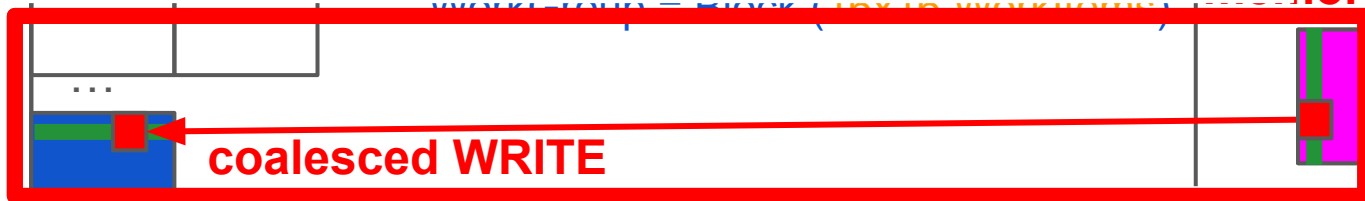
- 1) Каждый WorkItem считал элемент: VRAM -> Local Memory
- 2) Дождались всю WorkGroup: memory barrier
- 3) Каждый WorkItem записал элемент: Local Memory -> VRAM

Есть ли здесь bank conflicts?



Что будет если
Tile - **33x32?**

А на что влияет
объем используемой
Local Memory?



- 1) Каждый WorkItem считал элемент: **VRAM** ->
- 2) Дождались всю WorkGroup: **memory barrier**
- 3) Каждый WorkItem записал элемент: **Local M**

Есть ли здесь **bank conflicts**?



А на что влияет
объем используемой
Local Memory?
Occupancy!



RTX 3090 (Ampere) - per **SM**:

- **128 KB** Local Memory
- 4 x 16384 x 32-bit registers
- = **256 KB** Register File

А на что влияет
объем используемой
Local Memory?
Occupancy!



SM - Streaming Multiprocessor



RTX 3090 (Ampere) - per SM:

- **128 KB** Local Memory
- 4 x 16384 x 32-bit registers
- = **256 KB** Register File

[DEPRECATED] Excel spreadsheet based occupancy calculator is deprecated.

[Occupancy calculator is now available in Nsight Compute](#)

CUDA Occupancy Calculator

Just follow steps 1, 2, and 3 below! (or click here for help)

1) Select Compute Capability (click):	8.6
1.b) Select Shared Memory Size Config (bytes)	65536
1.c) Select CUDA version	11.1

2) Enter your resource usage:	
Threads Per Block	256
Registers Per Thread	32
User Shared Memory Per Block (bytes)	2048

(Don't edit anything below this line)

3) GPU Occupancy Data is displayed here and in the graphs:	
Active Threads per Multiprocessor	1536
Active Warps per Multiprocessor	48
Active Thread Blocks per Multiprocessor	6
Occupancy of each Multiprocessor	100%

Physical Limits for GPU Compute Capability:	8.6
Threads per Warp	32
Max Warps per Multiprocessor	48
Max Thread Blocks per Multiprocessor	16
Max Threads per Multiprocessor	1536
Maximum Thread Block Size	1024
Registers per Multiprocessor	65536
Max Registers per Thread Block	65536
Max Registers per Thread	255
Shared Memory per Multiprocessor (bytes)	65536
Max Shared Memory per Block	65536
Register allocation unit size	256
Register allocation granularity	warp
Shared Memory allocation unit size	128
Warp allocation granularity	4
Shared Memory Per Block (bytes) (CUDA runtime use)	1024

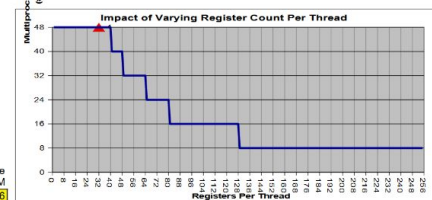
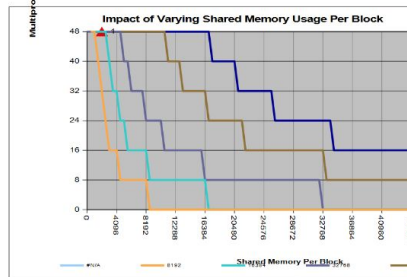
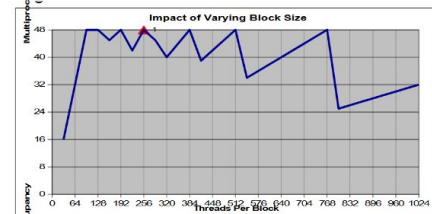
Allocated Resources	Per Block	Limit Per SM	= Allocatable Blocks Per SM
Warps (Threads Per Block / Threads Per Warp)	8	48	6
Registers (Warp limit per SM due to per-warp reg count)	8	64	8
Shared Memory (Bytes)	2048	65536	32

Note: SM is an abbreviation for (Streaming) Multiprocessor

Maximum Thread Blocks Per Multiprocessor	Blocks/SM	* Warps/Block = Warps/SM	
Limited by Max Warps or Max Blocks per Multiprocessor	6	8	48

[Click Here for detailed instructions on how to use this occupancy calculator.](#)
[For more information on NVIDIA CUDA, visit <http://developer.nvidia.com/cuda>](#)

Your chosen resource usage is indicated by the red triangle on the graphs. The other data points represent the range of possible block sizes, register counts, and shared memory allocation.





Compute Capability:

8.6

Threads Per Block:

256

Shared Memory Size Config (bytes):

102400

Registers Per Thread:

32

Global Load Cache Mode:

L1+L2 (ca)

User Shared Memory Per Block (bytes):

2048

Block Barriers:

1

☒ Apply Automatically

Apply

Reset



The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.



Compute Capability:	8.6	Threads Per Block:	<input type="range"/>	256
Shared Memory Size Config (bytes):	102400	Registers Per Thread:	<input type="range"/>	32
Global Load Cache Mode:	L1+L2 (ca)	User Shared Memory Per Block (bytes):	<input type="range"/>	2048
		Block Barriers:	<input type="range"/>	1
☒ Apply Automatically Apply Reset				

i The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.

RTX 3090 (Ampere) - Compute Capability 8.6



Compute Capability:	8.6	Threads Per Block:	256
Shared Memory Size Config (bytes):	102400	Registers Per Thread:	32
Global Load Cache Mode:	L1+L2 (ca)	User Shared Memory Per Block (bytes):	2048
		Block Barriers:	1
☒ Apply Automatically Apply Reset			

i The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.

RTX 3090 (Ampere) - Compute Capability 8.6
WorkGroup size - 256



Compute Capability:	8.6	Threads Per Block:	<input type="range"/>	256
Shared Memory Size Config (bytes):	102400	Registers Per Thread:	<input type="range"/>	32
Global Load Cache Mode:	L1+L2 (ca)	User Shared Memory Per Block (bytes):	<input type="range"/>	2048
		Block Barriers:	<input type="range"/>	1

☒ Apply Automatically

i The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.

RTX 3090 (Ampere) - Compute Capability 8.6

WorkGroup size - 256

Registers Per Thread - 32



Compute Capability:	8.6	Threads Per Block:	<input type="range"/>	256
Shared Memory Size Config (bytes):	102400	Registers Per Thread:	<input type="range"/>	32
Global Load Cache Mode:	L1+L2 (ca)	User Shared Memory Per Block (bytes):	<input type="range"/>	2048
		Block Barriers:	<input type="range"/>	1

☒ Apply Automatically

i The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.

RTX 3090 (Ampere) - Compute Capability **8.6**

WorkGroup size - 256

Registers Per Thread - 32

Local memory Per Thread - 8 bytes



NVIDIA Nsight

Occupancy Calculator

Compute Capability: 8.6 Threads Per Block: 256

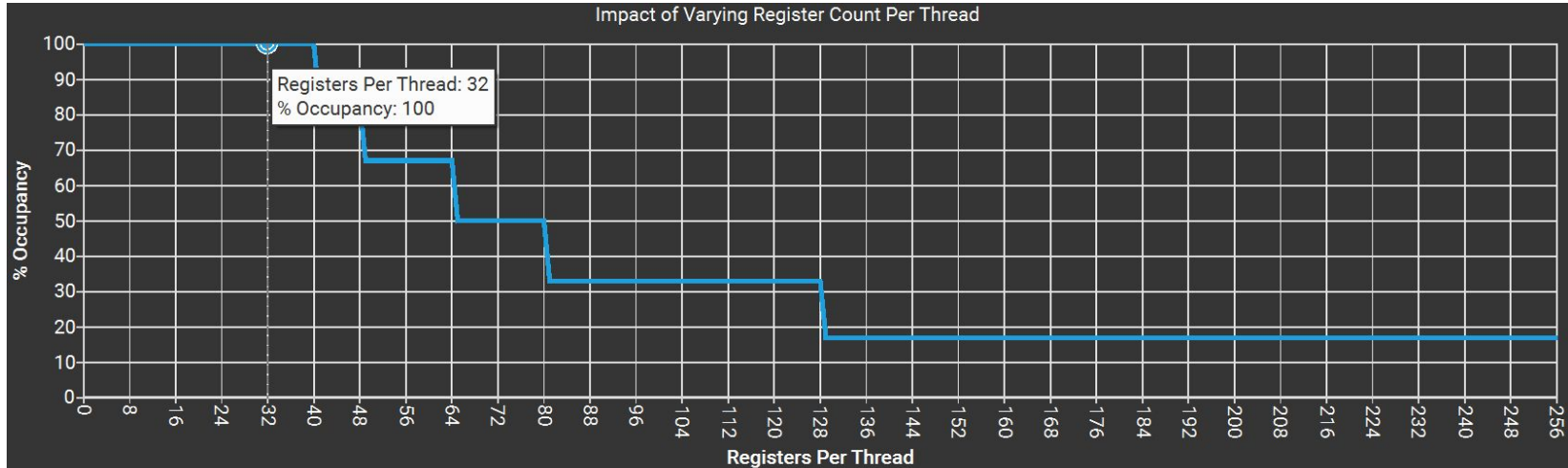
Shared Memory Size Config (bytes): 102400 Registers Per Thread: 32

Global Load Cache Mode: L1+L2 (ca) User Shared Memory Per Block (bytes): 2048

Block Barriers: 1

☒ Apply Automatically

i The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.





NVIDIA Nsight

Occupancy Calculator

Compute Capability: 8.6 Threads Per Block: 256

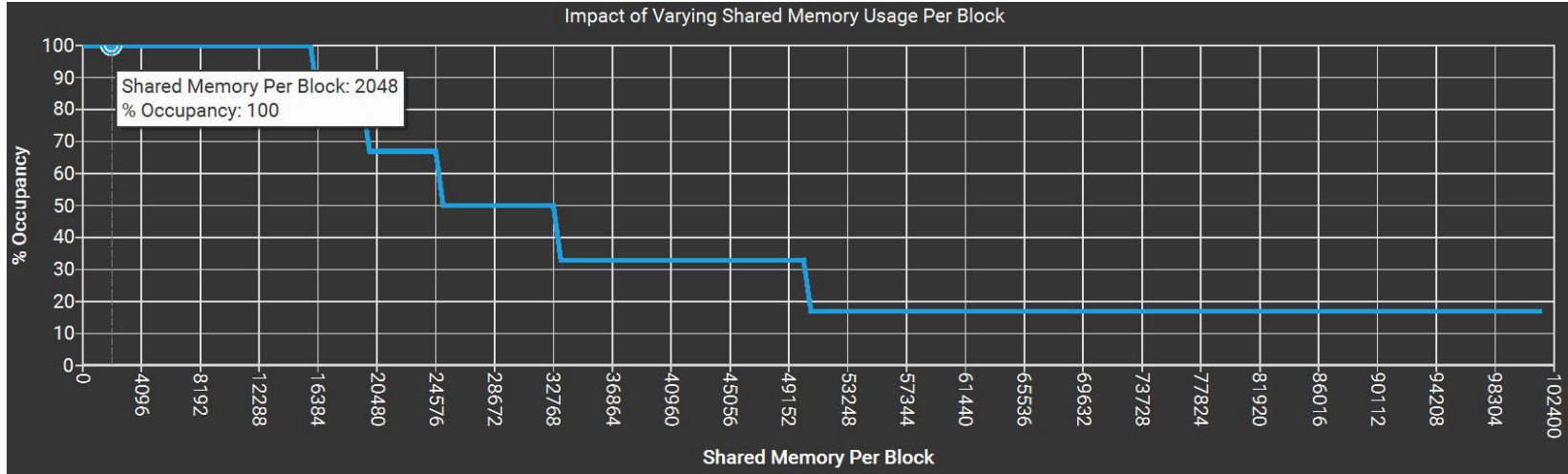
Shared Memory Size Config (bytes): 102400 Registers Per Thread: 32

Global Load Cache Mode: L1+L2 (ca) User Shared Memory Per Block (bytes): 2048

Block Barriers: 1

✓ Apply Automatically Apply Reset

i The CUDA Occupancy Calculator allows you to compute the multiprocessor occupancy of a GPU by a given CUDA kernel. Occupancy is the ratio of active warps on an SM to the maximum number of active warps supported by the SM. The launch parameters can be configured with the input widget to the left. This tool also provides various important statistics such as resource allocation, occupancy limiters, GPU Data, etc in the form of tables and graphs.

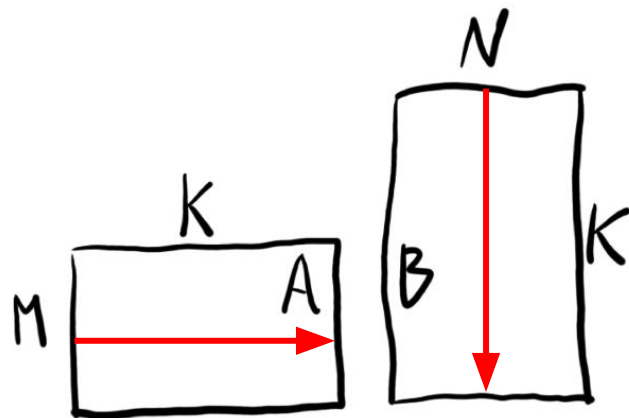




Глава 2: Умножение матриц

local memory

Умножение матриц



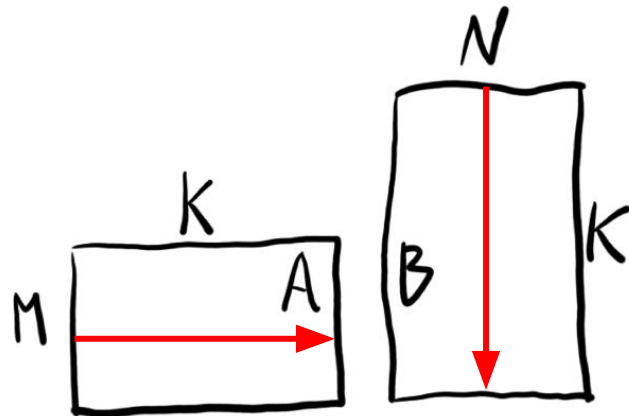
$$C = A \times B$$

Below the equation, a square represents the resulting matrix C. The left side is labeled 'M' and the top side is labeled 'N'. Inside the square, there is a red circle with a dot in the center, and the letter 'C' is written in the bottom-left corner.

Умножение матриц

Как выглядит:

- WorkRange?
- Задача WorkItem?

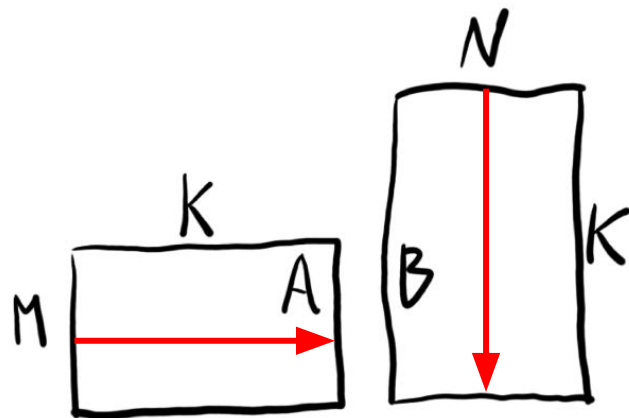


$$C = A \times B$$

Умножение матриц

Как выглядит:

- WorkRange?
- Задача WorkItem?
- WorkGroup?



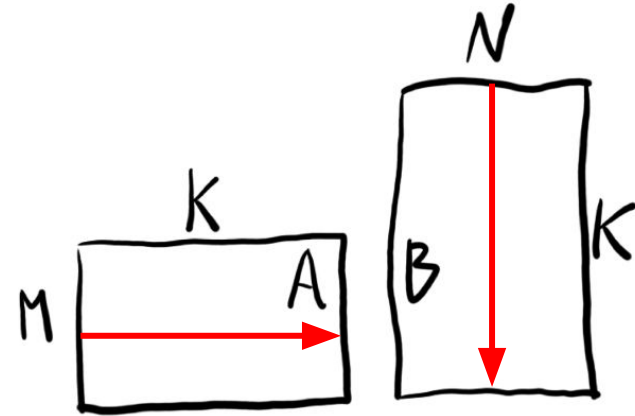
$$C = A \times B$$

Умножение матриц

Сколько у нас вычислений?

Сколько у нас чтений/записей данных?

Какая пропорция?



$$C = A \times B$$

Diagram illustrating the resulting matrix C. Matrix C is labeled with dimensions M (rows) and N (columns). A red circle with a dot inside is shown within the matrix C, representing a single element of the result.

Умножение матриц

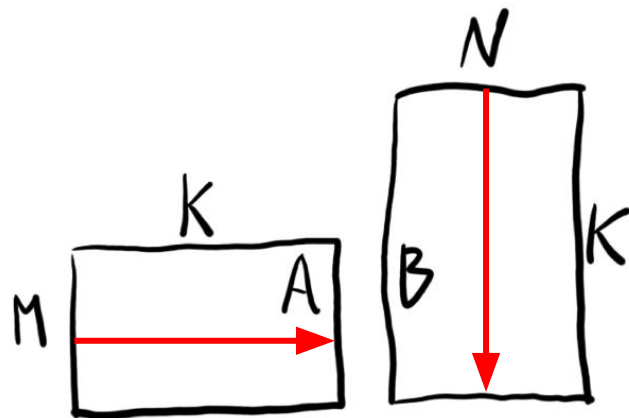
Сколько у нас вычислений?

$O(N \times M \times K)$

Сколько у нас чтений/записей данных?

$O(N \times M \times K)$

Какая пропорция?



$$C = A \times B$$

Умножение матриц

Сколько у нас вычислений?

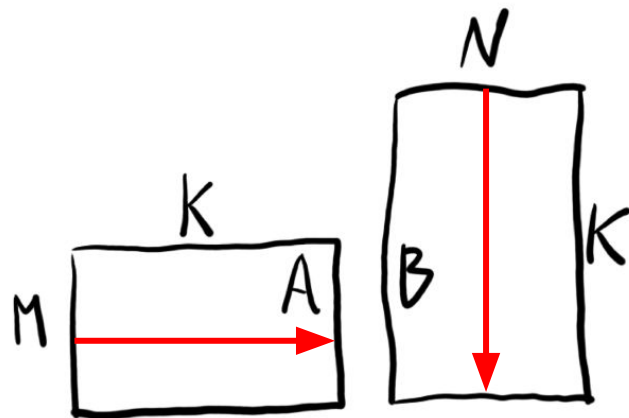
$$O(N \times M \times K)$$

Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 Это хороший результат?



$$C = A \times B$$

Умножение матриц

Сколько у нас вычислений?

$$O(N \times M \times K)$$

Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 Это хороший результат?

$100 \cdot 10^{12}$ FLOPS

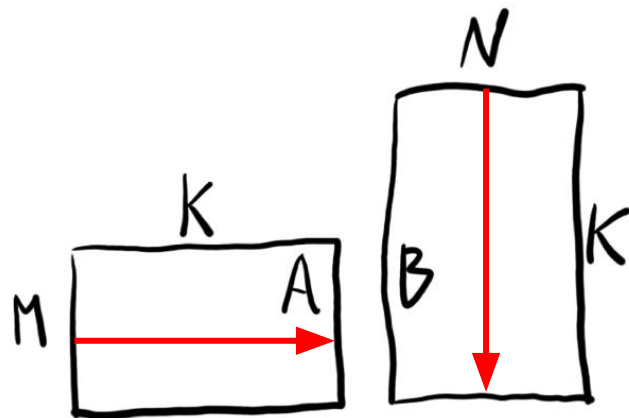


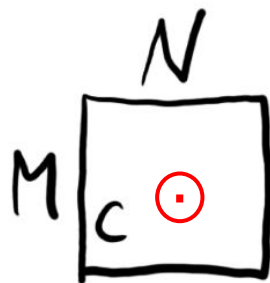
400:1

1000 GB/s



VRAM - GDDR6



$$C = A \times B$$


Умножение матриц

Сколько у нас вычислений?

$$O(N \times M \times K)$$

Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 **Memory-bound!**

100 · 10¹² FLOPS



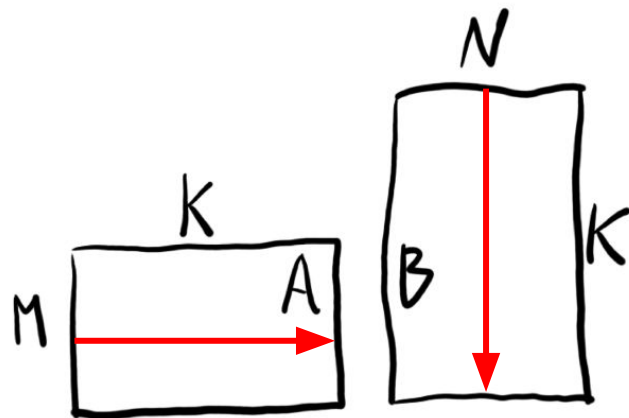
400:1



1000 GB/s



VRAM - GDDR6



$$C = A \times B$$

Умножение матриц

Сколько у нас вычислений?

$$O(N \times M \times K)$$

Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 **Memory-bound!**

$100 \cdot 10^{12}$ FLOPS



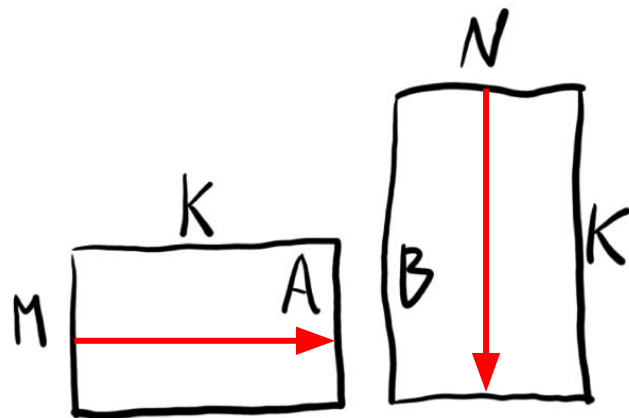
400:1



1000 GB/s



VRAM - GDDR6



$$C = A \times B$$
A hand-drawn diagram of matrix C, a square with width N and height M, with a red circle in the center.

Как увеличить объем вычислений на считанный байт?

Умножение матриц

Сколько у нас вычислений?

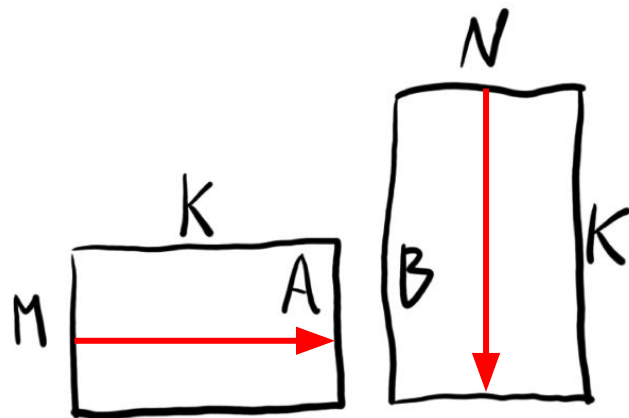
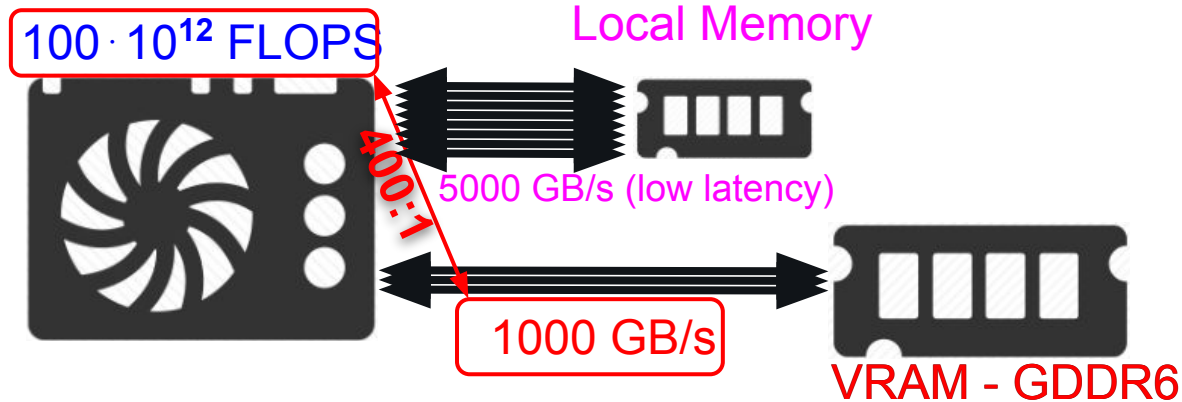
$$O(N \times M \times K)$$

Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 **Memory-bound!**

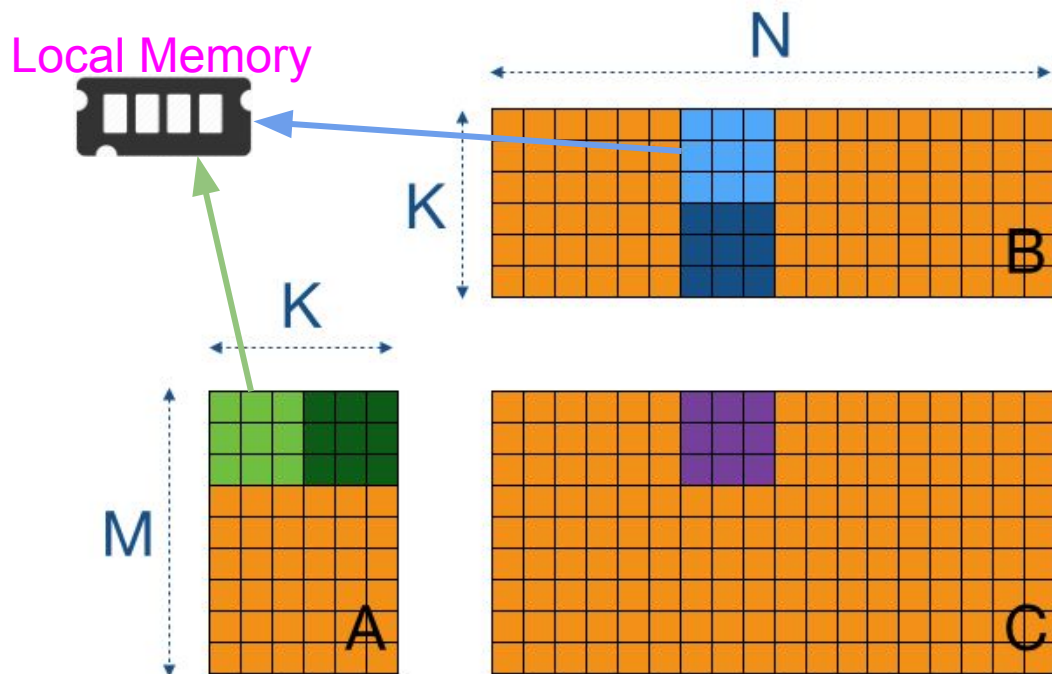


$$C = A \times B$$

Hand-drawn diagram showing the resulting matrix C, which is a square with width N and height M, with a red circle in the center.

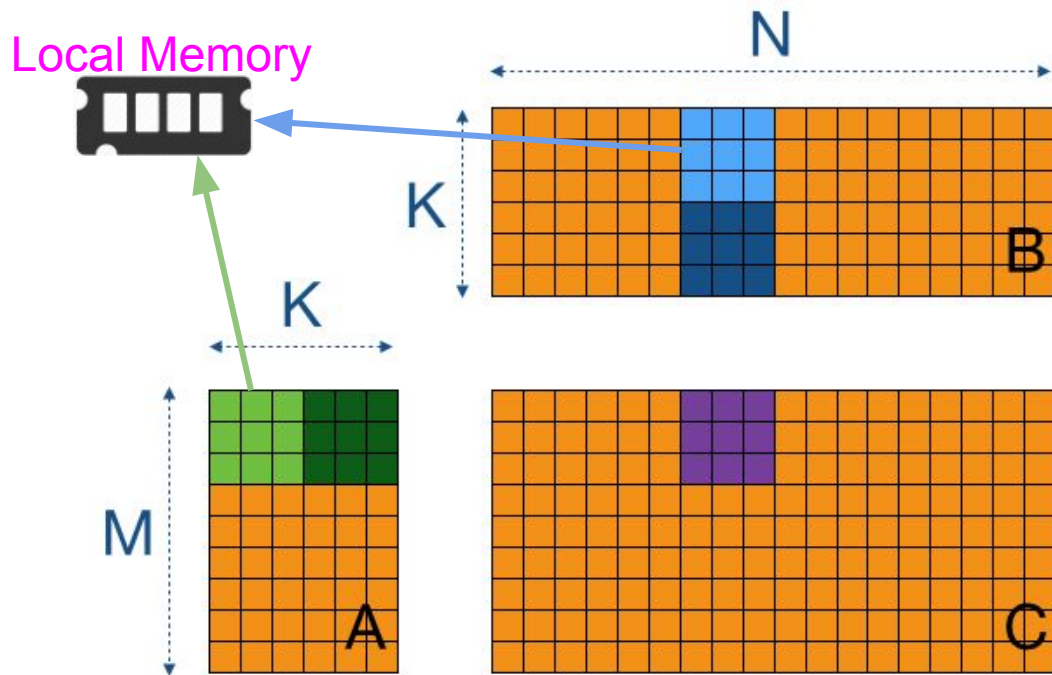
Как увеличить объем вычислений на считанный байт?

Умножение матриц



Умножение матриц

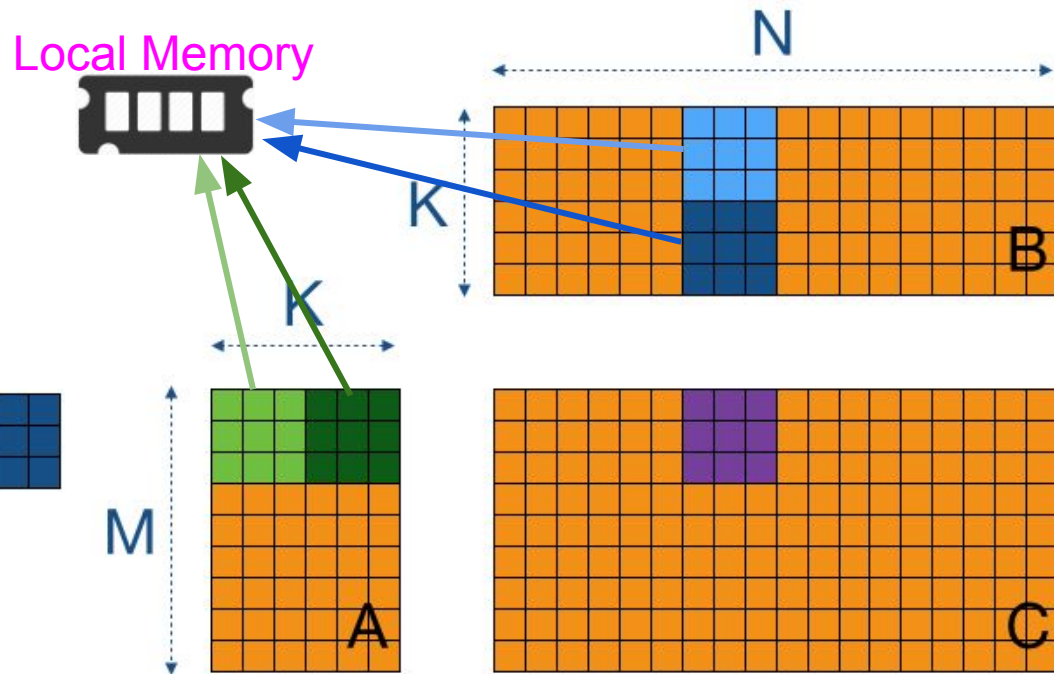
$$\begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{bmatrix} = \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{bmatrix} \times \begin{bmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{bmatrix} + \dots$$



Умножение матриц

$$\begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \times \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} + \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \times \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix}$$

За счет чего это ускоряет?



Умножение матриц

Сколько у нас вычислений?

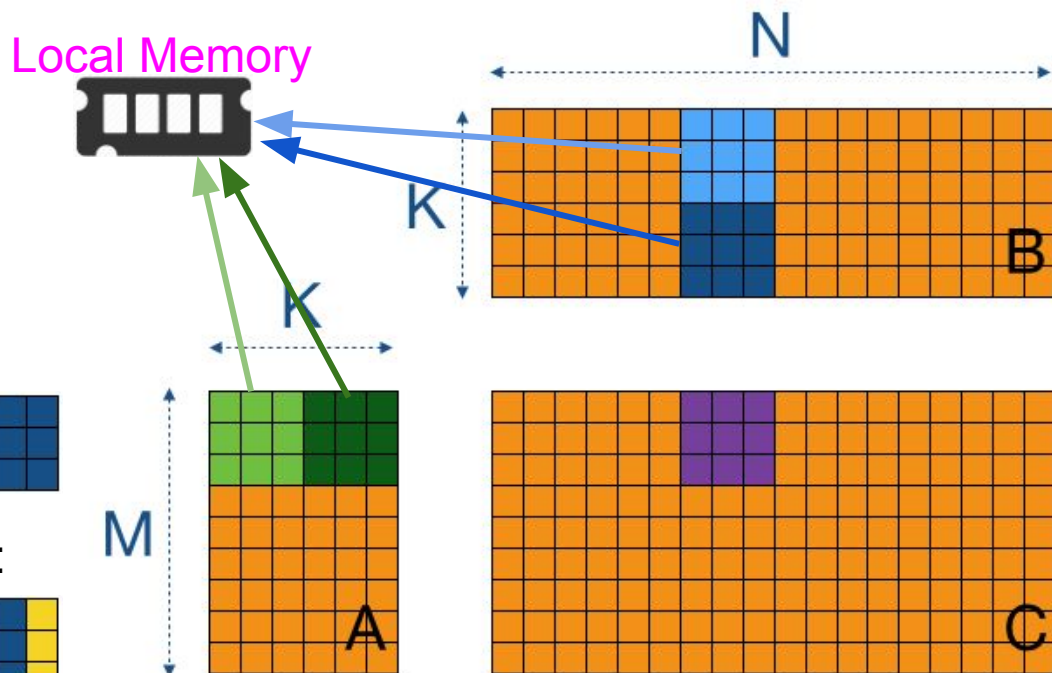
Сколько у нас чтений/записей?

Какая пропорция?

$$\begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} = \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \times \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} + \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \times \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix}$$

Переиспользование данных:

$$\left\{ \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \right\}_{32} = \left\{ \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \right\}_{32} \times \left\{ \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \right\}_{32} + \left\{ \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \right\}_{32} \times \left\{ \begin{bmatrix} \times & \times & \times \\ \times & \times & \times \\ \times & \times & \times \end{bmatrix} \right\}_{32}$$



Умножение матриц

Сколько у нас вычислений?

$$O(N \times M \times K)$$

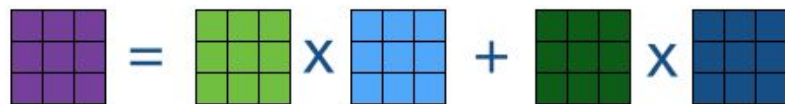
Сколько у нас чтений/записей?

$$O(N \times M \times K /$$

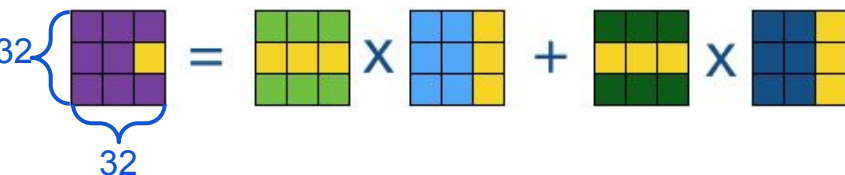
32)

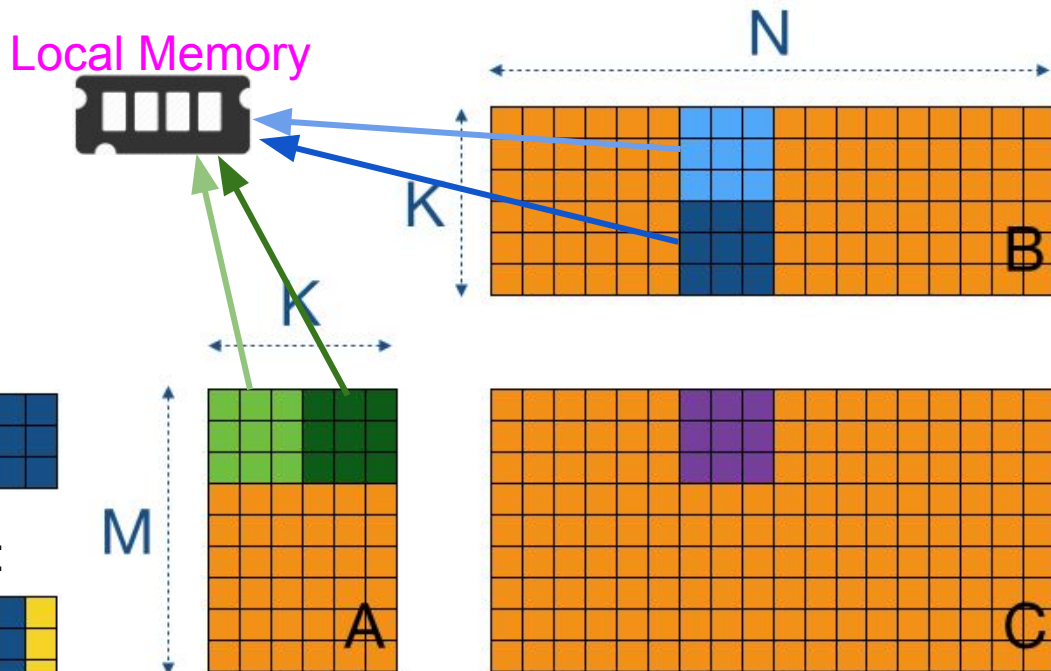
Какая пропорция?

32:1


$$\begin{bmatrix} \text{purple} \end{bmatrix} = \begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{blue} \end{bmatrix} + \begin{bmatrix} \text{dark green} \end{bmatrix} \times \begin{bmatrix} \text{dark blue} \end{bmatrix}$$

Переиспользование данных:


$$\begin{bmatrix} \text{purple} \end{bmatrix} = \begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{blue} \end{bmatrix} + \begin{bmatrix} \text{dark green} \end{bmatrix} \times \begin{bmatrix} \text{dark blue} \end{bmatrix}$$



Умножение матриц

Сколько у нас вычислений?

$$O(N \times M \times K)$$

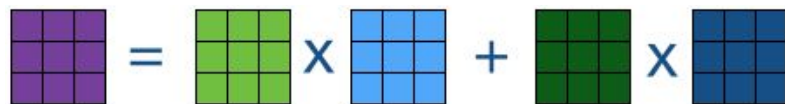
Сколько у нас чтений/записей?

$$O(N \times M \times K /$$

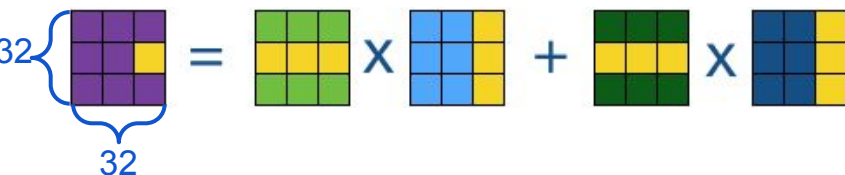
32)

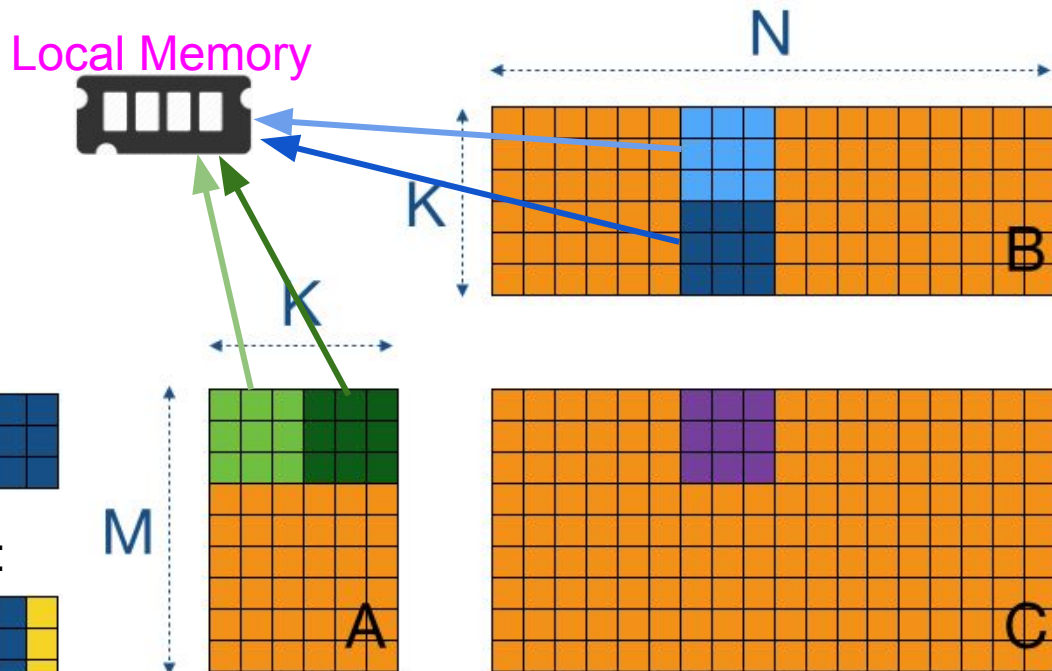
Какая пропорция?

32:1


$$\begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{blue} \end{bmatrix} + \begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{blue} \end{bmatrix} = \begin{bmatrix} \text{purple} \end{bmatrix}$$

Переиспользование данных:


$$\begin{bmatrix} \text{purple} \end{bmatrix} = \begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{blue} \end{bmatrix} + \begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{blue} \end{bmatrix}$$



Как пойти еще дальше? Как еще увеличить пропорцию?

Умножение матриц



Неравная битва за гигафлопсы при умножении матриц
(хорошо описанная аналитика, профилирование, оптимизация):

- AMD RDNA3 - <https://seb-v.github.io/optimization/update/2025/01/20/Fast-GPU-Matrix-multiplication.html>
- NVIDIA Kepler - <https://cnugeteren.github.io/tutorial/pages/page15.html>
- <https://siboehm.com/articles/22/CUDA-MMM>

Перерыв!





Глава 3: Умножение матриц

Tensor Cores, WMMA

Умножение матриц

Tensor Cores

	VRAM <i>TB/s</i>	FP 32 <i>TFlops</i>	FP 16 <i>TFlops</i>	FP 16 (tensor) <i>TFlops</i>
RTX 3090	0.93	29	29	142
RTX 4090	1.00	73	73	330
RTX 5090	1.79	105	105	419
Tesla H100	3.35	67	268 (4:1)	990 (16:1)



Умножение матриц

Tensor Cores

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 or FP32

4x4



Умножение матриц

Tensor Cores

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 4x4 FP16 or FP32

Tensor Cores (CUDA kernel)

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,\dots} & A_{0,15} \\ A_{1,0} & A_{1,1} & A_{1,\dots} & A_{1,15} \\ A_{\dots,0} & A_{\dots,1} & A_{\dots,\dots} & A_{\dots,15} \\ A_{15,0} & A_{15,1} & A_{15,\dots} & A_{15,15} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,\dots} & B_{0,15} \\ B_{1,0} & B_{1,1} & B_{1,\dots} & B_{1,15} \\ B_{\dots,0} & B_{\dots,1} & B_{\dots,\dots} & B_{\dots,15} \\ B_{15,0} & B_{15,1} & B_{15,\dots} & B_{15,15} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,\dots} & C_{0,15} \\ C_{1,0} & C_{1,1} & C_{1,\dots} & C_{1,15} \\ C_{\dots,0} & C_{\dots,1} & C_{\dots,\dots} & C_{\dots,15} \\ C_{15,0} & C_{15,1} & C_{15,\dots} & C_{15,15} \end{pmatrix}$$

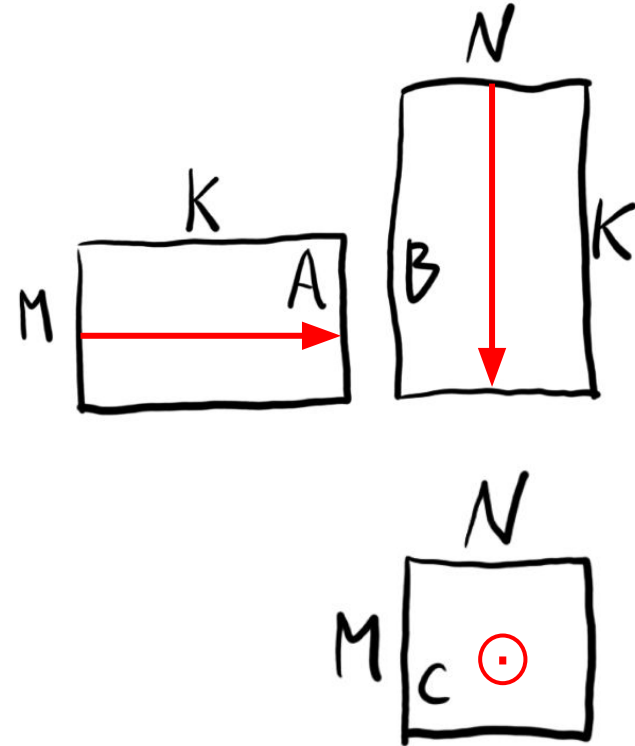
FP16 or FP32 FP16 FP16 16x16 FP16 or FP32



69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)

75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {

$$C = \alpha A * B + \beta C$$



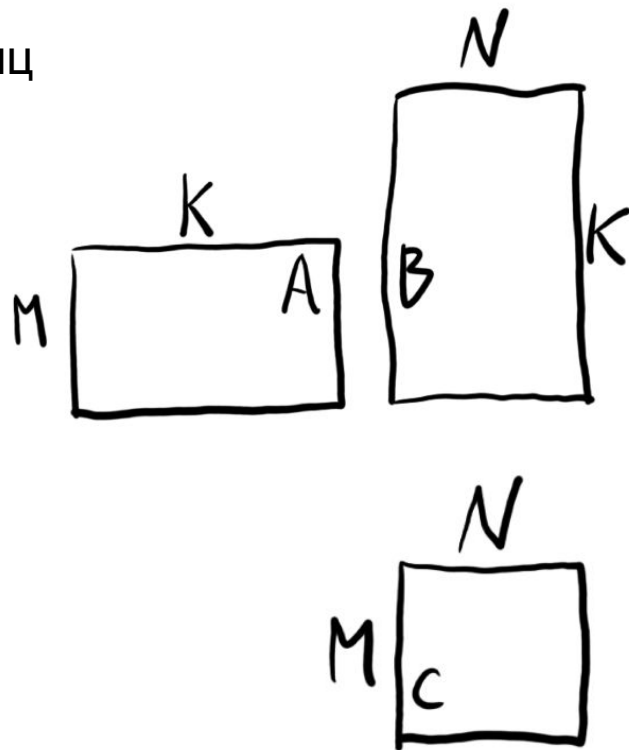
```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;

```

Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

$$C = \alpha A * B + \beta C$$



```
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
```

```
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
```

```
85 // Declare the fragments
```

```
86 wmma::fragment(wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
```

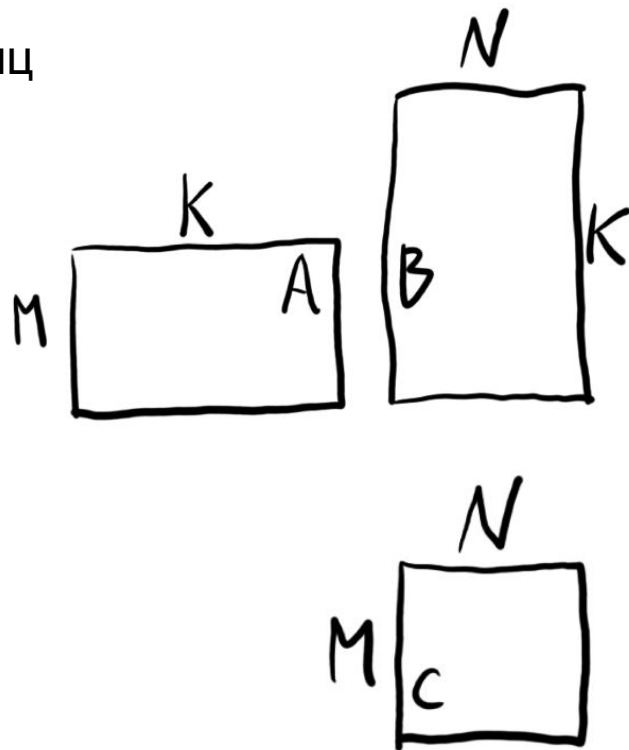
```
87 wmma::fragment(wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
```

```
88 wmma::fragment(wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
```

```
89 wmma::fragment(wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
```

Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

$$C = \alpha A * B + \beta C$$




```

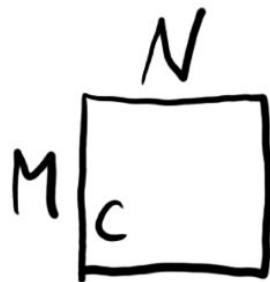
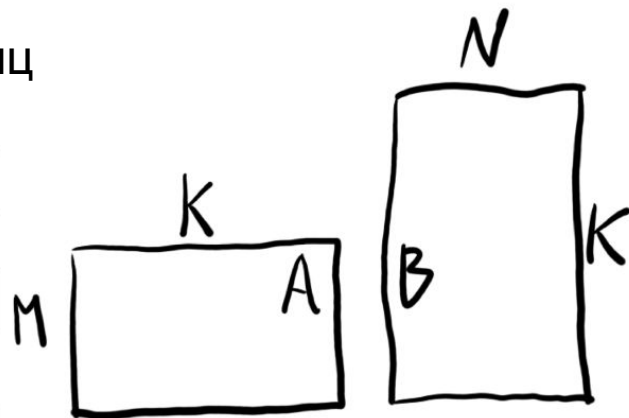
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;

```

Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
<u>__half</u>	<u>__half</u>	float	<u>16x16x16</u>
__half	__half	float	32x8x16
__half	__half	float	8x32x16
__half	__half	__half	16x16x16
__half	__half	__half	32x8x16
__half	__half	__half	8x32x16
unsigned char	unsigned char	int	16x16x16
...	...	...	...

$$C = \alpha A * B + \beta C$$



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/#wmma-type-sizes>

<https://github.com/NVIDIA-developer-blog/code-samples/blob/master/posts/tensor-cores/simpleTensorCoreGEMM.cu>

```

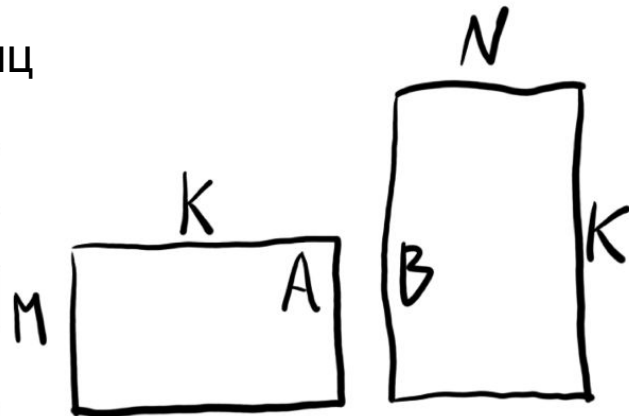
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;

```

$$C = \alpha A * B + \beta C$$

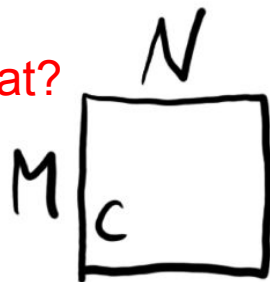
Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
<u>__half</u>	<u>__half</u>	float	<u>16x16x16</u>
__half	__half	float	32x8x16
__half	__half	float	8x32x16
__half	__half	__half	16x16x16
__half	__half	__half	32x8x16
__half	__half	__half	8x32x16
unsigned char	unsigned char	int	16x16x16
...	...	...	...



Почему A и B - half?

Но C и аккумулятор - float?



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/#wmma-type-sizes>

<https://github.com/NVIDIA-developer-blog/code-samples/blob/master/posts/tensor-cores/simpleTensorCoreGEMM.cu>

```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;

```

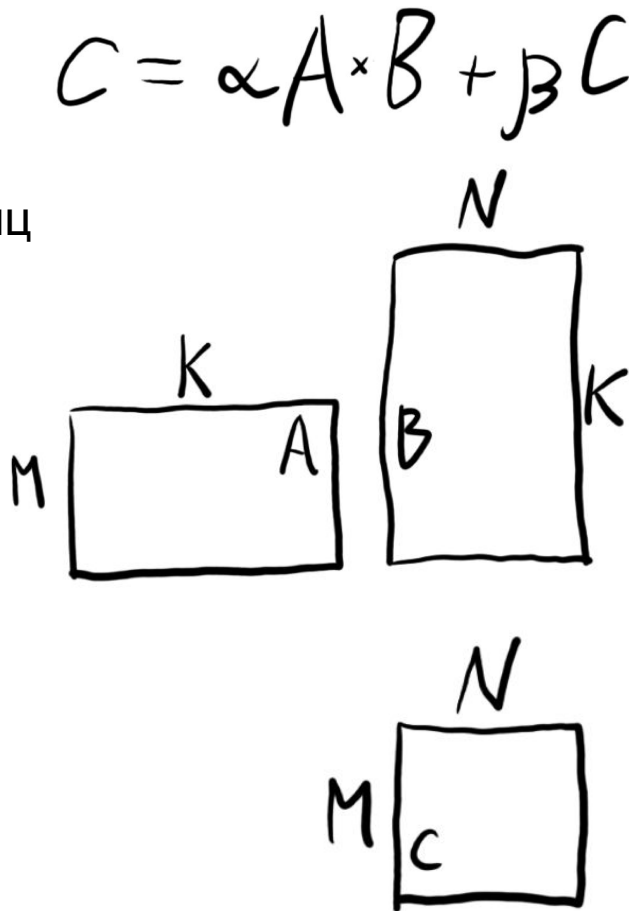
Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

Почему A и B - half?

Но C и аккумулятор - float?

Подсказка:

- A и B перемножаются
- аккумулятор суммируется



```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;

```

Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

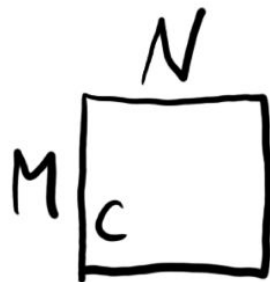
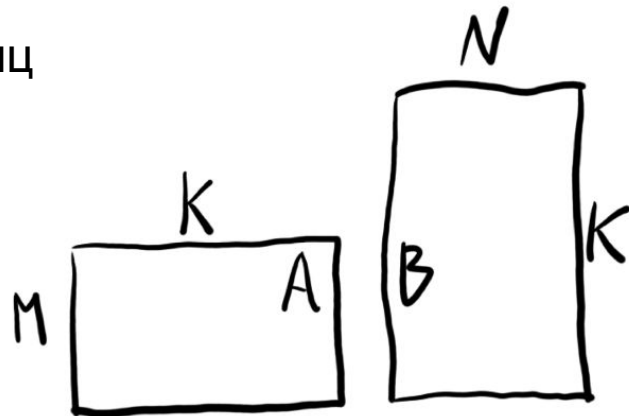
Почему A и B - half?
 Но C и аккумулятор - float?

Что будет если
сложить огромное
 и малое число?



$$\pm (1 + \text{mantissa}) \times 2^{(\text{exponent} - 127)}$$

$$C = \alpha A \times B + \beta C$$



```
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
```

```
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {  
85     // Declare the fragments  
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;  
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;  
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;  
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
```

Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate

Почему A и B - half?
Но C и аккумулятор - float?

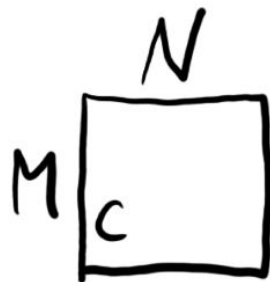
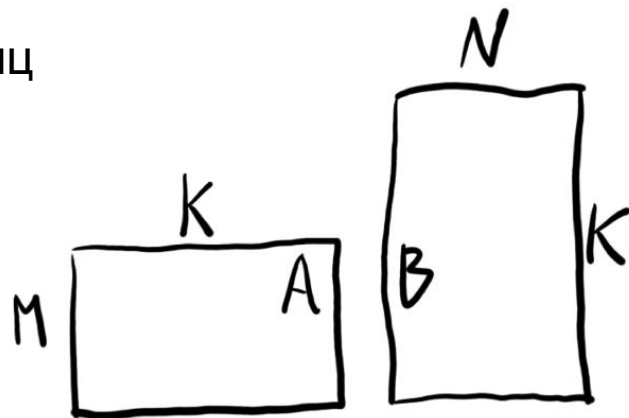
Что будет если
сложить огромное
и малое число?



Что будет если
умножить огромное
и малое число?

$$\pm (1 + \text{mantissa}) \times 2^{(\text{exponent} - 127)}$$

$$C = \alpha A * B + \beta C$$



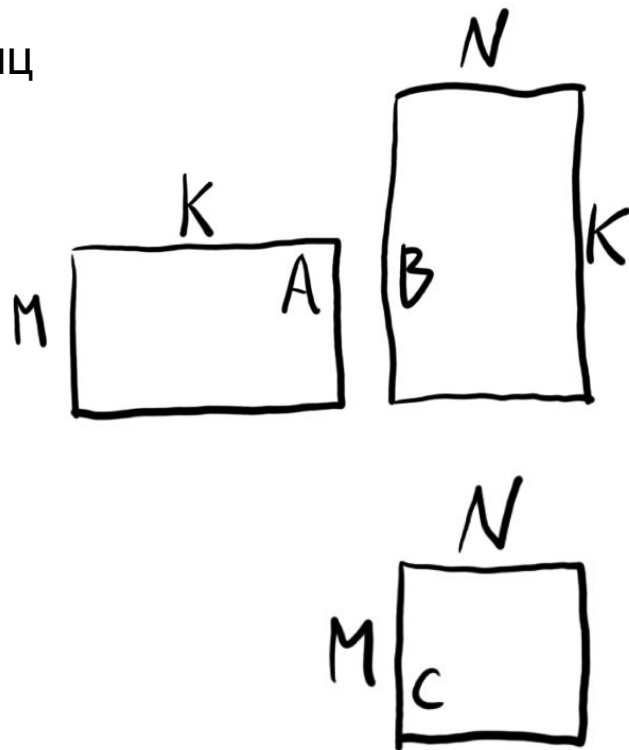
```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;

```

**Общие на весь warp 16x16 фрагменты матриц
WMMA = Warp Matrix Multiply-Accumulate**

$$C = \alpha A * B + \beta C$$



69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)

75 `__global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {`

85 `// Declare the fragments`

86 `wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;`

87 `wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;`

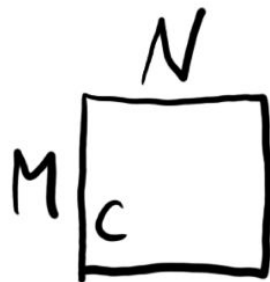
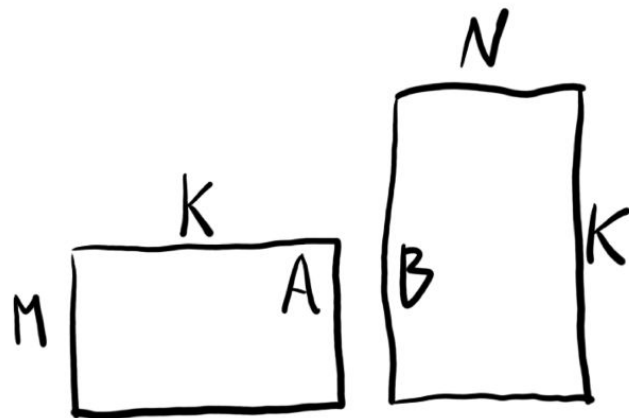
88 `wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;`

89 `wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;`

91 `wmma::fill_fragment(acc_frag, 0.0f);`

 16x16
acc_frag

$$C = \alpha A * B + \beta C$$

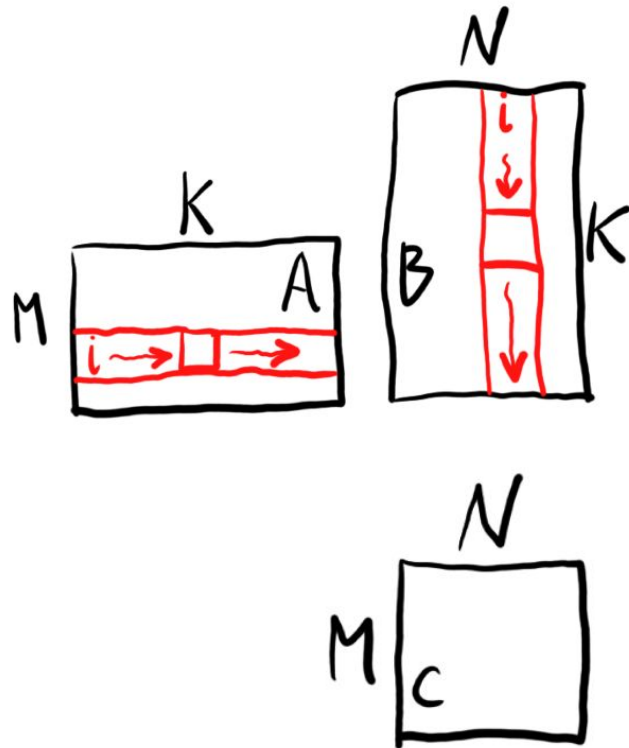


69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)

```
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {  
85     // Declare the fragments  
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;  
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;  
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;  
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;  
91     wmma::fill_fragment(acc_frag, 0.0f);  
94     for (int i = 0; i < K; i += WMMA_K) {
```

 16x16
acc_frag

$$C = \alpha A * B + \beta C$$

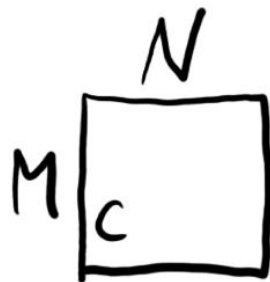
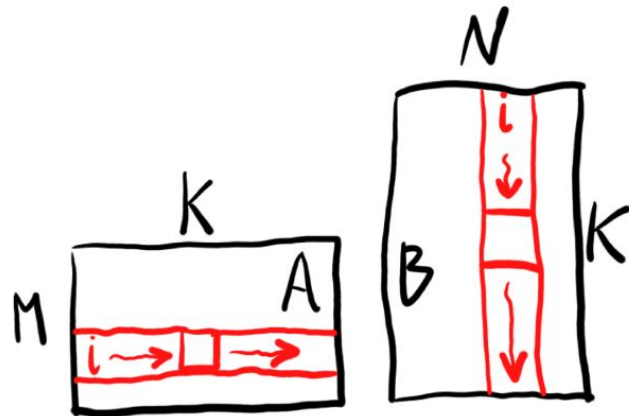


69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)

```
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {  
85     // Declare the fragments  
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;  
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;  
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;  
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;  
  
91     wmma::fill_fragment(acc_frag, 0.0f);  
94     for (int i = 0; i < K; i += WMMA_K) {  
95         int aRow = warpM * WMMA_M;  
96         int aCol = i;  
  
97  
98         int bRow = i;  
99         int bCol = warpN * WMMA_N;  
  
100  
101         // Bounds checking  
102         if (aRow < M && aCol < K && bRow < K && bCol < N) {  
103             // Load the inputs  
104             wmma::load_matrix_sync(a_frag, a + aRow + aCol * lda, lda);  
105             wmma::load_matrix_sync(b_frag, b + bRow + bCol * ldb, ldb);
```

 16x16
acc_frag

$$C = \alpha A * B + \beta C$$

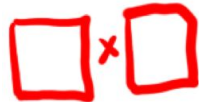


```

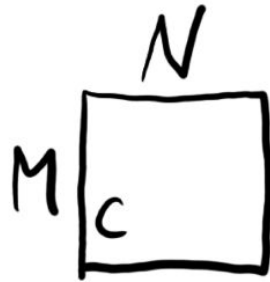
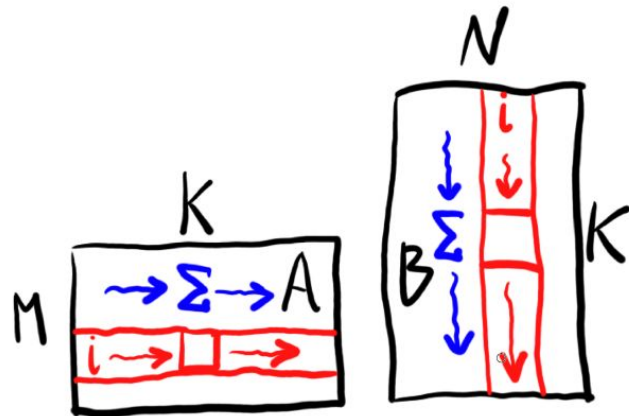
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
71 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
73     // Declare the fragments
74     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
75     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
76     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
77     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
79     wmma::fill_fragment(acc_frag, 0.0f);
81     for (int i = 0; i < K; i += WMMA_K) {
82         int aRow = warpM * WMMA_M;
83         int aCol = i;
85         int bRow = i;
86         int bCol = warpN * WMMA_N;
88         // Bounds checking
89         if (aRow < M && aCol < K && bRow < K && bCol < N) {
91             // Load the inputs
92             wmma::load_matrix_sync(a_frag, a + aRow + aCol * lda, lda);
93             wmma::load_matrix_sync(b_frag, b + bRow + bCol * ldb, ldb);
95             // Perform the matrix multiplication
96             wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
98         }
99     }
100 }

```

 16x16
acc_frag

16x16
 → acc_frag

$$C = \alpha A * B + \beta C$$

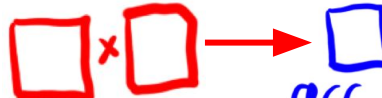


```

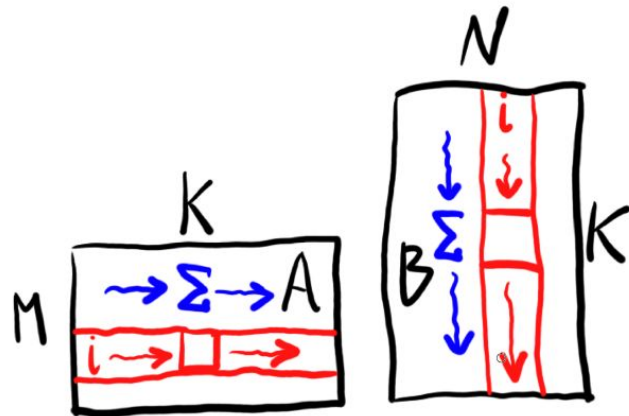
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
71
72 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
73
74     // Declare the fragments
75     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
76     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
77     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
78     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
79
80     wmma::fill_fragment(acc_frag, 0.0f);
81
82     for (int i = 0; i < K; i += WMMA_K) {
83         int aRow = warpM * WMMA_M;
84         int aCol = i;
85
86         int bRow = i;
87         int bCol = warpN * WMMA_N;
88
89         // Bounds checking
90         if (aRow < M && aCol < K && bRow < K && bCol < N) {
91             // Load the inputs
92             wmma::load_matrix_sync(a_frag, a + aRow + aCol * lda, lda);
93             wmma::load_matrix_sync(b_frag, b + bRow + bCol * ldb, ldb);
94
95             // Perform the matrix multiplication
96             wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
97         }
98     }
99 }

```

 16x16
acc_frag

16x16
 acc_frag

$$C = \alpha A * B + \beta C$$



Чем этот код отличается от классического умножения в локальной памяти?

```

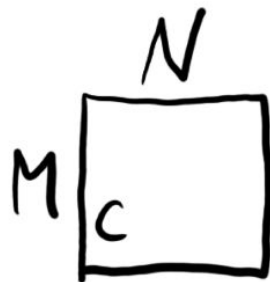
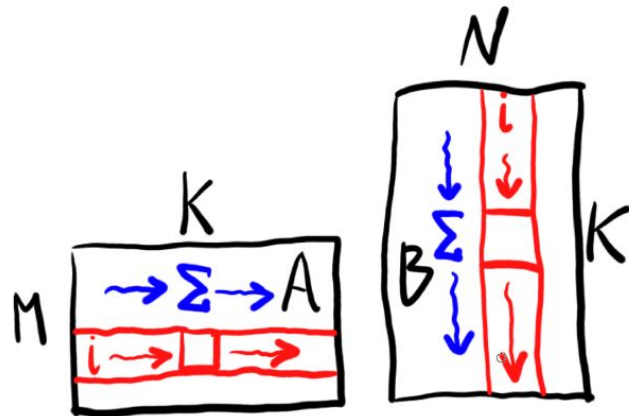
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
71
72 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
73
74     // Declare the fragments
75     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
76     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
77     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
78     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
79
80     wmma::fill_fragment(acc_frag, 0.0f);
81
82     for (int i = 0; i < K; i += WMMA_K) {
83         int aRow = warpM * WMMA_M;
84         int aCol = i;
85
86         int bRow = i;
87         int bCol = warpN * WMMA_N;
88
89         // Bounds checking
90         if (aRow < M && aCol < K && bRow < K && bCol < N) {
91             // Load the inputs
92             wmma::load_matrix_sync(a_frag, a + aRow + aCol * lda, lda);
93             wmma::load_matrix_sync(b_frag, b + bRow + bCol * ldb, ldb);
94
95             // Perform the matrix multiplication
96             wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
97         }
98     }
99 }

```

16x16
acc_frag

16x16
acc_frag

$$C = \alpha A * B + \beta C$$



Что нам осталось сделать?

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)

75 `__global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {`

85 `// Declare the fragments`

86 `wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;`

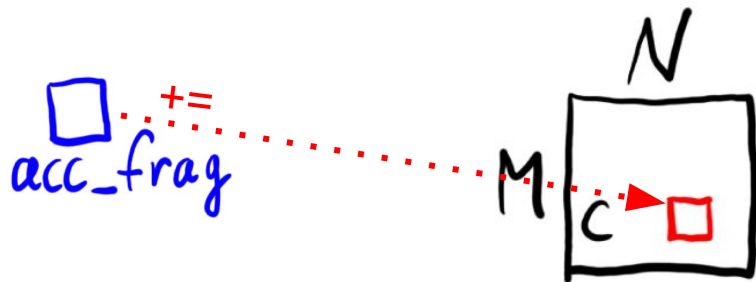
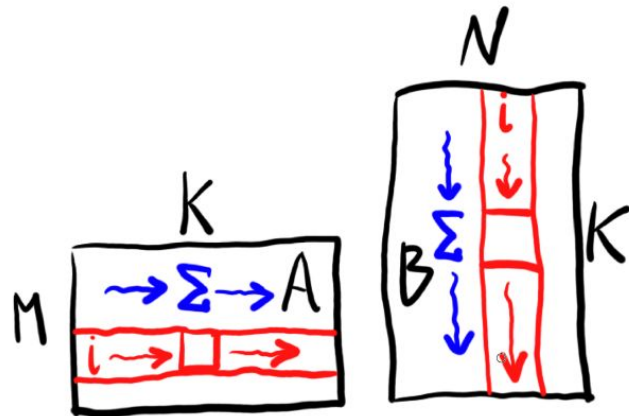
87 `wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;`

88 `wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;`

89 `wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;`

.....

$$C = \alpha A * B + \beta C$$

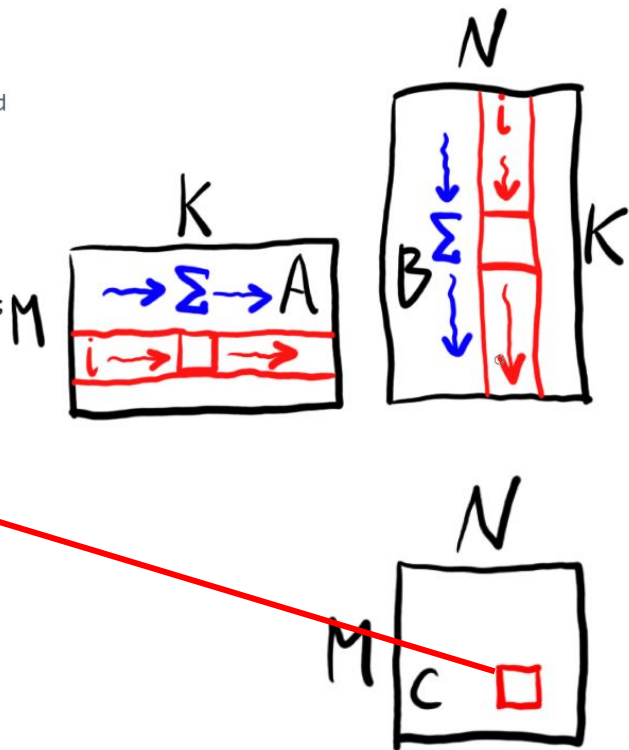


```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
90     .....
113    // Load in the current value of c, scale it by beta, and add this our result scaled
114    int cRow = warpM * WMMA_M;
115    int cCol = warpN * WMMA_N;
116
117    if (cRow < M && cCol < N) {
118        wmma::load_matrix_sync(c_frag, c + cRow + cCol * ldc, ldc, wmma::mem_col_major);

```

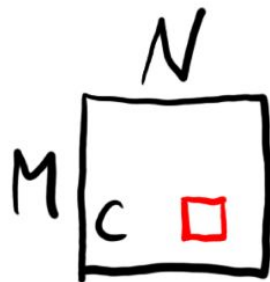
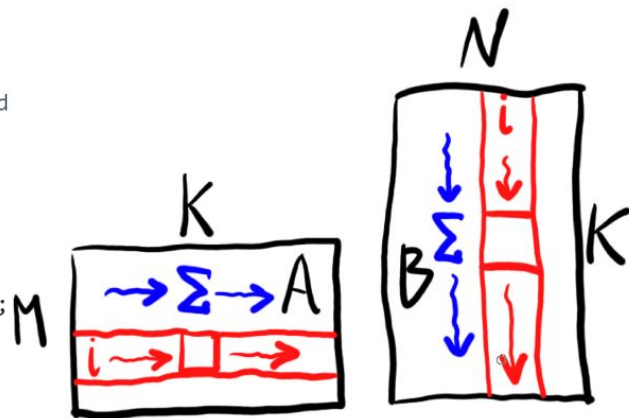
$$C = \alpha A * B + \beta C$$



```
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
```

```
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
76     // Declare the fragments
77     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
78     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
79     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
80     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
81     .....
82     // Load in the current value of c, scale it by beta, and add this our result scaled
83     int cRow = warpM * WMMA_M;
84     int cCol = warpN * WMMA_N;
85
86     if (cRow < M && cCol < N) {
87         wmma::load_matrix_sync(c_frag, c + cRow + cCol * ldc, ldc, wmma::mem_col_major);
88
89     #pragma unroll
90     for(int i=0; i < c_frag.num_elements; i++) {
91         c_frag.x[i] = alpha * acc_frag.x[i] + beta * c_frag.x[i];
92     }
93 }
```

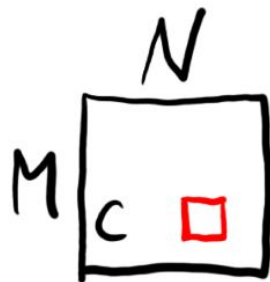
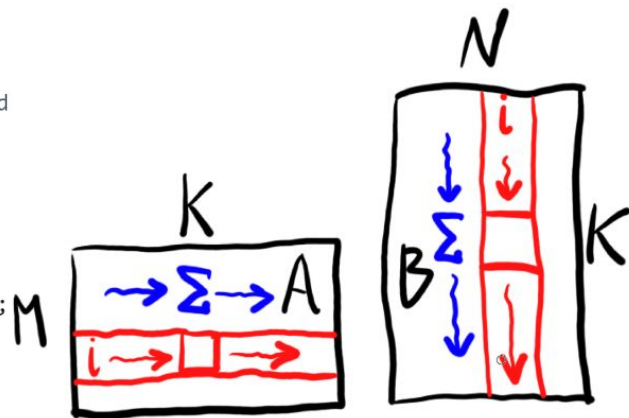
$$C = \alpha A * B + \beta C$$



```
69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
```

```
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {  
76     // Declare the fragments  
77     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;  
78     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;  
79     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;  
80     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;  
81     .....  
82     // Load in the current value of c, scale it by beta, and add this our result scaled  
83     int cRow = warpM * WMMA_M;  
84     int cCol = warpN * WMMA_N;  
85  
86     if (cRow < M && cCol < N) {  
87         wmma::load_matrix_sync(c_frag, c + cRow + cCol * ldc, ldc, wmma::mem_col_major);  
88  
89     #pragma unroll  
90     for(int i=0; i < c_frag.num_elements; i++) {  
91         c_frag.x[i] = alpha * acc_frag.x[i] + beta * c_frag.x[i];  
92     }  
93 }
```

$$C = \alpha A * B + \beta C$$



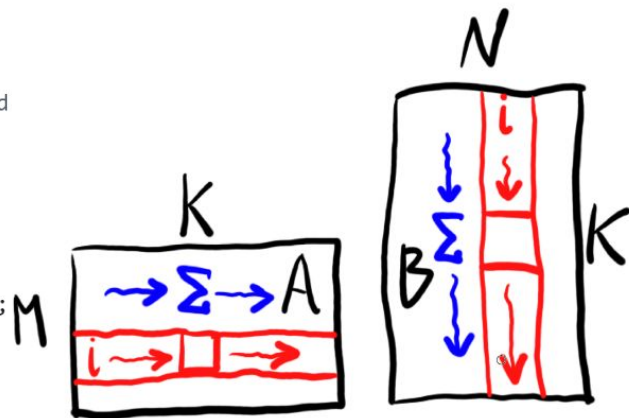
А не будет тормозить?
Почему не спец. функция в железе?

```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
75 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
85     // Declare the fragments
86     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
87     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
88     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
89     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
90
91     .....
113    // Load in the current value of c, scale it by beta, and add this our result scaled
114    int cRow = warpM * WMMA_M;
115    int cCol = warpN * WMMA_N;
116
117    if (cRow < M && cCol < N) {
118        wmma::load_matrix_sync(c_frag, c + cRow + cCol * ldc, ldc, wmma::mem_col_major);
119
120    #pragma unroll
121    for(int i=0; i < c_frag.num_elements; i++) {
122        c_frag.x[i] = alpha * acc_frag.x[i] + beta * c_frag.x[i];
123    }

```

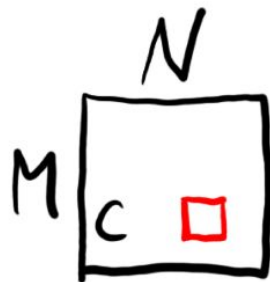
$$C = \alpha A * B + \beta C$$



А не будет тормозить?

Почему не спец. функция в железе?

А почему бы не добавить к C в отдельном кернеле?




```
// Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
```

```
__global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
```

```
// Declare the fragments
```

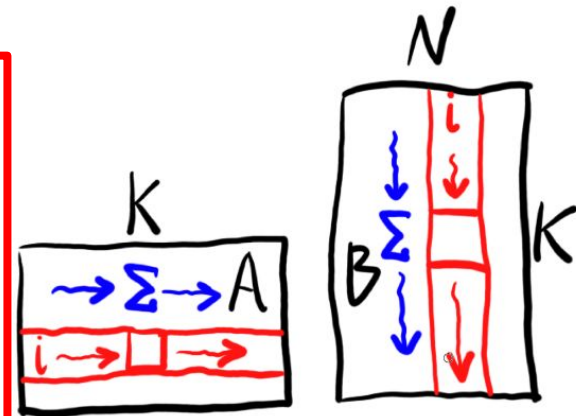
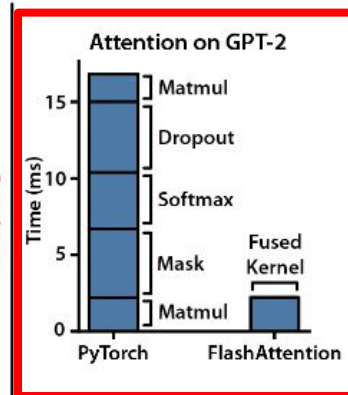
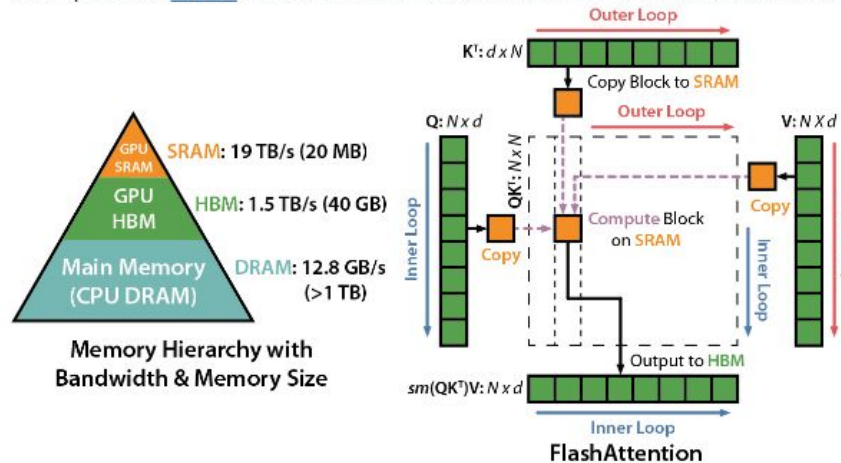
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré

Paper: <https://arxiv.org/abs/2205.14135> <https://github.com/Dao-AI-Lab/flash-attention>

IEEE Spectrum [article](#) about our submission to the MLPerf 2.0 benchmark using FlashAttention.

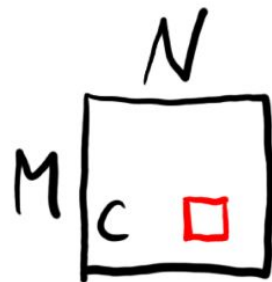
$$C = \alpha A * B + \beta C$$



А не будет тормозить?

Почему не спец. функция в железе?

А почему бы не добавить к C в отдельном кернеле?

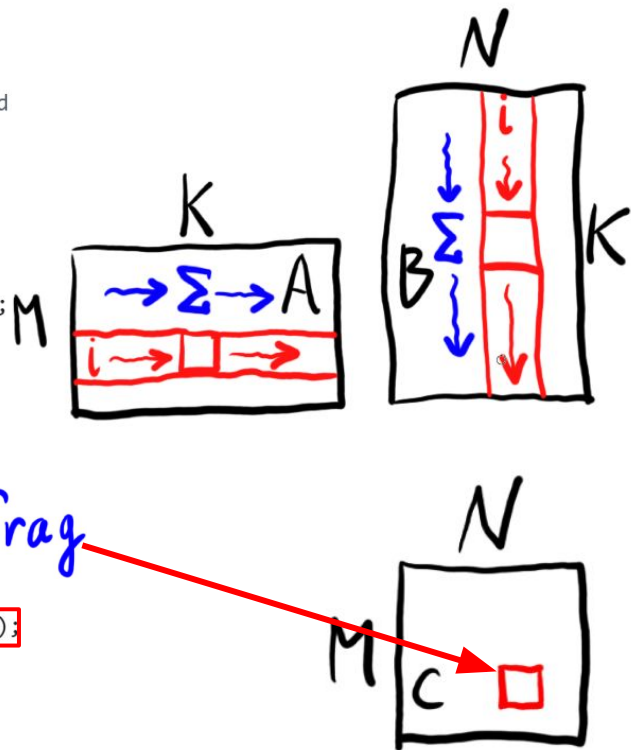



```

69 // Performs an MxNxK GEMM (C=alpha*A*B + beta*C)
71 __global__ void wmma_example(half *a, half *b, float *c, int M, int N, int K, float alpha, float beta) {
72     // Declare the fragments
73     wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> a_frag;
74     wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K, half, wmma::col_major> b_frag;
75     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;
76     wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> c_frag;
77     .....
78     // Load in the current value of c, scale it by beta, and add this our result scaled
79     int cRow = warpM * WMMA_M;
80     int cCol = warpN * WMMA_N;
81
82     if (cRow < M && cCol < N) {
83         wmma::load_matrix_sync(c_frag, c + cRow + cCol * ldc, ldc, wmma::mem_col_major);
84
85 #pragma unroll
86         for(int i=0; i < c_frag.num_elements; i++) {
87             c_frag.x[i] = alpha * acc_frag.x[i] + beta * c_frag.x[i];
88         }
89
90         // Store the output
91         wmma::store_matrix_sync(c + cRow + cCol * ldc, c_frag, ldc, wmma::mem_col_major);
92     }
93 }

```

$$C = \alpha A * B + \beta C$$



Умножение матриц - Tensor Cores, WMMA

Сколько у нас вычислений?

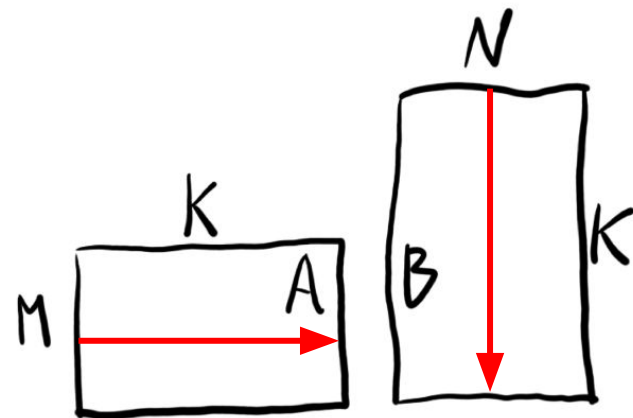
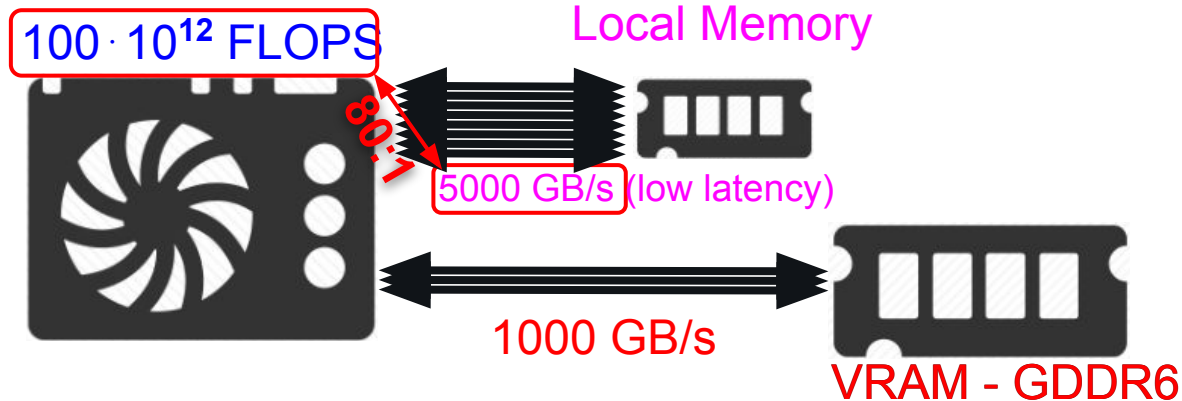
$$O(N \times M \times K)$$

Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 **Memory-bound!**



$$C = A \times B$$

The diagram shows the resulting matrix C, which is a square with width N and height M. A red circle with a dot in the center is located in the bottom right corner of the matrix.

Какая у нас теперь пропорция вычислений к чтениям?

Умножение матриц - Tensor Cores, WMMA

Сколько у нас вычислений?

$$O(N \times M \times K)$$

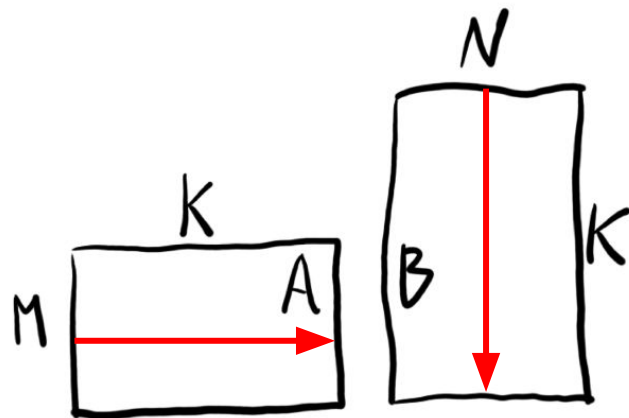
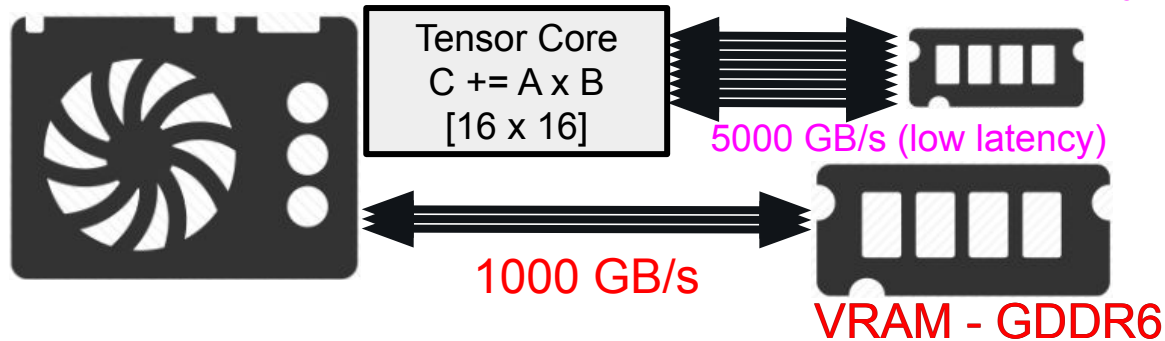
Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

Какая пропорция?

1:1 **Memory-bound!**

100 · 10¹² FLOPS



$$C = A \times B$$

The diagram shows the resulting matrix C, which is a square with dimensions M (height) and N (width). A red circle with a dot in the center is located in the bottom right corner of the matrix C.

Какая у нас теперь пропорция вычислений к чтениям?

Умножение матриц

Сколько у нас вычислений?

$$O(N \times M \times K)$$

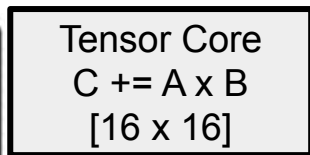
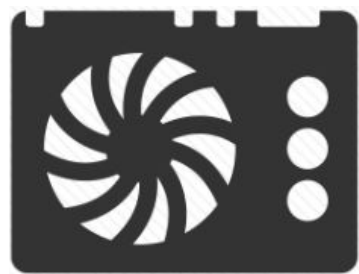
Сколько у нас чтений/записей данных?

$$O(N \times M \times K)$$

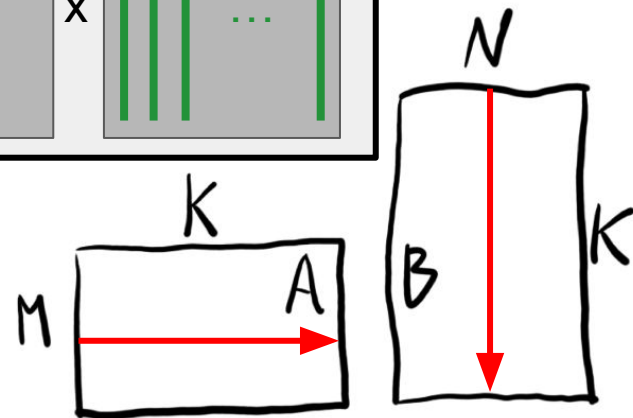
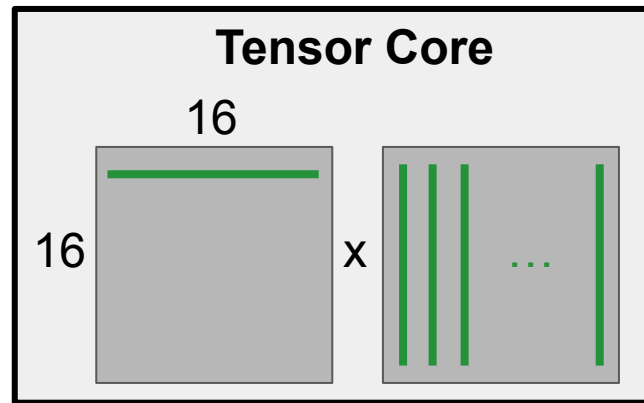
Какая пропорция?

1:1 **Memory-bound!**

$100 \cdot 10^{12}$ FLOPS



1000 GB/s



$$C = A \times B$$

M

N

C

Какая у нас теперь пропорция вычислений к чтениям?

secret techniques

python/tutorials/gluon/01-attention-forward.py

```
...  @@ -834,13 +834,20 @@ def _attn_fwd_correction(config, chnls, desc_s, M, STAGE: gl.constexpr):
834 834         config, prog, s1_tmem, M, corr1_consumer, epi_producer, o_consumer)
835 835
836 836
837 - @gluon.jit(do_not_specialize=["Z"])
838 - def gluon_attn(sm_scale, M, Z, H, N_CTX, #
839 -               desc_q, desc_k, desc_v, desc_o, #
840 -               BLOCK_M: gl.constexpr, BLOCK_N: gl.constexpr, HEAD_DIM: gl.constexpr, #
841 -               GROUP_SIZE_N: gl.constexpr, NUM_SMS: gl.constexpr, #
842 -               STAGE: gl.constexpr, dtype: gl.constexpr, #
843 -               num_warps: gl.constexpr):
837 + def attention_repr(specialization):
838 +     name = "gluon_attention"
839 +     # Up to 150 TFLOPS faster for fp8!
840 +     if specialization.constants["dtype"] == gl.float8e5:
841 +         name = "cutlass_" + name
```



peterbell10 on Jul 9

Contributor ...

very cool... did you check if other names change the scheduling (e.g. because of non-determinism or code alignment) or if it's literally just special cased for cutlass.



Mogball on Jul 9

Collaborator Author ...

it's literally just special cased for cutlass.

Yup



46



8



5

Умножение матриц

Tensor Cores

<https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9/>

<https://github.com/NVIDIA-developer-blog/code-samples/blob/master/posts/tensor-cores/simpleTensorCoreGEMM.cu>

https://github.com/NVIDIA/cutlass/blob/main/examples/00_basic_gemm/basic_gemm.cu

<https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda>

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

Неравная битва за гигафлопсы при умножении матриц
(хорошо описанная аналитика, профилирование, оптимизация):

- AMD RDNA3 - <https://seb-v.github.io/optimization/update/2025/01/20/Fast-GPU-Matrix-multiplication.html>
- NVIDIA Kepler - <https://cnugeteren.github.io/tutorial/pages/page15.html>
- <https://siboehm.com/articles/22/CUDA-MMM>



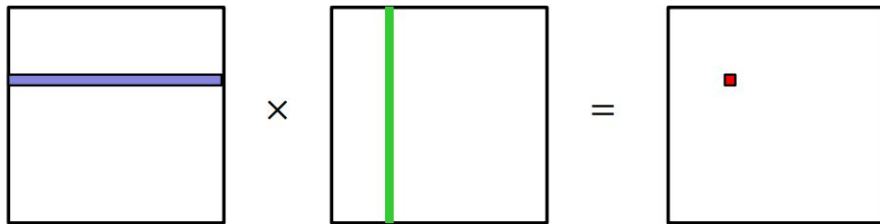
Scalar Loop

- Each invocation computes one element of the result matrix
- Lots of redundant loads
- Example of what NOT to do

```
uint i = gl_GlobalInvocationID.y;
uint j = gl_GlobalInvocationID.x;
float16_t C = inputC.x[sC * i + j];

for (uint k = 0; k < K; ++k) {
    float16_t A = inputA.x[sA * i + k];
    float16_t B = inputB.x[sB * k + j];
    C += A*B;
}

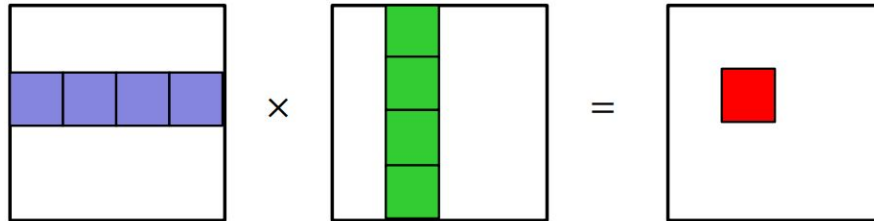
outputD.x[sD * i + j] = C;
```



GPU	TFLOPS
RTX 2070	0.232
RTX TITAN	0.528

Simple Cooperative Multiply

- A bit more coordinate calculation, but still a simple $\text{sum}(A_{ik}B_{kj})$
- Still very memory-limited, but improved



```

LM = 16; LN = 8; LK = 8;
fcoopmatNV<16, gl_ScopeSubgroup, LM, LK> matA;
fcoopmatNV<16, gl_ScopeSubgroup, LK, LN> matB;
fcoopmatNV<16, gl_ScopeSubgroup, LM, LN> matC;

uvec2 matrixID = uvec2(gl_WorkGroupID);
uint cRow = LM * matrixID.y;
uint cCol = LN * matrixID.x;

coopMatLoadNV matC, inputC.x, sC * cRow + cCol, sC, false);

for (uint k = 0; k < K; k += LK) {
    uint aRow = LM * matrixID.y;
    uint aCol = k;
    coopMatLoadNV matA, inputA.x, sA * aRow + aCol, sA, false);

    uint bRow = k;
    uint bCol = LN * matrixID.x;
    coopMatLoadNV matB, inputB.x, sB * bRow + bCol, sB, false);

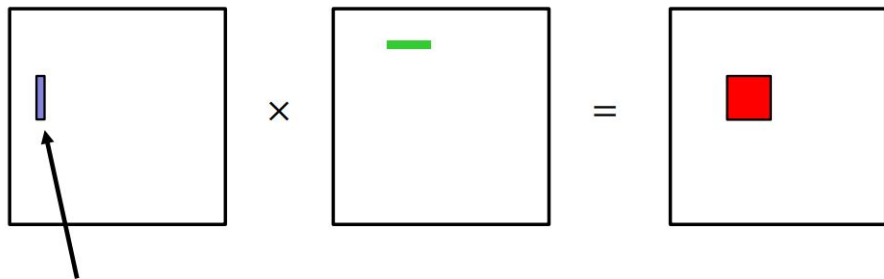
    matC = coopMatMulAddNV(matA, matB, matO);
}

coopMatStoreNV(matC, outputD.x, sD * cRow + cCol, sD, false);
    
```

GPU	TFLOPS
RTX 2070	2.00
RTX TITAN	3.70

Tiled Scalar Multiply

- Illustrates a way to tile the multiply
 - Load row, load column, outer product
 - Name of the game is to load once, then perform as many multiplies as possible
 - Limited by register file size
 - Maybe 8x8 tile size per invocation



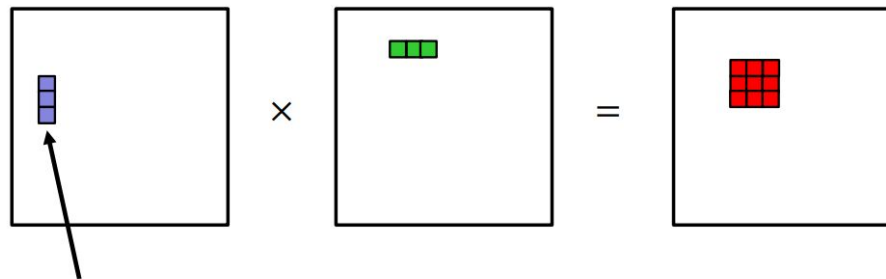
One iteration in 'k'

```
float16_t C[C_ROWS][C_COLS];
uvec2 tileID = uvec2(gl_WorkGroupID.xy);
uvec2 inv = uvec2(gl_LocalInvocationID.xy);
// load C
...
// iterate through K dimension and accumulate
for (uint k = 0; k < K; ++k) {
    float16_t A[C_ROWS];
    for (uint i = 0; i < C_ROWS; ++i) {
        uint gi = TILE_M * tileID.y + (C_ROWS * inv.y + i);
        uint gk = k;
        A[i] = inputA.x[sA * gi + gk];
    }
    float16_t B;
    for (uint j = 0; j < C_COLS; ++j) {
        uint gk = k;
        uint gj = TILE_N * tileID.x + (C_COLS * inv.x + j);
        B = inputB.x[sB * gk + gj];
        for (uint i = 0; i < C_ROWS; ++i) {
            C[i][j] = A[i] * B + C[i][j];
        }
    }
}
// store C
...
```

GPU	TFLOPS
RTX 2070	1.45
RTX TITAN	2.79

Tiled Cooperative Matrix Multiply

- Apply tiling approach to coop matrices
- Effectively 32x the register file, since matrices are spread over a subgroup
 - 112x112x16 multiply per outer loop iter
 - Takes advantage of cheap communication within subgroups to maximize reuse
- Shader code is no more complex than what you'd write for scalar

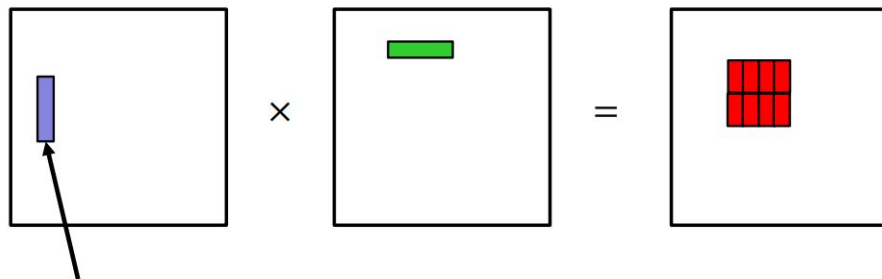


```
fcoopmatNV<16, gl_ScopeSubgroup, 1M, 1N> matC[C_ROWS][C_COLS];
uvec2 tileID = uvec2(gl_WorkGroupID.xy);
// load matC
...
for (uint k = 0; k < K; k += TILE_K) {
    fcoopmatNV<16, gl_ScopeSubgroup, 1M, 1K> matA[C_ROWS];
    for (uint i = 0; i < C_ROWS; ++i) {
        uint gi = TILE_M * tileID.y + 1M * i;
        uint gk = k;
        coopMatLoadNV(matA[i], inputA.x, sA * gi + gk, sA, false);
    }
    fcoopmatNV<16, gl_ScopeSubgroup, 1K, 1N> matB;
    for (uint j = 0; j < C_COLS; ++j) {
        uint gk = k;
        uint gj = TILE_N * tileID.x + 1N * j;
        coopMatLoadNV(matB, inputB.x, sB * gk + gj, sB, false);
        for (uint i = 0; i < C_ROWS; ++i) {
            matC[i][j] = coopMatMulAddNV(matA[i], matB, matC[i][j]);
        }
    }
}
// store matC
...
```

GPU	TFLOPS
RTX 2070	21.8
RTX TITAN	42.1

Staging Through Shared Memory

- Subgroups cooperate to copy data from buffer to shared, then load out of shared
 - Overlap buffer loads with matrix math (i.e. pipeline load for next iteration)
- **Example (FP16): 8 subgroups split a 256x256 tile into 8 128x64 tiles (K=32)**
 - Cooperate to copy A block (256x32) and B block (32x256) into shared memory
 - Then each subgroup loads the portions it needs from shared memory
 - Only 128B of buffer loads per thread per K iteration overlapping with 8K FMADs
- **Example (INT8): 8 subgroups split a 128x256 tile into 8 64x64 tiles (K=64)**
 - Accumulator is INT32, requires 2x register file, hurts latency hiding
 - Double math rate, half tile size = 4x harder to hide latency, much farther from SOL



One iteration in 'k'

GPU	INT8 TOPS
RTX 2070	66.0
RTX TITAN	~130

GPU	FP16 TFLOPS
RTX 2070	49.0
RTX TITAN	~100

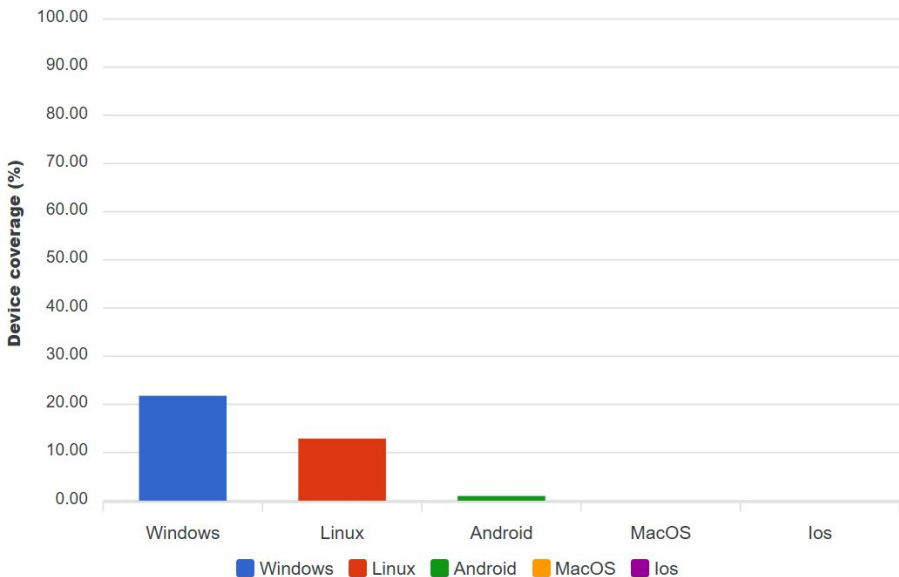
Device coverage for VK_KHR_cooperative_matrix

[Vulkan registry manpage](#)

This extension has additional [features](#) and [properties](#)

Extension was first submitted at 2023-06-27

Vulkan extension: VK_KHR_cooperative_matrix



Platform	Coverage
Windows	21.79%
Linux	12.9%
Android	0.98%

Listing first known driver version support for VK_KHR_cooperative_matrix

This extension has additional [features](#) and [properties](#)

Extension was first submitted at 2023-06-27

[All platforms](#)

[Windows](#)

[Linux](#)

[Android](#)

[macOS](#)

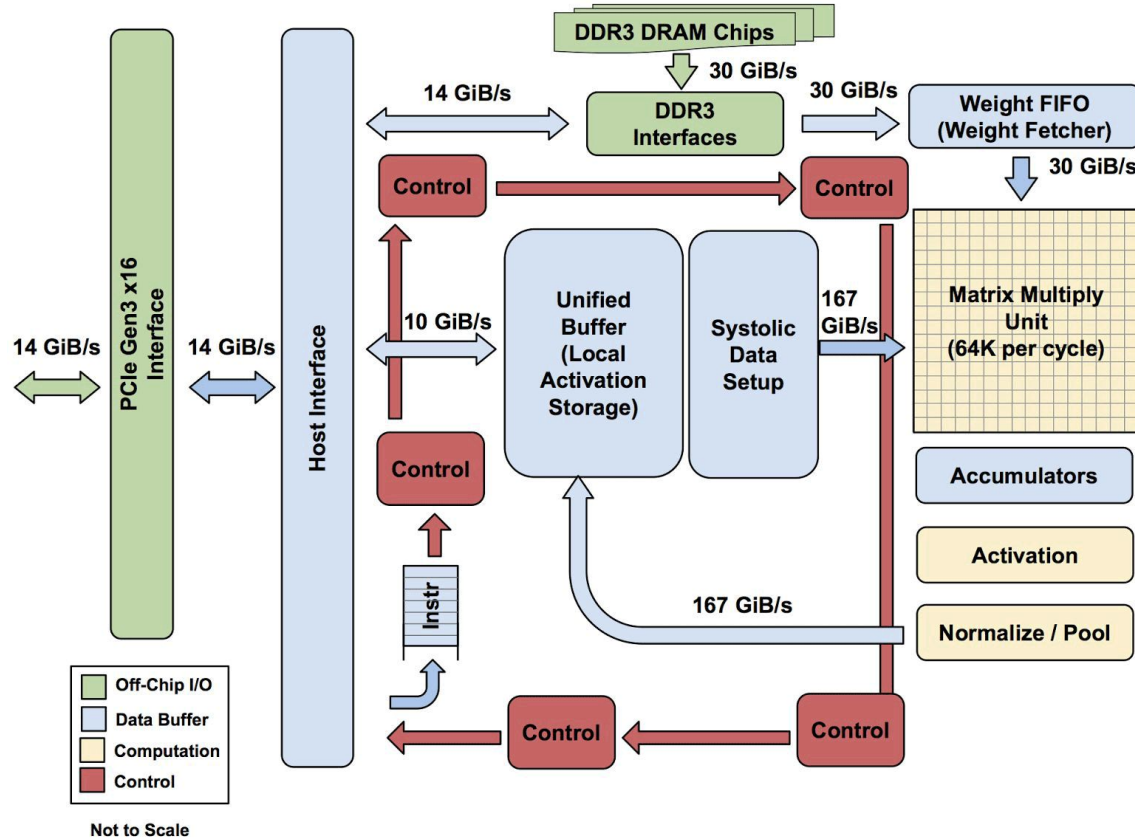
[iOS](#)

Show 50 entries

amd	Type to filter	Type to filter		
Device	Vendor	Driver [?]	Date	Compare
AMD Radeon(TM) 740M	AMD	2.0.302	2025-09-16	Add
AMD Radeon RX 5500	AMD	2.0.310	2025-08-26	Add
AMD Radeon Pro 5500M	AMD	2.0.317	2025-08-13	Add
AMD Radeon(TM) 860M Graphics	AMD	2.0.331	2025-07-23	Add

intel	Type to filter	Type to filter		
Device	Vendor	Driver [?]	Date	Compare
Intel(R) Iris(R) Xe Graphics (ADL GT2)	INTEL	6400.4099	2025-07-09	Add
Intel(R) Arc(TM) 130V GPU (8GB)	INTEL	101.6314	2025-07-06	Add
Intel(R) Arc(TM) 130V GPU (16GB)	INTEL	101.6556	2025-06-23	Add
Intel(R) Arc(TM) 140T GPU (32GB)	INTEL	101.6554	2025-05-31	Add
Intel(R) Arc(TM) B570 Graphics	INTEL	101.6790	2025-05-19	Add

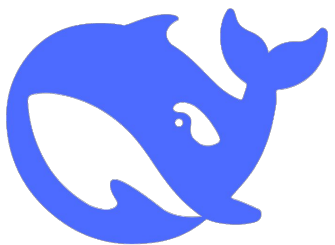
Tensor Processor Unit - TPU (Google)





Глава 4: Умножение матриц

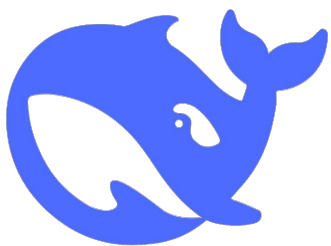
DeepSeek



DeerSeek: x2 ускорение обучения (fp16 → fp8)

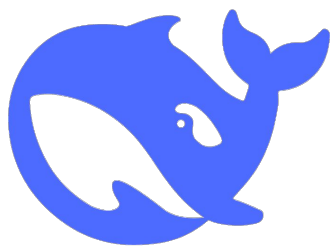


$$\pm (1 + \text{mantissa}) \times 2^{(\text{exponent} - 127)}$$



DeerSeek: x2 ускорение обучения (fp16 → fp8)

	sign	exponent					mantissa										
FP16	0	0	1	1	0	1	1	0	0	1	0	1	0	0	1	1	= 0.395264
BF16	0	0	1	1	1	1	1	0	1	1	0	0	1	0	1	0	= 0.394531
FP8 E4M3	0	0	1	0	1	1	0	1									= 0.40625
FP8 E5M2	0	0	1	1	0	1	1	0									= 0.375

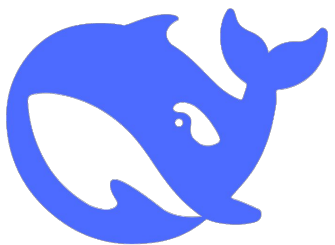


DeerSeek: x2 ускорение обучения (fp16 → fp8)



NVIDIA H100 (почти то же что и H800*):

- Memory bandwidth: **3.35 TB/sec**
- FP 32: **67 TFlops**
- FP 16: **268 TFlops**
- FP 32 (tensor): **495 TFlops**
- FP 16 (tensor): **990 TFlops**
- FP 8 (tensor): **1979 TFlops**

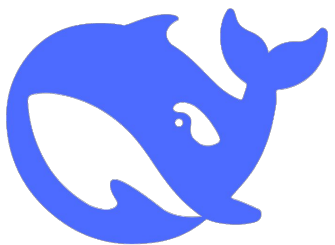


DeerSeek: x2 ускорение обучения (fp16 → fp8)

NVIDIA H100 (почти то же что и H800*):

- Memory bandwidth: **3.35 TB/sec**
- FP 32: **67 TFlops**
- FP 16: **268 TFlops**
- FP 32 (tensor): **495 TFlops**
- FP 16 (tensor): **990 TFlops**
- FP 8 (tensor): **1979 TFlops**

x2



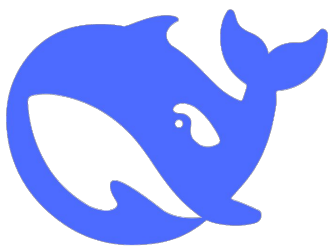
DeerSeek: x2 ускорение обучения (fp16 → fp8)

NVIDIA H100 (почти то же что и H800*):

- Memory bandwidth: **3.35 TB/sec**
- FP 32: **67 TFlops**
- FP 16: **268 TFlops**
- FP 32 (tensor): **495 TFlops**
- FP 16 (tensor): **990 TFlops**
- FP 8 (tensor): **1979 TFlops**

Хватит ли пропускной способности памяти чтобы насытить ALU (tensor cores)?

x2



DeerSeek: x2 ускорение обучения (fp16 → fp8)

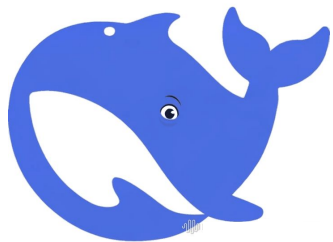
NVIDIA H100 (почти то же что и H800*):

- Memory bandwidth: **3.35 TB/sec**
- FP 32: **67 TFlops**
- FP 16: **268 TFlops**
- FP 32 (tensor): **495 TFlops**
- FP 16 (tensor): **990 TFlops**
- FP 8 (tensor): **1979 TFlops**

x2

Какие риски?

Почему так не делают все?



DeepSeek: fine-grained fp8 quantization

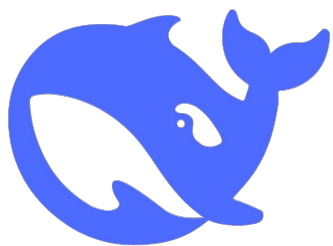


NVIDIA H100 (почти то же что и H800*):

- Memory bandwidth: **3.35 TB/sec**
- FP 32: **67 TFlops**
- FP 16: **268 TFlops**
- FP 32 (tensor): **495 TFlops**
- FP 16 (tensor): **990 TFlops**
- FP 8 (tensor): **1979 TFlops**
- DeepGEMM достиг **1550 TFlops** на H800!

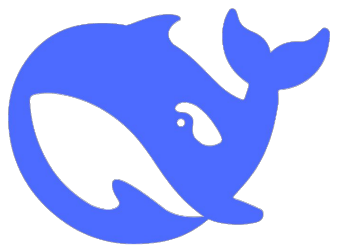
DeepSeek-V3 Technical Report - <https://arxiv.org/abs/2412.19437>

Репозиторий - <https://github.com/deepseek-ai/DeepGEMM> (GEMM - General Matrix Multiplications)

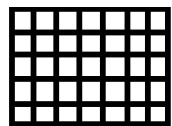


DeepSeek: fine-grained fp8 quantization





DeepSeek: fine-grained fp8 quantization

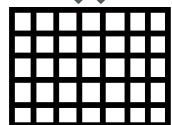


fp32

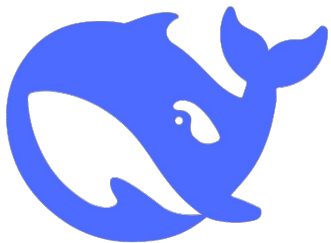


Scaling Factor

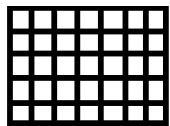
fp32



fp8



DeepSeek: fine-grained fp8 quantization



fp32

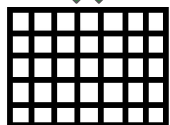
Input Values



Scaling Factor

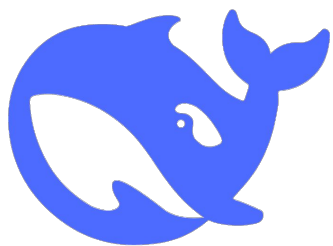
fp32

Каким его выбрать?

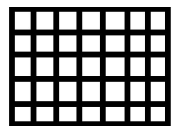


fp8

Input Values / Scaling Factor



DeepSeek: fine-grained fp8 quantization



fp32

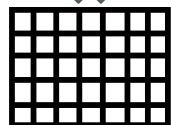
Input Values



Scaling Factor

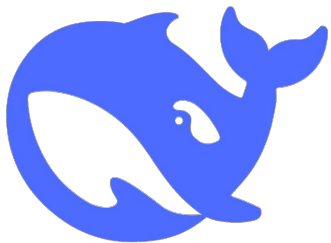
fp32

$\text{absmax}(\text{Input Values}) / 448$

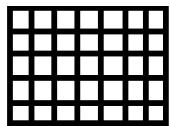


fp8

Input Values / Scaling Factor



DeepSeek: fine-grained fp8 quantization



fp32

Input Values

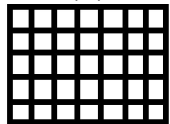


Scaling Factor

fp32

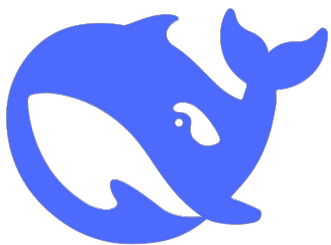
$\text{absmax}(\text{Input Values}) / 448$

Что это за волшебное число? 🦄

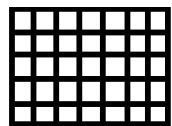


fp8

Input Values / Scaling Factor



DeepSeek: fine-grained fp8 quantization

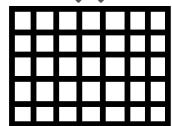


fp32 Input Values



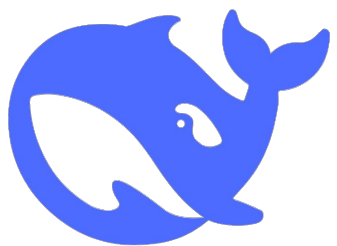
Scaling Factor

fp32 $\text{absmax}(\text{Input Values}) / 448 = \text{FP8_MAX}$



fp8

$\text{Input Values} / \text{Scaling Factor} \in [-448; +448]$

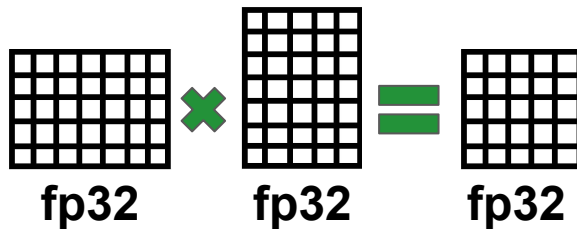


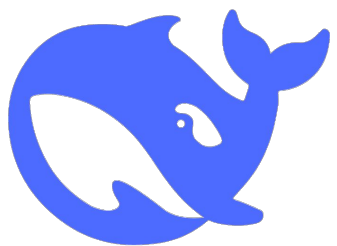
DeepSeek: fine-grained fp8 quantization



Scaling Factor

fp32





DeepSeek: fine-grained fp8 quantization

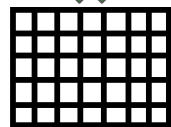


fp32

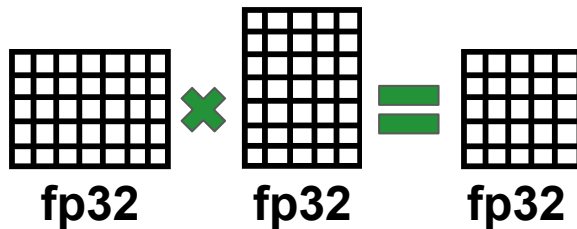


Scaling Factor

fp32



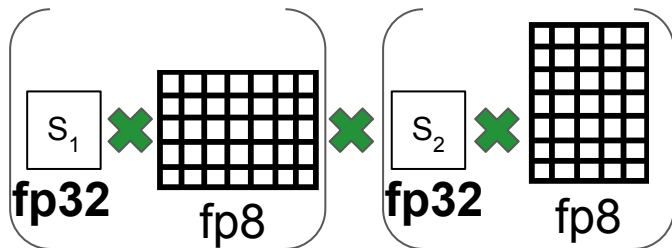
fp8



fp32

fp32

fp32

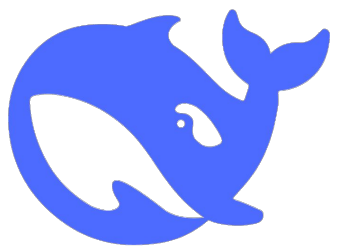


fp32

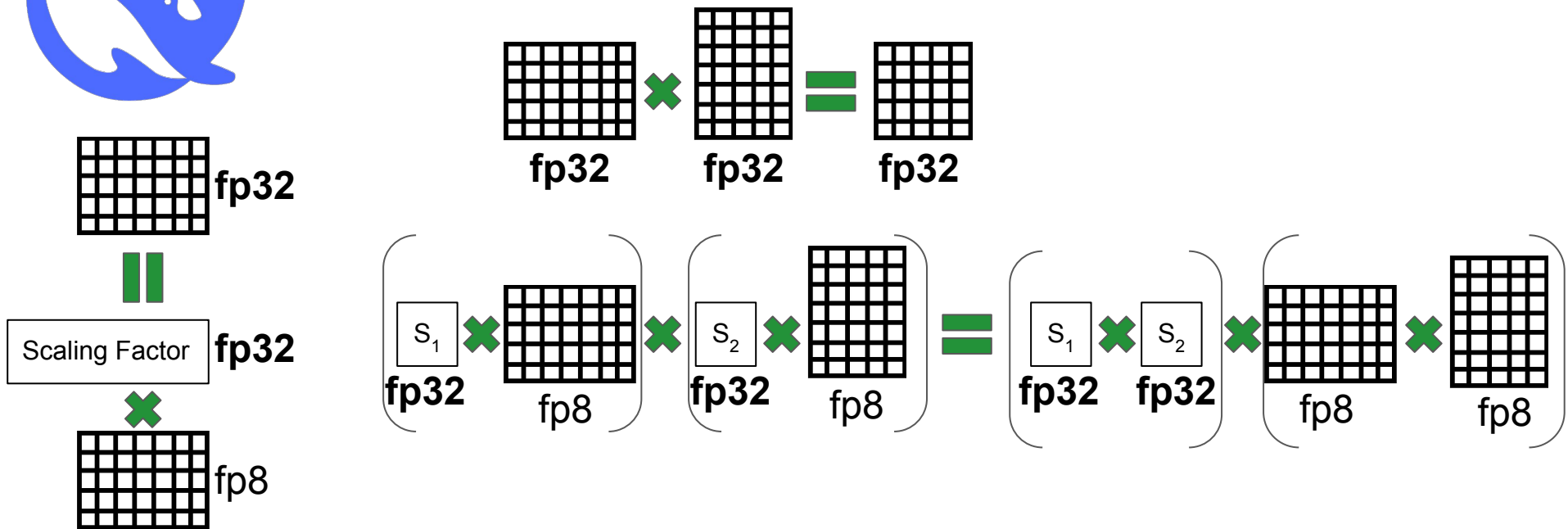
fp8

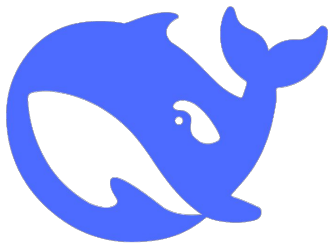
fp32

fp8



DeepSeek: fine-grained fp8 quantization





DeepSeek: fine-grained fp8 quantization

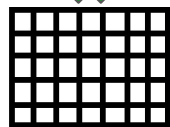


fp32

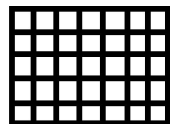


Scaling Factor

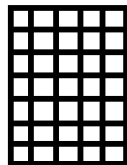
fp32



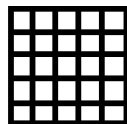
fp8



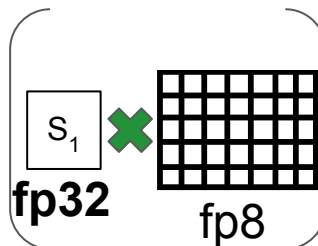
fp32



fp32

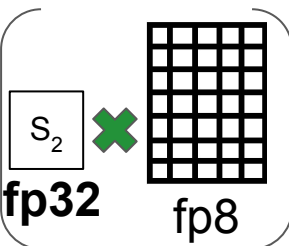


fp32



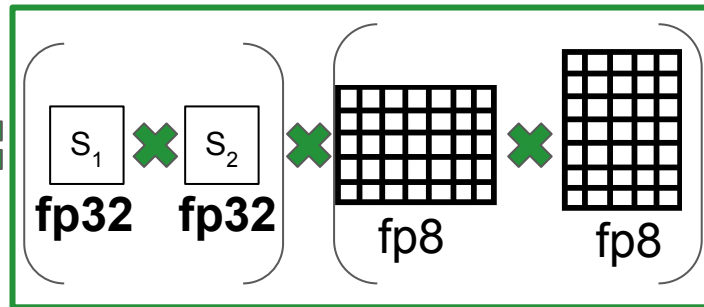
fp32

fp8



fp32

fp8



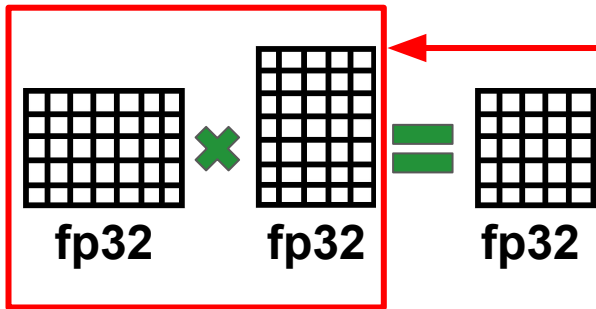
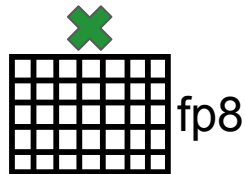
fp32

fp32

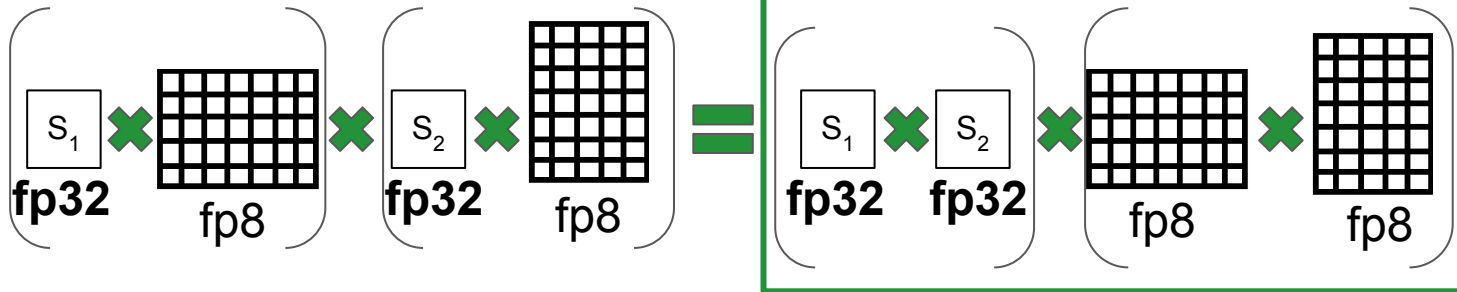
fp8

fp8

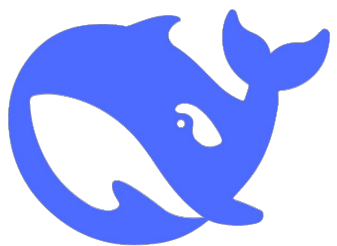
Насколько быстрее?



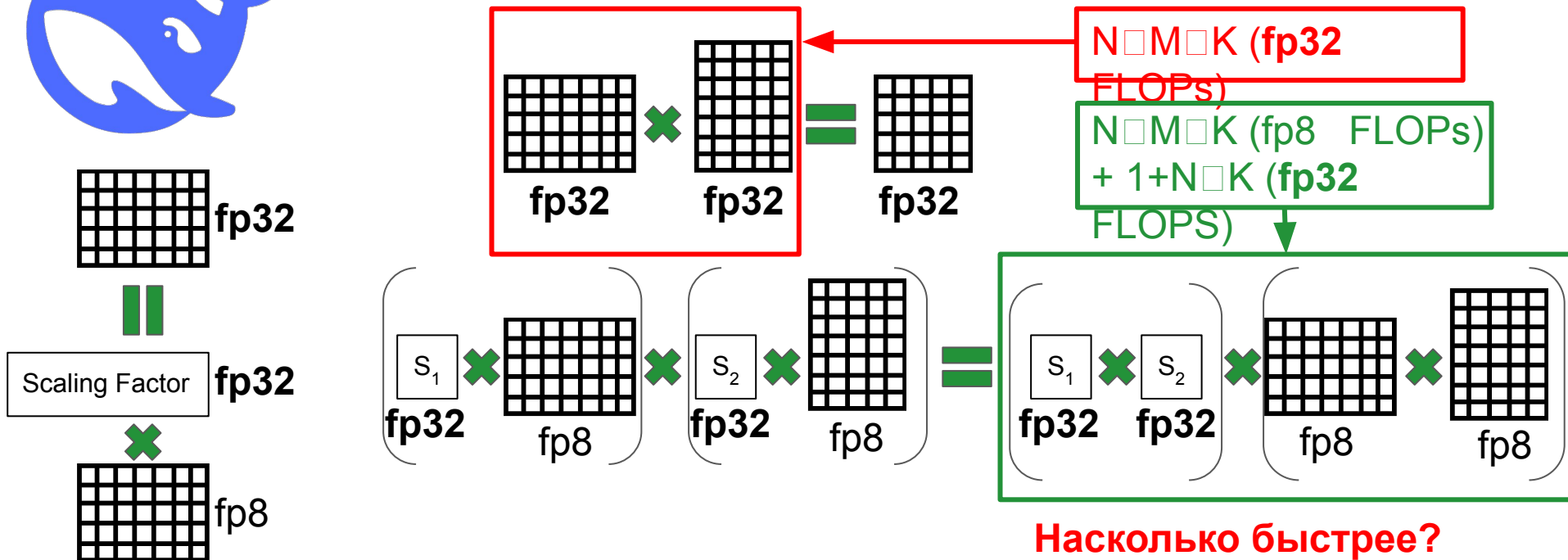
NOMK (fp32 FLOPs)

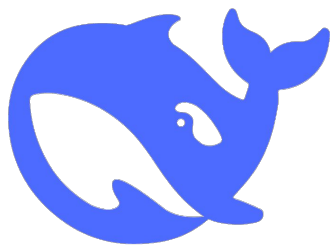


Насколько быстрее?



DeepSeek: fine-grained fp8 quantization





DeepSeek: fine-grained fp8 quantization

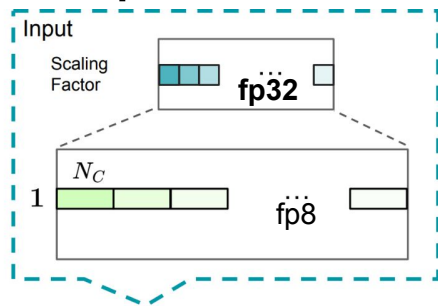


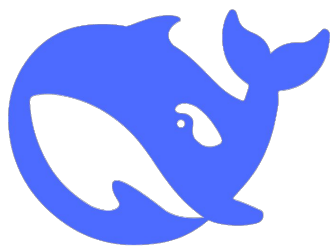
Scaling Factor

fp32



fp8





DeepSeek: fine-grained fp8 quantization

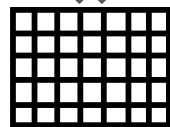


fp32

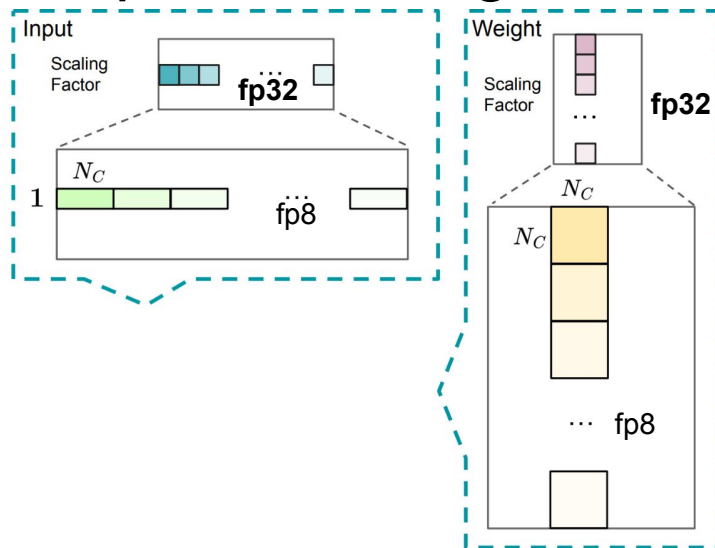


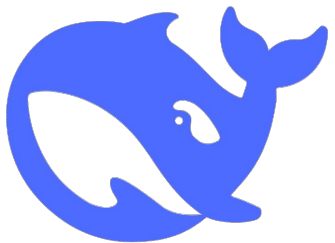
Scaling Factor

fp32



fp8





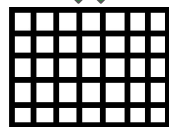
DeepSeek: fine-grained fp8 quantization



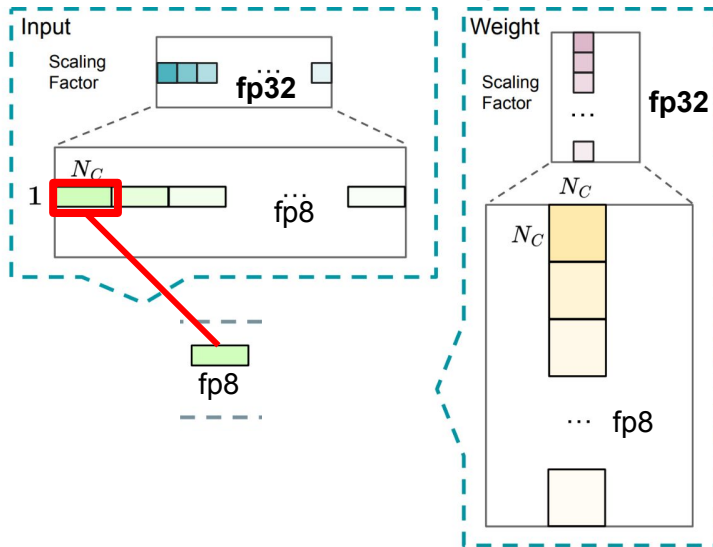
fp32

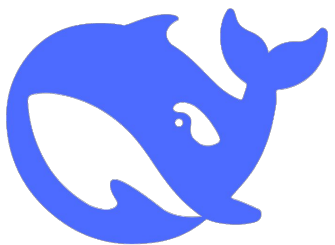
Scaling Factor

fp32



fp8

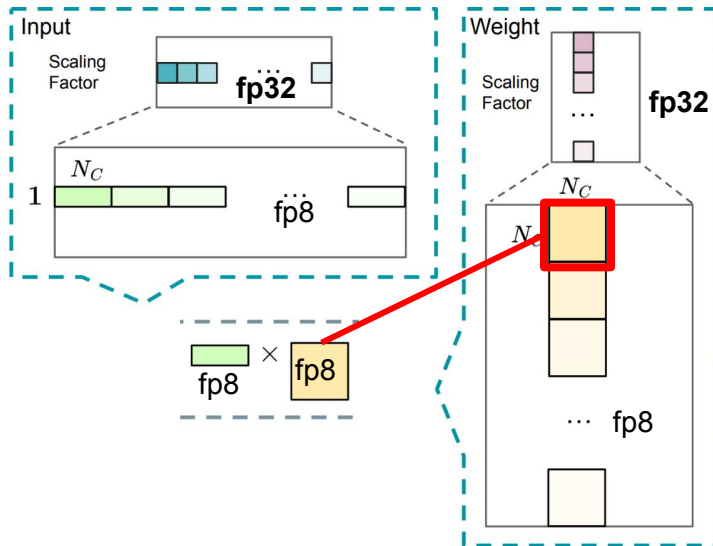


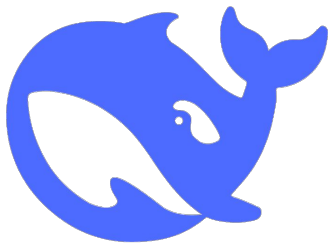


DeepSeek: fine-grained fp8 quantization

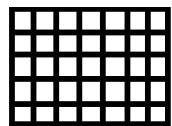


Scaling Factor **fp32**





DeepSeek: fine-grained fp8 quantization

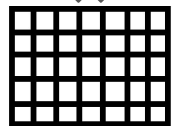


fp32

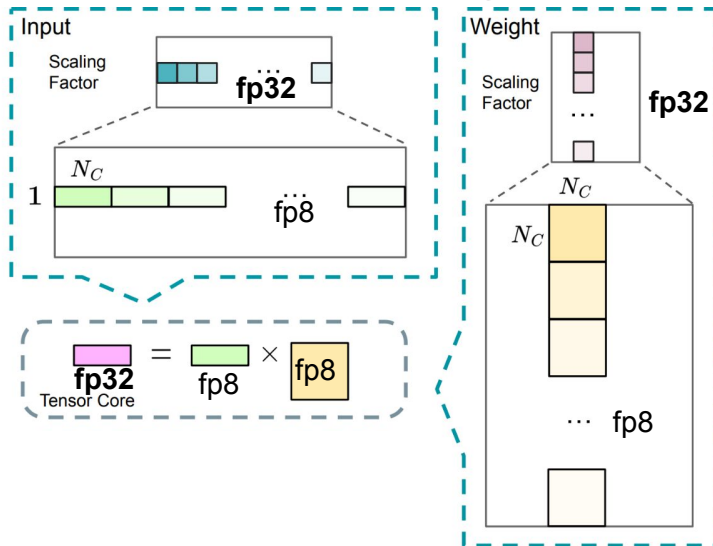


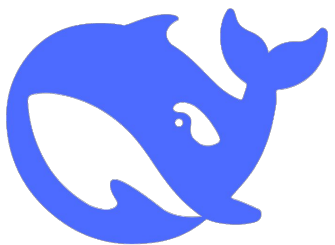
Scaling Factor

fp32

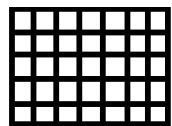


fp8





DeepSeek: fine-grained fp8 quantization

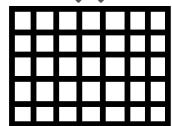


fp32

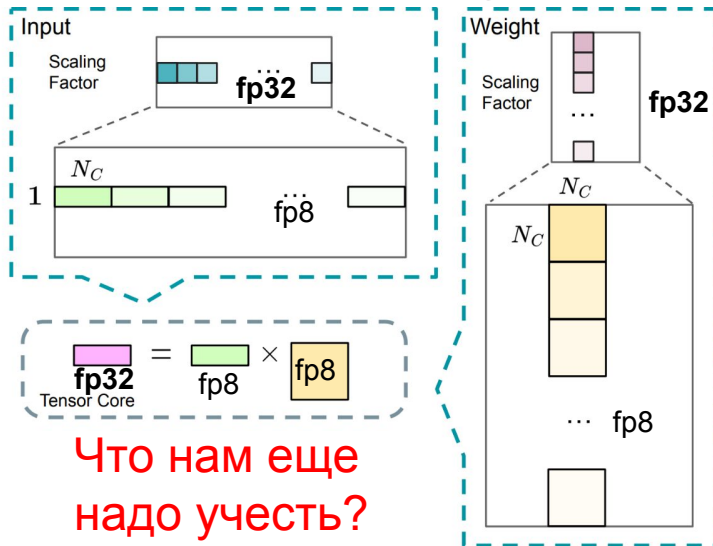


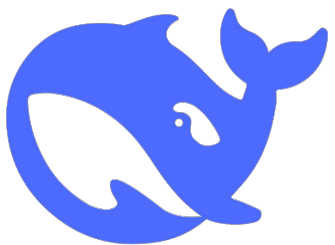
Scaling Factor

fp32



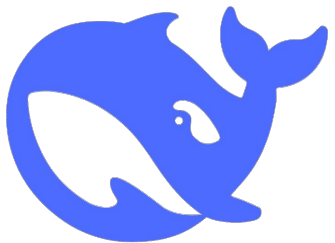
fp8



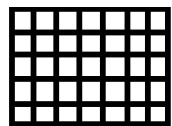


DeepSeek: fine-grained fp8 quantization





DeepSeek: fine-grained fp8 quantization

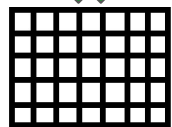


fp32

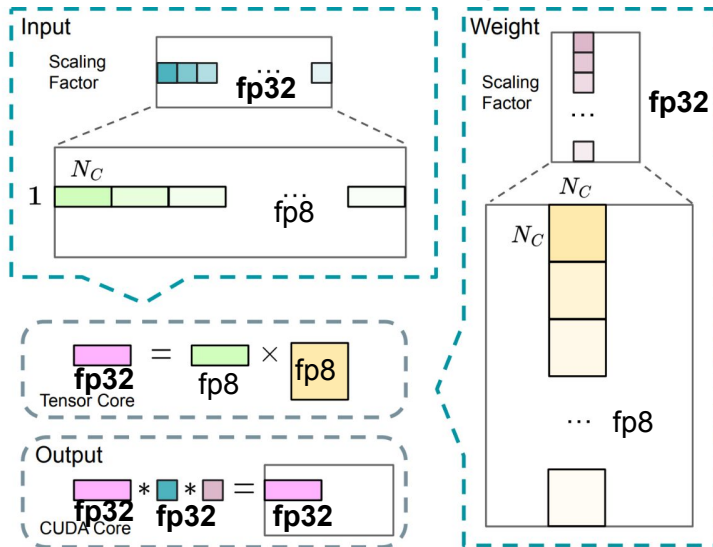


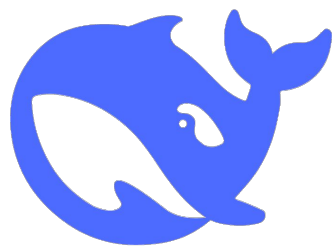
Scaling Factor

fp32



fp8

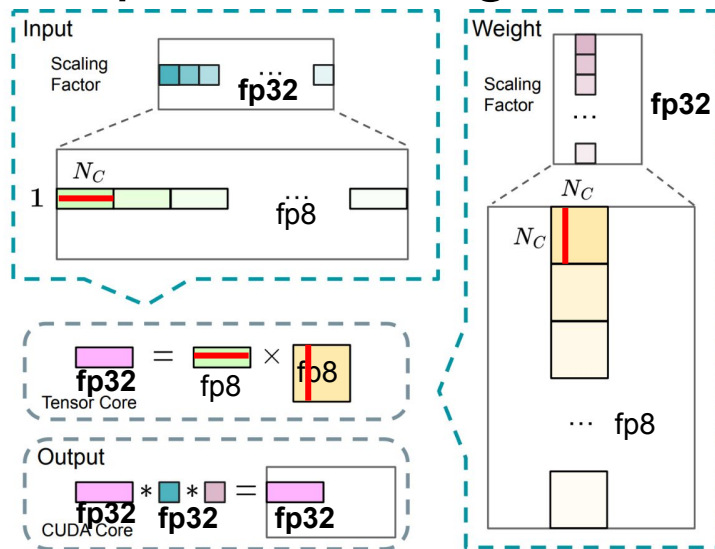




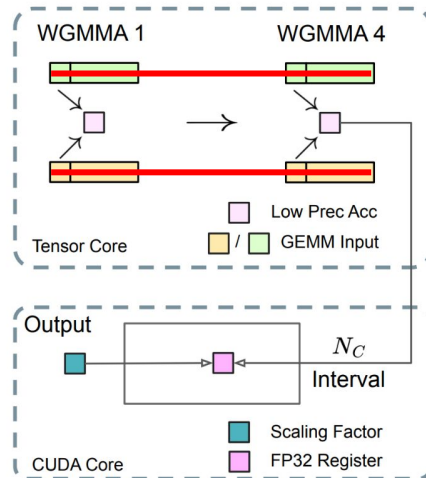
DeepSeek: fine-grained fp8 quantization



Scaling Factor fp32

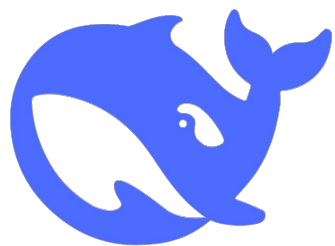


(a) Fine-grained quantization



(b) Increasing accumulation precision

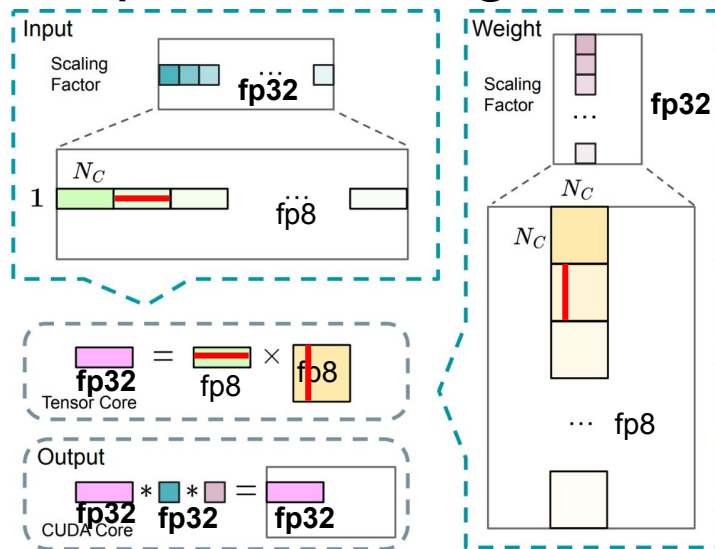
Figure 7 | (a) We propose a fine-grained quantization method to mitigate quantization errors caused by feature outliers; for illustration simplicity, only Fprop is illustrated. (b) In conjunction with our quantization strategy, we improve the FP8 GEMM precision by promoting to CUDA Cores at an interval of $N_C = 128$ elements MMA for the high-precision accumulation.



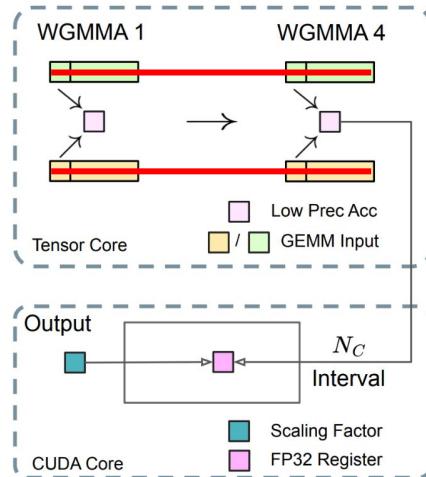
DeepSeek: fine-grained fp8 quantization



Scaling Factor fp32

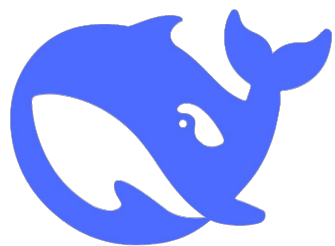


(a) Fine-grained quantization



(b) Increasing accumulation precision

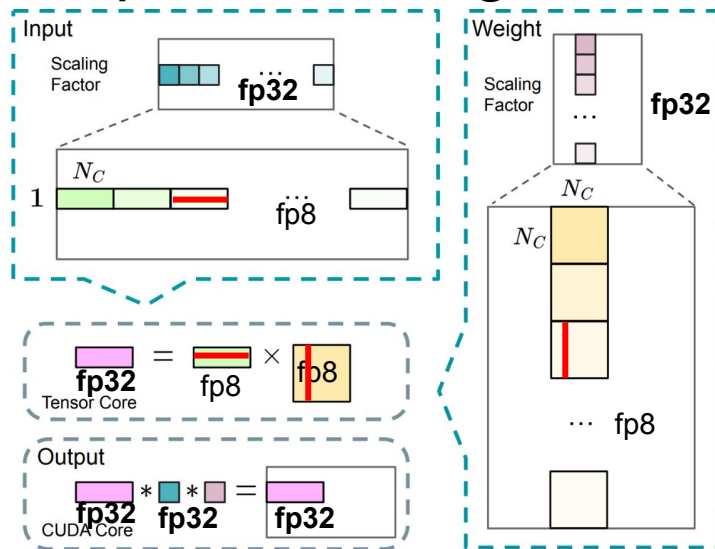
Figure 7 | (a) We propose a fine-grained quantization method to mitigate quantization errors caused by feature outliers; for illustration simplicity, only Fprop is illustrated. (b) In conjunction with our quantization strategy, we improve the FP8 GEMM precision by promoting to CUDA Cores at an interval of $N_C = 128$ elements MMA for the high-precision accumulation.



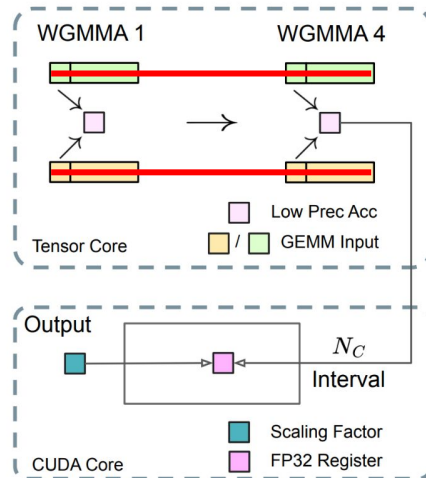
DeepSeek: fine-grained fp8 quantization



Scaling Factor fp32

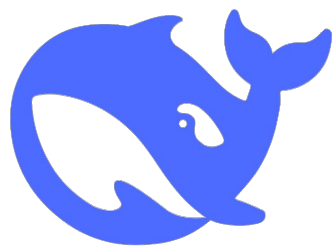


(a) Fine-grained quantization



(b) Increasing accumulation precision

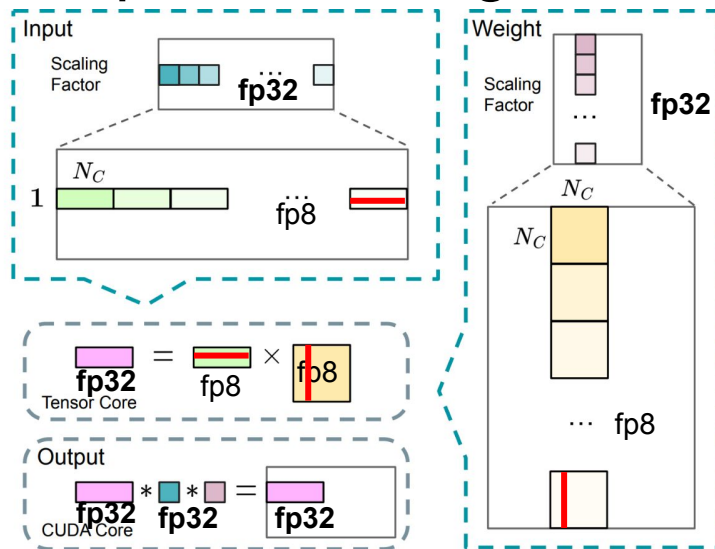
Figure 7 | (a) We propose a fine-grained quantization method to mitigate quantization errors caused by feature outliers; for illustration simplicity, only Fprop is illustrated. (b) In conjunction with our quantization strategy, we improve the FP8 GEMM precision by promoting to CUDA Cores at an interval of $N_C = 128$ elements MMA for the high-precision accumulation.



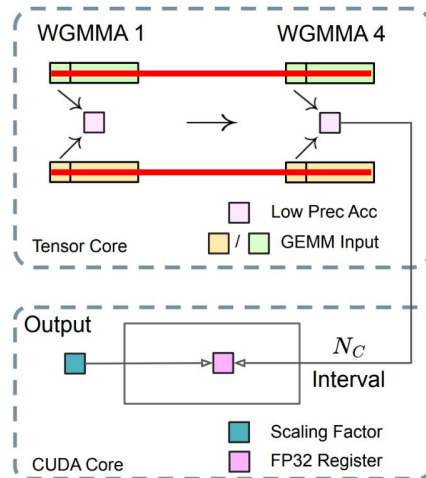
DeepSeek: fine-grained fp8 quantization



Scaling Factor fp32

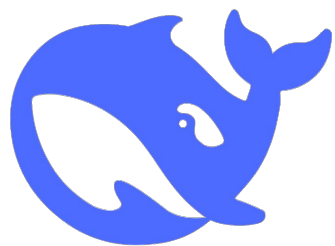


(a) Fine-grained quantization



(b) Increasing accumulation precision

Figure 7 | (a) We propose a fine-grained quantization method to mitigate quantization errors caused by feature outliers; for illustration simplicity, only Fprop is illustrated. (b) In conjunction with our quantization strategy, we improve the FP8 GEMM precision by promoting to CUDA Cores at an interval of $N_C = 128$ elements MMA for the high-precision accumulation.



DeepSeek: fine-grained fp8 quantization

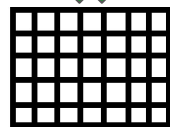


fp32

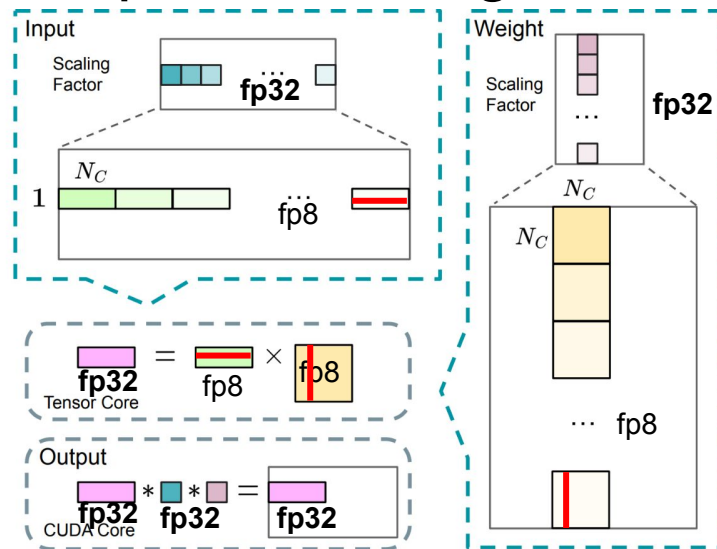


Scaling Factor

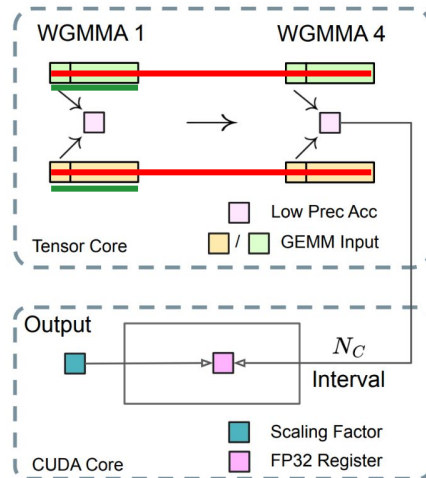
fp32



fp8



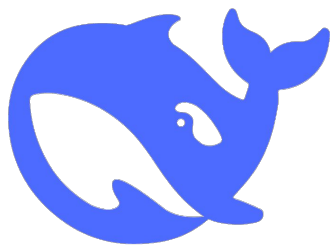
(a) Fine-grained quantization



(b) Increasing accumulation precision

Зачем 4xWGMMMA?
Почему не 1xWGMMMA?

Figure 7 | (a) We propose a fine-grained quantization method to mitigate quantization errors caused by feature outliers; for illustration simplicity, only Fprop is illustrated. (b) In conjunction with our quantization strategy, we improve the FP8 GEMM precision by promoting to CUDA Cores at an interval of $N_C = 128$ elements MMA for the high-precision accumulation.



DeepSeek: fine-grained fp8 quantization

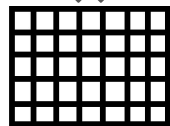


fp32

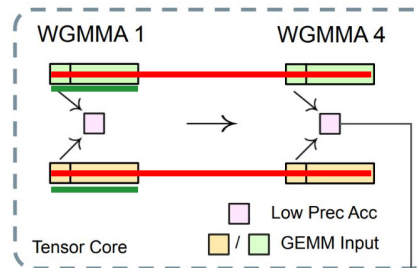


Scaling Factor

fp32



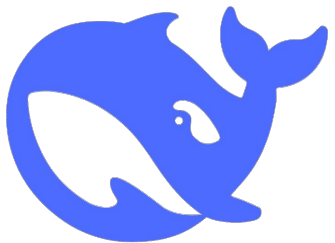
fp8



Зачем 4xWGMMAs?

Почему не 1xWGMMAs?

It is worth noting that this modification reduces the WGMMAs (Warpgroup-level Matrix Multiply-Accumulate) instruction issue rate for a single warpgroup. However, on the H800 architecture, it is typical for two WGMMAs to persist concurrently: while one warpgroup performs the promotion operation, the other is able to execute the MMA operation. This design enables overlapping of the two operations, maintaining high utilization of Tensor Cores. Based on our experiments, setting $N_C = 128$ elements, equivalent to 4 WGMMAs, represents the minimal accumulation interval that can significantly improve precision without introducing substantial overhead.



DeepSeek: fine-grained fp8 quantization

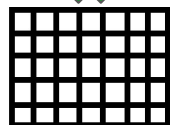


fp32

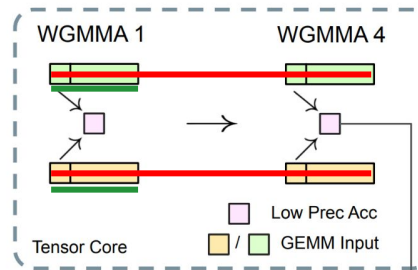


Scaling Factor

fp32



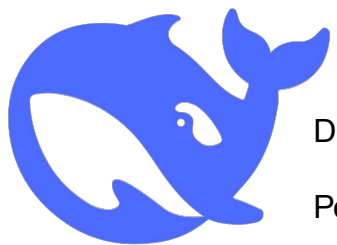
fp8



Зачем 4xWGMMMA?
Почему не 1xWGMMMA?

It is worth noting that this modification reduces the WGMMMA (Warpgroup-level Matrix Multiply-Accumulate) instruction issue rate for a single warpgroup. However, on the H800 architecture, it is typical for two WGMMMA to persist concurrently: while one warpgroup performs the promotion operation, the other is able to execute the MMA operation. This design enables overlapping of the two operations, maintaining high utilization of Tensor Cores. Based on our experiments, setting $N_C = 128$ elements, equivalent to 4 WGMMAs, represents the minimal accumulation interval that can significantly improve precision without introducing substantial overhead.

Не знаю почему нужно делать 4xWGMMMA + PROMOTION
вместо 4x(WGMMMA + PROMOTION)



DeepSeek: x2 ускорение обучения (fp16 → fp8)

DeepSeek-V3 Technical Report - <https://arxiv.org/abs/2412.19437>

Репозиторий - <https://github.com/deepseek-ai/DeepGEMM> (GEMM - General Matrix Multiplications)

[Fast Matrix-Multiplication with WGMMMA on NVIDIA® Hopper™ GPUs](#)

https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html



Глава 5: Умножение матриц

Метод Штрассена

Умножение матриц - Метод Штрассена

$$\begin{array}{c} \left[\begin{array}{c|c} a & b \\ \hline c & d \end{array} \right] \times \left[\begin{array}{c|c} e & f \\ \hline g & h \end{array} \right] = \left[\begin{array}{c|c} ae + bg & af + bh \\ \hline ce + dg & cf + dh \end{array} \right] \\ A \qquad \qquad B \qquad \qquad \qquad C \end{array}$$

Умножение матриц - Метод Штрассена

~~$$\begin{array}{c}
 \left[\begin{array}{cc|cc} a & b & e & f \\ c & d & g & h \end{array} \right] \times \left[\begin{array}{cc|cc} e & f & g & h \end{array} \right] = \left[\begin{array}{cc|cc} ae + bg & af + bh & ce + dg & cf + dh \end{array} \right] \\
 A \qquad B \qquad C
 \end{array}$$~~

$$\begin{aligned}
 p1 &= a(f - h) \\
 p3 &= (c + d)e \\
 p5 &= (a + d)(e + h) \\
 p7 &= (a - c)(e + f)
 \end{aligned}$$

$$\begin{aligned}
 p2 &= (a + b)h \\
 p4 &= d(g - e) \\
 p6 &= (b - d)(g + h)
 \end{aligned}$$

$$\begin{array}{c}
 \left[\begin{array}{cc|cc} a & b & e & f \\ c & d & g & h \end{array} \right] \times \left[\begin{array}{cc|cc} e & f & g & h \end{array} \right] = \left[\begin{array}{cc|cc} p5 + p4 - p2 + p6 & p1 + p2 & p3 + p4 & p1 + p5 - p3 - p7 \end{array} \right] \\
 A \qquad B \qquad C
 \end{array}$$

Умножение матриц - Метод Штрассена

~~$$\begin{matrix}
 \begin{bmatrix} a & b \\ c & d \end{bmatrix} \\
 A
 \end{matrix}
 \times
 \begin{matrix}
 \begin{bmatrix} e & f \\ g & h \end{bmatrix} \\
 B
 \end{matrix}
 =
 \begin{matrix}
 \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix} \\
 C
 \end{matrix}$$~~

$$\begin{aligned}
 p1 &= a(f - h) \\
 p3 &= (c + d)e \\
 p5 &= (a + d)(e + h) \\
 p7 &= (a - c)(e + f)
 \end{aligned}$$

$$\begin{aligned}
 p2 &= (a + b)h \\
 p4 &= d(g - e) \\
 p6 &= (b - d)(g + h)
 \end{aligned}$$

$$\begin{matrix}
 \begin{bmatrix} a & b \\ c & d \end{bmatrix} \\
 A
 \end{matrix}
 \times
 \begin{matrix}
 \begin{bmatrix} e & f \\ g & h \end{bmatrix} \\
 B
 \end{matrix}
 =
 \begin{matrix}
 \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix} \\
 C
 \end{matrix}$$

Получили рекуррентную асимптотику $T(N) = 7T(N/2) + 8O(N^2) \Rightarrow$
 $T(N) = O(N^{\log 7}) = O(N^{2.8})$

Умножение матриц - Метод Штрассена

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

A B C

$$\begin{aligned} p1 &= a(f - h) \\ p3 &= (c + d)e \\ p5 &= (a + d)(e + h) \\ p7 &= (a - c)(e + f) \end{aligned}$$

$$\begin{aligned} p2 &= (a + b)h \\ p4 &= d(g - e) \\ p6 &= (b - d)(g + h) \end{aligned}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

A B C

Получили рекуррентную асимптотику $T(N) = 7T(N/2) + 8O(N^2) \Rightarrow$
 $T(N) = O(N^{\log 7}) = O(N^{2.8})$

Метод Виноградова - тоже $O(N^{2.8})$

<https://www.geeksforgeeks.org/strassens-matrix-multiplication/>

Глава 6: PyTorch и Манифест коммунистической партии



PyTorch

PyTorch is a Python package that provides two high-level features:

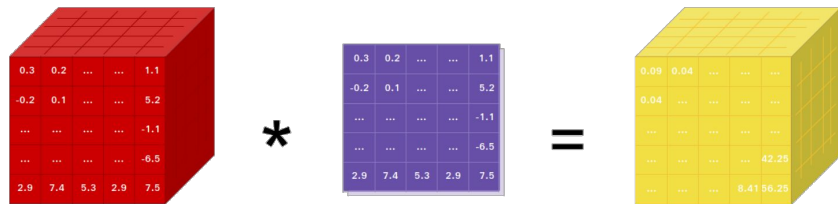
- Tensor computation (like NumPy) with strong GPU acceleration
- Deep neural networks built on a tape-based autograd system

You can reuse your favorite Python packages such as NumPy, SciPy, and Cython to extend PyTorch when needed.

...

A GPU-Ready Tensor Library

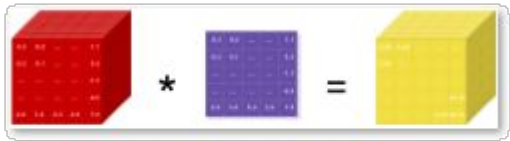
If you use NumPy, then you have used Tensors (a.k.a. ndarray).



PyTorch provides Tensors that can live either on the CPU or the GPU and accelerates the computation by a huge amount.

We provide a wide variety of tensor routines to accelerate and fit your scientific computation needs such as slicing, indexing, mathematical operations, linear algebra, reductions. And they are fast!

<https://github.com/pytorch/pytorch>



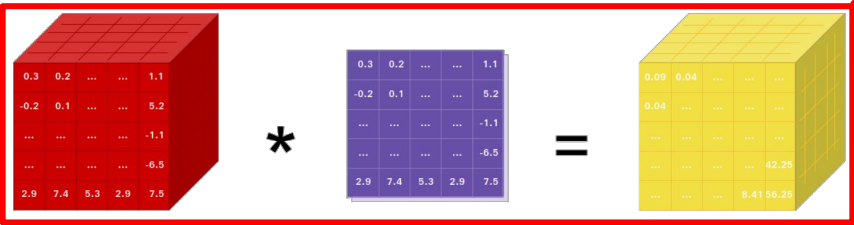
■ ■ ■

- Tensor computation (like NumPy) with strong GPU acceleration
- Deep neural networks built on a tape-based autograd system

You can reuse your favorite Python packages such as NumPy, SciPy, and Cython to extend PyTorch when needed.

A GPU-Ready Tensor Library

If you use NumPy, then you have used Tensors (a.k.a. ndarray).



PyTorch provides Tensors that can live either on the CPU or the GPU and accelerates the computation by a huge amount.

We provide a wide variety of tensor routines to accelerate and fit your scientific computation needs such as slicing, indexing, mathematical operations, linear algebra, reductions. And they are fast!

<https://github.com/pytorch/pytorch>

PyTorch

PyTorch is a Python package that provides two high-level features:

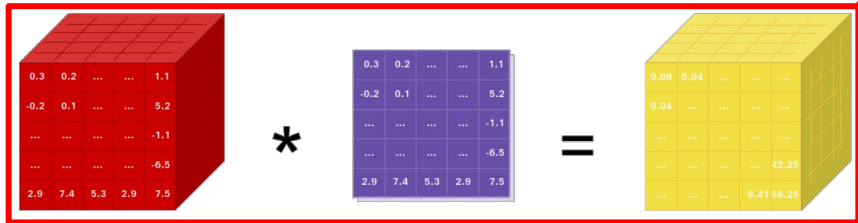
- Tensor computation (like NumPy) with strong GPU acceleration
- Deep neural networks built on a tape-based autograd system

You can reuse your favorite Python packages such as NumPy, SciPy, and Cython to extend PyTorch when needed.

...

A GPU-Ready Tensor Library

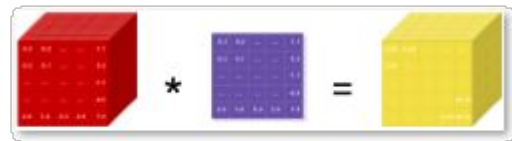
If you use NumPy, then you have used Tensors (a.k.a. ndarray).



PyTorch provides Tensors that can live either on the CPU or the GPU and accelerates the computation by a huge amount.

We provide a wide variety of tensor routines to accelerate and fit your scientific computation needs such as slicing, indexing, mathematical operations, linear algebra, reductions. And they are fast!

<https://github.com/pytorch/pytorch>



tensor_illustration.png

<https://pngchunk.com/>

PyTorch

PyTorch is a Python package that provides two high-level features:

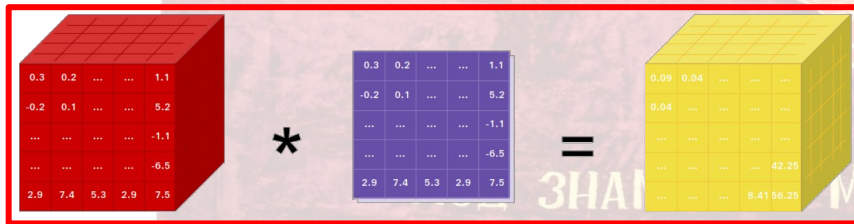
- Tensor computation (like NumPy) with strong GPU acceleration
- Deep neural networks built on a tape-based autograd system

You can reuse your favorite Python packages such as NumPy, SciPy, and Cython to extend PyTorch when needed.

...

A GPU-Ready Tensor Library

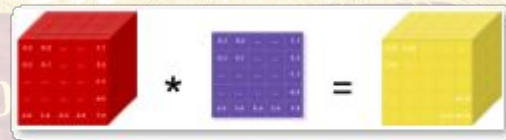
If you use NumPy, then you have used Tensors (a.k.a. ndarray).



PyTorch provides Tensors that can live either on the CPU or the GPU and accelerates the computation by a huge amount.

We provide a wide variety of tensor routines to accelerate and fit your scientific computation needs such as slicing, indexing, mathematical operations, linear algebra, reductions. And they are fast!

<https://github.com/pytorch/pytorch>



tensor_illustration.png

<https://pngchunk.com/>

Chunk inspector for prompts

Easy tool to get PNG info, prompts, generation and model information of generated stable diffusion

Some tools add information about the parameters used for generating the image inside the png chunk. With PNGCHUNK you can load the image or image URL and within seconds you get the details of the generation. You don't need to start up a server.

Upload

- Only .png image files
- Filesize max 25MB
- 1 file at a time

Result

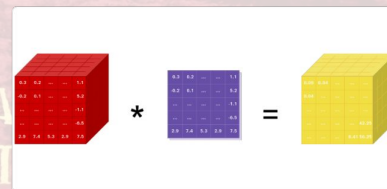
Prompt

Negative Prompt

Sampler

CFG scale

Size



RAW

```
{
  "Comment": "Пролетарии всех стран, соединяйтесь!"
}
```


LIVE

BREAKING NEWS

**RUSSIAN HACKERS HACK PYTORCH, CAUSE
\$239,000,000.000.000 DAMAGE TO ENTIRE GALAXY**

LIVE



PyTorch

VULNERABILITY

BREAKING NEWS

**RUSSIAN HACKERS HACK PYTORCH, CAUSE
\$239,000,000.000.000 DAMAGE TO ENTIRE GALAXY**

LIVE

BREAKING NEWS

**RUSSIAN HACKERS HACK PYTORCH, CAUSE
\$239,000,000.000.000 DAMAGE TO ENTIRE GALAXY**



CS Space

Клуб технологий и науки



ВОПРОСЫ?



[@UnicornGlade](#)



[@PolarNick239](#)



polarnick239@gmail.com



Николай Полярный